

Contents

Base64 函数参考	6
函数列表	6
使用示例	6
错误处理	7
字符串操作函数参考	7
函数列表	7
contains	8
starts_with	9
ends_with	9
find	9
substring	9
char_at	9
replace	10
to_upper	10
to_lower	10
trim	10
trim_start	10
trim_end	11
repeat	11
reverse	11
byte_len	11
split	11
join	12
to_int	12
to_float	12
to_str	13
内置函数参考	14
函数一览	14
print	17
Some	18
length	18
has_key	18
add	19
remove	19
range	19
exit	20
arguments	20
sleep	20
env	20
append	21
pop	21
reverse (列表)	22
slice	22
take	22
tap	22
filter	22
map	23
sort	23
sort!	23
reverse!	23
appended	24

append!	24
first	24
last	24
get (映射)	24
iter	24
next	25
to_list	25
type_of	25
集合参考 (元组、列表、映射、集合)	27
元组	27
列表	27
Copy-on-Write (CoW) 语义	35
映射	36
集合	37
迭代器	40
契约式设计 (Design by Contract)	41
前置条件 (require)	41
后置条件 (ensure)	41
组合示例	42
结构体不变量 (invariant)	42
规则	42
控制流参考	43
if / else	43
while	44
for	44
async / await	46
@parallel for	47
break	47
continue	48
... (Ellipsis)	48
case	48
case 带主题 (模式匹配)	49
作用域规则	52
指令	53
语法	53
支持的目标	53
内建指令	53
注意事项	59
错误参考	60
错误格式	60
编译错误	60
运行时错误	61
文件系统函数参考	62
函数一览	62
示例	62
注意事项	64
函数参考	65
函数定义语法	65

参数与返回值的类型	65
嵌套函数	66
递归	67
重载	68
默认参数	69
Unit 类型函数	70
Task 与异步函数	70
Lambda 函数	70
闭包	71
函数类型	72
高阶函数	73
泛型函数	73
UFCS (统一函数调用语法)	75
运算符重载	75
检查/饱和算术	76
GC 参考	77
概述	77
导入	77
函数	77
工作原理	78
静态分析优化	78
示例	78
HTTP 参考	78
类型	78
函数 (来自 http)	79
使用示例	80
行为	82
支持的状态码	83
HTTP 客户端	84
错误处理	85
I/O 函数参考手册	85
函数列表	85
使用示例	86
错误处理	87
备注	87
JSON 函数参考	87
概述	88
函数列表	88
解开 Result<JsonValue, Error>	89
使用示例	89
注意事项	90
數學函式 (math)	90
概述	90
常數	90
絕對值	91
捨入	91
幕次與平方根	91
對數	92
指數	92
三角函式	92

反三角函数	93
雙参数函数	93
判定函数	93
网络 (TCP) 参考手册	93
类型	93
函数 (来自 <code>net</code>)	93
内建重载函数	94
使用示例	95
超时配置	96
错误处理	97
字节转换	97
运算符参考	97
优先级表	97
算术运算符	97
比较运算符	98
逻辑运算符	98
位运算符	98
错误传播运算符 (<code>? / !!</code>)	99
<code>case</code> : 条件表达式	100
范围运算符	100
空值合并运算符 (<code>??</code>)	100
复合赋值运算符	101
递增 / 递减运算符	102
运算的类型规则	102
运算符重载	102
包参考	106
概述	106
导入语法	106
包解析	107
标准库 (<code>std</code>)	107
搜索路径优先级	109
<code>RY_PATH</code> 环境变量	109
约束	109
创建包文件	109
原生函数命名约定	110
路径函数参考	110
函数一览	111
示例	111
项目管理	112
CLI 概述	112
<code>ry init</code> - 项目初始化	113
<code>ry new</code> - 创建新项目	113
<code>ry run</code> - 运行项目脚本	114
<code>ry fmt</code> - 代码格式化工具	114
<code>ry test</code> - 运行测试	115
<code>ry self-update</code> - 自我更新	115
<code>package.toml</code> 配置文件	116
正则表达式参考	117
正则表达式字面量语法	117

函数一览	117
支持的模式语法	118
使用示例	119
结构体参考	120
概述	120
定义语法	120
构造函数	120
字段访问	120
字段赋值	120
作为函数参数与返回值使用	121
嵌套结构体	121
比较 (== / !=)	122
记录子类型 (继承)	122
约束与错误	123
枚举类型 (enum)	123
测试功能	125
运行测试	125
语法	126
输出格式	128
示例	128
模拟 (Mock)	129
参数化测试 (@each)	130
基于属性的测试 (@property)	130
测试覆盖率	130
测试大纲	131
测试描述风格	131
限制	132
线程参考	132
类型	132
导入	132
Thread	132
Lock (互斥锁)	133
RWLock (读写锁)	134
Semaphore	134
Barrier	134
AtomicInt	134
AtomicBool	135
与 async/await 的比较	135
类型参考	135
类型一览	135
类型标注语法	137
可用类型名称一览	137
类型别名	138
数值字面量	139
字面量类型	139
范围类型	140
none 关键字与 Option 类型简写	140
弱引用 (weak T)	141
F-String (字符串插值)	142
类型转换 (as)	142
带关联数据的 enum (ADT)	143

泛型 enum	144
Error 类型	144
Type	145
union 类型	146
any 类型	146
类型规则（运算时的类型转换）	148
类型安全约束	150

[English](#) | [日本語](#) | [繁體中文](#)

Base64 函数参考

Base64 编码与解码。所有函数需要从 base64 明确导入。

```
from base64 import encode, decode, encode_url_safe, decode_url_safe
```

函数列表

函数	签名	说明
encode	(str) -> str	将字符串编码为标准 base64
decode	(str) -> Result<str, Error>	解码标准 base64 字符串
encode_url_safe	(str) -> str	将字符串编码为 URL-safe base64 (无填充)
decode_url_safe	(str) -> Result<str, Error>	解码 URL-safe base64 字符串

使用示例

基本编码与解码

```
from base64 import encode, decode

encoded = encode("Hello, World!")
print(encoded)  # SGVsbG8sIFdvcmxkIQ==

case decode(encoded):
    Ok(s):
        print(s)  # Hello, World!
    Err(e):
        print(e.message)
```

URL-safe Base64

URL-safe base64 使用 - 和 _ 取代 + 和 /，并省略填充 (=)。适用于 URL、文件名和令牌。

```
from base64 import encode_url_safe, decode_url_safe

encoded = encode_url_safe("data with special chars: ?&=")
# No + / or = in the output

case decode_url_safe(encoded):
    Ok(s):
        print(s)
    Err(e):
        print(e.message)
```

处理字节数据

要编码/解码字节数据，请结合 `io` 中的 `to_bytes` / `bytes_to_str` 使用。

```
from base64 import encode, decode
from io import to_bytes, bytes_to_str

bytes = to_bytes("binary data")
encoded = encode(bytes_to_str(bytes))
```

错误处理

`decode` 和 `decode_url_safe` 返回 `Result<str, Error>`。当输入包含无效的 base64 字符时，解码会失败。

```
case decode("!!!not-valid!!!"):
  Ok(s):
    print(s)
  Err(e):
    print(e.message) # "invalid base64 character at position 0"
```

使用 ? 运算符:

```
function process(input: str) -> Result<str, Error>:
  decoded = decode(input)?
  return Ok(decoded)
```

[English](#) | [日本語](#) | [简体中文](#)

字符串操作函数参考

针对字符串 (`str`) 的操作函数一览。所有函数均可使用 UFCS 记法。

注意：所有字符串操作均支持 UTF-8。length()、char_at()、substring()、find() 和 reverse() 以 Unicode 码位为单位操作，而非字节。如需获取字节长度，请使用 `byte_len()`。

函数列表

搜索与判定

函数	签名	说明
contains	(str, str, bool = false) -> bool	返回是否包含子字符串
starts_with	(str, str, bool = false) -> bool	返回是否以前缀开头
ends_with	(str, str, bool = false) -> bool	返回是否以后缀结尾
find	(str, str) -> Option<int>	返回子字符串的字符位置 (未找到为 None)

提取与转换

函数	签名	说明
substring	(str, int, int) -> str	提取子字符串 (字符索引)
char_at	(str, int) -> str	获取指定位置的 UTF-8 字符
replace	(str, str, str) -> str	替换所有匹配的子字符串

大小写转换

函数	签名	说明
to_upper	str -> str	转换为 ASCII 大写
to_lower	str -> str	转换为 ASCII 小写

去除空白

函数	签名	说明
trim	str -> str	去除前后的空白
trim_start	str -> str	去除开头的空白
trim_end	str -> str	去除结尾的空白

生成与加工

函数	签名	说明
repeat	(str, int) -> str	将字符串重复 n 次
reverse	str -> str	反转字符串 (UTF-8 感知)
byte_len	str -> int	返回字符串的字节长度

分割与连接

函数	签名	说明
split	(str, str) -> List<str>	以分隔符分割
join	(List<str>, str) -> str	以分隔符连接

类型转换

函数	签名	说明
to_int	str -> Result<int, Error>	将字符串转换为整数
to_float	str -> Result<float, Error>	将字符串转换为浮点数
to_str	int/float/bool/str/enum/record -> str	将值转换为字符串

contains

签名: contains(string: str, substring: str, ignore_case: bool = false) -> bool

返回字符串 string 中是否包含子字符串 substring。当 ignore_case 为 true 时, 比较不区分大小写 (仅 ASCII)。

```
print(contains("hello", "ell"))           # true
print("hello".contains("xyz"))           # false (UFCS)
print(contains("Hello World", "hello", true)) # true (不区分大小写)
```


starts_with

签名: `starts_with(string: str, prefix: str, ignore_case: bool = false) -> bool`

返回字符串 `string` 是否以 `prefix` 开头。当 `ignore_case` 为 `true` 时, 比较不区分大小写 (仅 ASCII)。

```
print(starts_with("hello", "hel"))           # true
print("hello".starts_with("world"))          # false (UFCS)
print(starts_with("Hello", "hello", true))   # true (不区分大小写)
```

ends_with

签名: `ends_with(string: str, suffix: str, ignore_case: bool = false) -> bool`

返回字符串 `string` 是否以 `suffix` 结尾。当 `ignore_case` 为 `true` 时, 比较不区分大小写 (仅 ASCII)。

```
print(ends_with("hello", "llo"))             # true
print("hello".ends_with("world"))            # false (UFCS)
print(ends_with("Hello World", "WORLD", true)) # true (不区分大小写)
```

find

签名: `find(string: str, substring: str) -> Option<int>`

返回字符串 `string` 中子字符串 `substring` 首次出现的字符位置。未找到时返回 `None`。

```
print(find("hello world", "world"))          # Some(6)
print(find("hello", "xyz"))                  # None
print("abcdef".find("cd"))                   # Some(2) (UFCS)
```

substring

签名: `substring(string: str, start: int, end: int) -> str`

返回字符串 `string` 从 `start` 到 `end` (不含) 的子字符串。索引为字符位置 (UTF-8 感知)。

超出范围的索引会被钳制到 `[0, length]`。钳制后若 `end < start`, 返回空字符串。

```
print(substring("hello world", 0, 5))        # hello
print(substring("hello world", 6, 11))       # world
print("abcdef".substring(1, 4))              # bcd (UFCS)
print(substring("hello", -1, 100))           # hello (钳制)
```

char_at

签名: `char_at(string: str, i: int) -> str`

返回字符串 `string` 第 `i` 个位置的 UTF-8 字符作为字符串。索引超出范围时产生运行时错误。

负数索引从末尾开始计算 (Python 风格): `-1` 指最后一个字符, `-2` 指倒数第二个, 以此类推。

```
print(char_at("hello", 0))                   # h
print(char_at("hello", -1))                  # o (最后一个字符)
print("abc".char_at(2))                      # c (UFCS)
```

replace

签名: `replace(string: str, old: str, new: str) -> str`

返回将字符串 `string` 中所有 `old` 替换为 `new` 后的新字符串。

如果 `old` 是空字符串, 则原样返回输入 (作为新副本)。

```
print(replace("hello world", "world", "ry"))    # hello ry
print(replace("aaa", "a", "bb"))                # bbbbbb
print("foo bar foo".replace("foo", "baz"))       # baz bar baz (UFCS)
print(replace("hello", "", "X"))                 # hello (空模式为 no-op)
```

to_upper

签名: `to_upper(string: str) -> str`

返回将 ASCII 小写字母 (a-z) 转换为大写后的新字符串。

```
print(to_upper("hello"))                        # HELLO
print("Hello World".to_upper())                 # HELLO WORLD (UFCS)
```

to_lower

签名: `to_lower(string: str) -> str`

返回将 ASCII 大写字母 (A-Z) 转换为小写后的新字符串。

```
print(to_lower("HELLO"))                       # hello
print("Hello World".to_lower())                 # hello world (UFCS)
```

trim

签名: `trim(string: str) -> str`

返回去除字符串前后空白字符 (空格、制表符、换行、回车) 后的新字符串。

```
print(trim(" hello "))                         # hello
print(" hi ".trim())                           # hi (UFCS)
```

trim_start

签名: `trim_start(string: str) -> str`

返回去除字符串开头空白字符后的新字符串。

```
print(trim_start(" hello "))                   # hello
print(" hi ".trim_start())                     # hi (UFCS)
```

trim_end

签名: `trim_end(string: str) -> str`

返回去除字符串结尾空白字符后的新字符串。

```
print(trim_end(" hello "))    # hello
print("hi ".trim_end())       # hi (UFCS)
```

repeat

签名: `repeat(string: str, count: int) -> str`

返回将字符串 `string` 重复 `count` 次后的新字符串。

```
print(repeat("ab", 3))        # ababab
print("ha".repeat(3))         # hahaha (UFCS)
```

reverse

签名: `reverse(string: str) -> str`

返回字符顺序反转后的新字符串 (UTF-8 感知)。

```
print(reverse("hello"))       # olleh
print("abc".reverse())        # cba (UFCS)
```

byte_len

签名: `byte_len(string: str) -> int`

返回字符串 `string` 的字节长度。与返回 UTF-8 字符数的 `length()` 不同, `byte_len()` 返回的是字节数。

```
print(byte_len("hello"))      # 5
print(byte_len("あいう"))     # 9
print(length("あいう"))       # 3 (characters)
```

split

签名: `split(string: str, delimiter: str) -> List<str>`

以分隔符 `delimiter` 分割字符串 `string`, 返回 `List<str>`。

当分隔符为空字符串 "" 时, 字符串会被分割为单个字符 (UTF-8 感知)。

```
parts = split("a,b,c", ",")
print(parts[0])    # a
print(parts[1])    # b
print(parts[2])    # c

for word in "hello world".split(" "):
    print(word)
# hello
# world
```

```
# 分割为单个字符
chars = split("hello", "")
print(chars)    # [h, e, l, l, o]
```

```
# UTF-8 字符
chars = split("あいう", "")
print(chars)    # [あ, い, う]
```

提示：要逐字符迭代字符串，您可以直接使用 `for` 循环而无需调用 `split`：`for c in s:` 会作为单字符 `str` 产生每个 UTF-8 码位。请参阅 [control-flow.md](#)。

join

签名：`join(values: List<str>, sep: str) -> str`

以分隔符 `sep` 连接字符串列表的元素，返回合并后的字符串。

```
parts = ["a", "b", "c"]
print(join(parts, ","))      # a,b,c
print(parts.join("-"))      # a-b-c (UFCS)
print(",".join(parts))      # a,b,c (UFCS, Python 风格)
```

to_int

签名：`to_int(string: str) -> Result<int, Error>`

将字符串转换为整数。允许前导空白。字符串为空、包含无效字符或溢出时返回 `Err`。

```
case to_int("42"):
  Ok(v):
    print(v)                # 42
  Err(e):
    print(e.message)

case "123".to_int():
  Ok(v):
    print(v)                # 123
  Err(e):
    print(e.message)

# 无效输入返回 Err
print(to_int("abc"))        # Err(Error("to_int: invalid character in 'abc'"))
print(to_int(""))          # Err(Error("to_int: empty string"))
```

to_float

签名：`to_float(string: str) -> Result<float, Error>`

将字符串转换为浮点数。字符串为空、包含无效字符或超出 `float` 范围时返回 `Err`。

```
case to_float("3.14"):
  Ok(v):
    print(v)                # 3.14
  Err(e):
```

```

        print(e.message)

case "2.5".to_float():
    Ok(v):
        print(v)                # 2.5
    Err(e):
        print(e.message)

# 无效输入返回 Err
print(to_float("abc"))          # Err(Error("to_float: invalid character in 'abc'"))
print(to_float(""))            # Err(Error("to_float: empty string"))
print(to_float("1e400"))        # Err(Error("to_float: out of range in '1e400'"))

```

to_str

签名: `to_str(v: int | float | bool | str | enum | record) -> str`

将值转换为字符串。

类型	转换格式
int	%ld
float	%g, 整数值会附带 .0 (例如 "3.0"、"0.0")
bool	"true" / "false"
str	直接返回
enum	变体名称 (例如 "Red")
record	TypeName(field1: val1, field2: val2)
List / Map / Set	递归格式化, 支持嵌套容器 (例如 Map<str, List<int>>)
union	格式化为活动变体; 支持 List、Map、Set 和 function 变体
function 值 (closure / lambda)	"<closure>"

整数值的 float 会附带 .0 格式化 (例如 `to_str(3.0) == "3.0"`), 以便与 int 视觉上区分。Record 类型自动生成 `to_str` 表示。如果提供了用户定义的 `function to_str(v: MyRecord) -> str`, 则优先使用用户定义版本。这也适用于 `print()` 和 f-string 插值。

```

print(to_str(42))              # 42
print(to_str(3.14))            # 3.14
print(to_str(3.0))             # 3.0          (整数值 float 保留 .0)
print(to_str(true))            # true
print(99.to_str())             # 99 (UFCS)

```

```
enum Color:
```

```
    Red
    Green
```

```
print(to_str(Color::Red))      # Red
```

```
record Point:
```

```
    x: int
    y: int
```

```

p = Point(10, 20)
print(to_str(p))              # Point(x: 10, y: 20)

```

```
print(p)                # Point(x: 10, y: 20)
print(f"pos={p}")       # pos=Point(x: 10, y: 20)
```

[English](#) | [日本語](#) | [简体中文](#)

内置函数参考

函数一览

核心

函数	说明
<code>print()</code> / <code>print(expr1, expr2, ...)</code> <code>length(value)</code>	将值输出到标准输出（以空格分隔） 返回列表、映射、集合的元素数量，或字符串的 UTF-8 字符数
<code>range(n)</code> / <code>range(start, end)</code> / <code>range(start, end, step)</code>	生成整数列表
<code>exit(code)</code>	以指定的退出码终止进程
<code>arguments()</code>	以 <code>List<str></code> 返回命令行参数
<code>available_parallelism()</code>	返回运行时工作线程数 (<code>int</code>)
<code>sleep(duration_ms)</code>	暂停执行指定的毫秒数
<code>env(key)</code>	返回环境变量为 <code>Option<str></code>
<code>env(key, default)</code>	返回环境变量，若未设置则返回 <code>default</code>
<code>send(stream, data)</code>	通过 <code>TcpStream</code> 或 <code>TlsStream</code> 发送 <code>List<u8></code> ，返回 <code>Result<int, Error></code>
<code>receive(stream, max)</code>	从 <code>TcpStream</code> 或 <code>TlsStream</code> 接收最多 <code>max</code> 字节，返回 <code>Result<List<u8>, Error></code>
<code>close(handle)</code>	关闭 <code>TcpStream</code> 、 <code>TlsStream</code> 或 <code>TcpListener</code>
<code>block_on(task)</code>	阻塞当前线程直到 <code>Task<T></code> 完成并返回其结果
<code>to_str(value)</code>	将值转换为其字符串表示。支持 <code>int</code> 、 <code>float</code> （整数值会附带 <code>.0</code> 输出）、 <code>bool</code> 、 <code>str</code> 、 <code>record</code> 、 <code>enum</code> 、 <code>tuple</code> 、 <code>List</code> 、 <code>Map</code> 、 <code>Set</code> （嵌套容器如 <code>Map<str, List<int>></code> 会递归格式化）、 <code>Result</code> 、 <code>Option</code> 、 <code>union</code> 类型（格式化为活动变体）以及 <code>function</code> 值（输出为 <code><closure></code> ）。集合内的字符串元素会用双引号包裹（例如 <code>["hello", "world"]</code> ）
<code>type_of(expr)</code>	返回 <code>expr</code> 的类型作为 <code>Type</code> 值。请参阅 type_of
<code>fail()</code> / <code>fail(message)</code>	将当前测试标记为失败（仅在 <code>ry test</code> 模式下可用）

Option

函数	说明
<code>Some(expr)</code>	构造 <code>Option</code> 类型的有值变体

Result / Error

函数	说明
<code>Ok(value)</code>	构造 <code>Result<T, Error></code> 的成功变体
<code>Err(error)</code>	构造 <code>Result<T, Error></code> 的错误变体
<code>Error(message)</code>	创建带有消息的 <code>Error</code> 值

函数	说明
<code>Error(message, code)</code>	创建带有消息和错误码的 <code>Error</code> 值
<code>result.and_then(closure)</code>	若为 <code>Ok</code> ，调用 <code>closure</code> （返回 <code>Result<U, E></code> ）；若为 <code>Err</code> ，传播错误
<code>result.map(closure)</code>	若为 <code>Ok</code> ，对值应用 <code>closure</code> 并将返回值包装在 <code>Ok</code> 中；若为 <code>Err</code> ，传播错误

检查算术

函数	说明
<code>checked_add(a, b)</code>	无溢出时返回 <code>Ok(a + b)</code> ，否则返回 <code>Err(Error("arithmetic overflow"))</code>
<code>checked_sub(a, b)</code>	无溢出时返回 <code>Ok(a - b)</code> ，否则返回 <code>Err(Error("arithmetic overflow"))</code>
<code>checked_mul(a, b)</code>	无溢出时返回 <code>Ok(a * b)</code> ，否则返回 <code>Err(Error("arithmetic overflow"))</code>
<code>saturating_add(a, b)</code>	返回 <code>a + b</code> ，溢出时钳制到 <code>int</code> 范围
<code>saturating_sub(a, b)</code>	返回 <code>a - b</code> ，溢出时钳制到 <code>int</code> 范围
<code>saturating_mul(a, b)</code>	返回 <code>a * b</code> ，溢出时钳制到 <code>int</code> 范围
<code>wrapping_add(a, b)</code>	返回溢出时回绕的 <code>a + b</code>
<code>wrapping_sub(a, b)</code>	返回溢出时回绕的 <code>a - b</code>
<code>wrapping_mul(a, b)</code>	返回溢出时回绕的 <code>a * b</code>

集合操作

函数	说明
<code>has_key(map, key)</code>	返回映射中是否存在该键
<code>add(set, value)</code>	向集合添加元素（重复则忽略）
<code>remove(set, value)</code>	从集合删除元素
<code>append(list, value) / append!(list, value)</code>	向列表末尾添加元素（就地修改）
<code>appended(list, value)</code>	返回添加元素后的新列表（非破坏性）
<code>pop(list)</code>	移除并返回列表的最后一个元素（ <code>Option<T></code> ）
<code>reverse(list)</code>	返回反转后的新列表（也适用于字符串）
<code>reverse!(list)</code>	就地反转列表（破坏性）
<code>slice(list, start, end)</code>	返回从 <code>start</code> 到 <code>end</code> 的新子列表
<code>take(list, count)</code>	返回包含前 <code>count</code> 个元素的新列表
<code>tap(list, function)</code>	对每个元素调用 <code>function</code> 以执行副作用，返回原始列表
<code>filter(list, pred)</code>	返回仅包含满足谓词的元素的新列表
<code>map(list, function)</code>	返回将每个元素转换后的新列表
<code>sort(list) / sort(list, comp)</code>	返回排序后的新列表（默认升序）
<code>sort!(list) / sort!(list, comp)</code>	就地排序列表（破坏性）
<code>insert(list, i, val)</code>	在索引 <code>i</code> 处插入元素
<code>remove_at(list, i)</code>	移除并返回索引 <code>i</code> 处的元素
<code>items(map)</code>	返回（键，值）元组的列表
<code>remove(map, key)</code>	删除指定键的条目
<code>get(map, key)</code>	返回键的值（ <code>Option<V></code> ）
<code>get(map, key, default)</code>	返回键的值，若不存在则返回默认值
<code>union(set, set)</code>	返回两个集合的并集
<code>intersection(set, set)</code>	返回两个集合的交集
<code>difference(set, set)</code>	返回两个集合的差集

函数	说明
<code>symmetric_difference(set, set)</code>	返回两个集合的对称差
<code>is_subset(set, set)</code>	返回第一个集合是否为第二个的子集
<code>is_superset(set, set)</code>	返回第一个集合是否为第二个的超集
<code>first(list)</code>	返回第一个元素 (<code>Option<T></code>)，列表为空时返回 <code>None</code>
<code>last(list)</code>	返回最后一个元素 (<code>Option<T></code>)，列表为空时返回 <code>None</code>
<code>remove(list, value)</code>	从列表中移除第一个匹配的值
<code>is_empty(list / map / set / str)</code>	返回集合或字符串是否为空
<code>distinct(list)</code>	返回移除重复元素后的新列表
<code>flatten(list)</code>	返回将嵌套列表展开后的新列表
<code>reduce(list, fn)</code>	使用归约函数将列表归约为单个值
<code>fold(list, init, fn)</code>	使用初始累加器值折叠列表
<code>any(list, pred)</code>	如果任一元素满足谓词则返回 <code>true</code>
<code>all(list, pred)</code>	如果所有元素都满足谓词则返回 <code>true</code>
<code>sum(list)</code>	返回所有元素的总和
<code>min(list)</code>	返回最小的元素
<code>max(list)</code>	返回最大的元素
<code>enumerate(list)</code>	返回 (<code>index, value</code>) 元组的列表。也接受 <code>str</code> ，每个 UTF-8 码位产生 (<code>int, str</code>)
<code>zip(list1, list2)</code>	返回将两个列表的元素配对的 (<code>a, b</code>) 元组列表。任一（或两个）参数可以是 <code>str</code>
<code>keys(map)</code>	以 <code>List<K></code> 返回所有键
<code>values(map)</code>	以 <code>List<V></code> 返回所有值
<code>merge(map1, map2)</code>	返回包含两个映射所有条目的新映射

迭代器

函数	说明
<code>iter(collection)</code>	从 <code>List</code> 、 <code>Set</code> 或 <code>Map</code> 创建惰性迭代器
<code>next(iter)</code>	返回下一个元素 (<code>Option<T></code>)，耗尽时返回 <code>None</code>
<code>to_list(iter)</code>	将迭代器剩余的所有元素收集到 <code>List<T></code>
<code>filter(iter, pred)</code>	返回只产出满足谓词的元素的惰性迭代器
<code>map(iter, function)</code>	返回转换每个元素的惰性迭代器
<code>take(iter, count)</code>	返回最多产出 <code>count</code> 个元素的惰性迭代器

字符串操作

函数	说明
<code>contains(string, substring)</code>	是否包含子字符串
<code>starts_with(string, prefix)</code>	是否以前缀开头
<code>ends_with(string, suffix)</code>	是否以后缀结尾
<code>find(string, substring)</code>	子字符串的字符位置 (<code>Option<int></code>)
<code>byte_len(string)</code>	返回字符串的字节长度
<code>substring(string, start, end)</code>	提取子字符串
<code>char_at(string, i)</code>	获取指定位置的字符
<code>replace(string, old, new)</code>	替换所有出现的子字符串
<code>to_upper(string) / to_lower(string)</code>	大小写转换
<code>trim(string) / trim_start(string) / trim_end(string)</code>	去除空白

函数	说明
<code>repeat(string, count)</code>	将字符串重复 <code>n</code> 次
<code>reverse(string)</code>	反转字符串
<code>split(string, delimiter)</code>	分割字符串并返回列表
<code>join(list, sep)</code>	以分隔符连接列表中的字符串
<code>to_int(s) / to_float(s) / to_str(v)</code>	类型转换 (<code>to_int</code> 与 <code>to_float</code> 返回 <code>Result<T, Error></code>)

-> 详细请参阅[字符串操作函数参考](#)

print

签名：`print() / print(expr1, expr2, ...)`

将一个或多个值输出到标准输出，以空格分隔。末尾会追加换行。不带参数调用时仅输出换行。

类型	输出格式
<code>int</code>	<code>%ld</code>
<code>float</code>	<code>%g</code> ，整数值会附带 <code>.0</code> （例如 <code>3.0</code> 、 <code>0.0</code> ）
<code>bool</code>	<code>true / false</code>
<code>str</code>	<code>%s</code>
<code>Result (Ok)</code>	<code>Ok(value)</code>
<code>Result (Err)</code>	<code>Err(value)</code>
<code>Option (Some)</code>	<code>Some(value)</code>
<code>Option (None)</code>	<code>None</code>
<code>list</code>	<code>[elem1, elem2, ...]</code>
<code>map</code>	<code>{key1: val1, key2: val2, ...}</code>
<code>set</code>	<code>{elem1, elem2, ...}</code>
<code>tuple</code>	<code>(elem1, elem2, ...)</code>
<code>enum</code>	变体名称（例如： <code>Red</code> ）
<code>record</code>	<code>RecordName(field: val, ...)</code>
<code>function 值 (closure / lambda)</code>	<code><closure></code>
<code>union</code>	格式化为活动变体的类型

整数值的 `float` 总是会附带 `.0` 输出，以便与 `int` 视觉上区分。嵌套集合（例如 `Map<str, List<int>>`）会使用内部元素的格式化器递归格式化。底层类型为 `List`、`Map` 或 `Set` 的 `union` 变体会格式化为该集合；底层类型为 `function` 值的变体会格式化为 `<closure>`。

```

print(42)           # 42
print(3.14)         # 3.14
print(3.0)          # 3.0      (整数值 float 保留 .0)
print(0.0)          # 0.0
print(true)         # true
print("hello")      # hello
print(Ok(42))       # Ok(42)
print(Err(Error("fail"))) # Err(Error: fail (code: 0))
print(Some(1))      # Some(1)
print(None)         # None
print([1, 2, 3])    # [1, 2, 3]
print({"a": 1})     # {"a": 1}
print({1, 2, 3})    # {1, 2, 3}

```

```

print((1, "hello")) # (1, "hello")

# 嵌套集合
m: Map<str, List<int>> = {"a": [1, 2, 3]}
print(m)                # {"a": [1, 2, 3]}

# 集合类型的 union 变体
x: int | List<int> = [1, 2, 3]
print(x)                # [1, 2, 3]

# Function 值
f = (x: int) => x * 2
print(f)                # <closure>

# 多个参数（以空格分隔）
print(1, 2, 3)          # 1 2 3
print("hello", "world") # hello world
print(1, "hello", true) # 1 hello true
print()                 # (空行)

```

Some

签名: `Some(expr) -> Option<T>`

构造 `Option` 类型的有值变体。

```

x: Option<int> = Some(42)
print(x)      # Some(42)

```

length

签名: `length(x: List<T> | Map<K, V> | Set<T> | str) -> int`

返回列表、映射、集合的元素数量，或字符串的 UTF-8 字符数。如需获取字节长度，请使用 `byte_len()`。

```

print(length([1, 2, 3]))      # 3
print(length({"a": 1, "b": 2})) # 2
print(length({1, 2, 3}))      # 3
print(length("hello"))        # 5
print(length("あい"))         # 3 (UTF-8 字符数)

```

has_key

签名: `has_key(m: Map<K, V>, key: K) -> bool`

返回映射中是否存在指定的键。也可使用 `UFCS` 记法。

```

m = {"a": 1, "b": 2}
print(has_key(m, "a")) # true
print(m.has_key("z"))  # false (UFCS)

```

add

签名: `add(s: Set<T>, value: T)`

向集合添加元素。若元素已存在则不做任何操作。也可使用 UFCS 记法。

```
s = {1, 2, 3}
s.add(4)           # UFCS
add(s, 5)          # 普通调用
s.add(1)           # 已存在，因此忽略
print(length(s))   # 5
```

remove

签名: `remove(s: Set<T>, value: T)`

从集合删除元素。也可使用 UFCS 记法。

```
s = {1, 2, 3}
s.remove(2)        # UFCS
print(2 in s)      # false
```

range

签名: `range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

生成整数列表。

形式	生成的值
<code>range(n)</code>	<code>[0, 1, ..., n-1]</code>
<code>range(start, end)</code>	<code>[start, start+1, ..., end-1]</code>
<code>range(start, end, step)</code>	<code>[start, start+step, start+2*step, ...]</code> (不包含 end)

- `step > 0` 时，从 `start` 向 `end` 递增生成。
- `step < 0` 时，从 `start` 向 `end` 递减生成。
- `step == 0` 时，会产生运行时错误。
- 如果范围为空（例如 `range(0, 10, -1)`），返回空列表。

```
print(range(3))           # [0, 1, 2]
print(range(2, 5))        # [2, 3, 4]
print(range(0, 10, 2))    # [0, 2, 4, 6, 8]
print(range(10, 0, -3))   # [10, 7, 4, 1]

for i in range(3):
    print(i)
# 0
# 1
# 2
```

exit

签名：`exit(code: int)`

以指定的退出码立即终止进程。`exit()` 之后的语句会被编译为不可达块，LLVM 会在优化期间将其移除，因此它们永远不会运行：

```
exit(0)          # 正常终止
exit(1)          # 错误终止

print("a")
exit(0)
print("b")       # 永远不会输出 — exit 之后不可达
```

相同的处理也适用于 `return`、`break` 和 `continue` — 任何发散控制流语句之后的代码都会被静默移除。

arguments

签名：`arguments() -> List<str>`

以字符串列表的形式返回传递给脚本的命令行参数。不包含解释器名称或脚本文件名——仅包含脚本路径之后的参数。

```
# 运行：ry script.ry hello world
a = arguments()
print(length(a))    # 2
print(a[0])         # hello
print(a[1])         # world

for x in arguments():
    print(x)
```

sleep

签名：`sleep(duration_ms: int) -> Unit`

暂停当前线程的执行指定的毫秒数。若 `duration_ms` 为 0 或负数，则函数立即返回。

```
sleep(1000)        # 等待 1 秒
sleep(0)           # 立即返回
```

env

签名：`env(key: str) -> Option<str>` / `env(key: str, default: str) -> str`

返回环境变量的值。单参数形式返回 `Option<str>`（若已设置则为 `Some(value)`，未设置则为 `None`）。双参数形式在变量未设置时返回 `default`。

若项目根目录（包含 `package.toml` 的目录）中存在 `.env` 文件，启动时会自动载入到进程环境中。现有的环境变量不会被 `.env` 的值覆盖。

安全提示：`.env` 文件通常包含密钥（API 密钥、数据库密码、令牌等）。请不要将 `.env` 提交到版本控制系统（添加到 `.gitignore` 或类似文件中），并将其内容视为敏感配置。

```
# 单参数形式：返回 Option<str>
path = env("PATH")
case path:
  Some(v):
    print(v)
  None:
    print("PATH not set")

# 双参数形式：带默认值返回 str
port = env("PORT", "8080")
print(port)    # 若 PORT 未设置则为 "8080"
```

.env 文件格式

```
# 注释以 # 开头
DATABASE_URL=postgres://localhost/mydb
API_KEY="secret-key-123"
EMPTY_VALUE=
QUOTED='single quoted'
```

环境专属 .env 文件

设置 `RY_ENV` 时，Ry 会按照以下优先顺序载入环境专属的 .env 文件：

- 先载入 `.env.<env>`（例如 `RY_ENV=dev` 时载入 `.env.dev`）
- 再载入 `.env`（已由 `.env.<env>` 设置的值不会被覆盖）
- `RY_ENV=prod` 时不载入任何 `.env` 文件（安全考虑）
- `RY_ENV` 未设置时仅载入 `.env`（向后兼容）

环境模式的详细信息请参阅 [RY_ENV](#)。

append

签名：`append(list: List<T>, value: T)`

向列表末尾添加元素。此为就地修改操作——列表会被直接修改。也可使用 UFCS 记法。

```
xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

pop

签名：`pop(list: List<T>) -> Option<T>`

移除并返回列表的最后一个元素（`Option<T>`）。列表为空时返回 `None`。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
v = xs.pop()
print(v)      # Some(3)
print(xs)     # [1, 2]
```

reverse (列表)

签名: `reverse(list: List<T>) -> List<T>`

返回元素顺序反转的新列表。原始列表不会被修改。也适用于字符串（请参阅[字符串操作](#)）。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
ys = reverse(xs)
print(ys)    # [3, 2, 1]
print(xs)    # [1, 2, 3] (未修改)
```

slice

签名: `slice(list: List<T>, start: int, end: int) -> List<T>`

返回从 `start`（含）到 `end`（不含）的新子列表。索引会被钳制在有效范围内（0 到 `length(list)`）。也可使用 UFCS 记法。

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (钳制)
```

take

签名: `take(list: List<T>, count: int) -> List<T>`

返回包含前 `count` 个元素的新列表。若 `count` 超过列表长度，返回整个列表的副本。若 `count <= 0`，返回空列表。原始列表不会被修改。也可使用 UFCS 记法。

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)    # [1, 2, 3]
print(xs.take(10))  # [1, 2, 3, 4, 5] (钳制)
print(xs.take(0))   # []
```

tap

签名: `tap(list: List<T>, function: function(T) -> R) -> List<T>`

对每个元素调用给定函数（忽略返回值），然后返回原始列表。适用于方法链中的调试或插入副作用。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
ys = xs.tap((x: int) => print(x)).map((x: int) => x * 2)
# 输出 1, 2, 3, 然后 ys = [2, 4, 6]
```

filter

签名: `filter(list: List<T>, pred: function(T) -> bool) -> List<T>`

返回仅包含谓词返回 `true` 的元素的新列表。原始列表不会被修改。也可使用 UFCS 记法。

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter((x: int) => x > 3)
print(ys)    # [4, 5]
print(xs)    # [1, 2, 3, 4, 5] (未修改)
```

map

签名: `map(list: List<T>, function: function(T) -> U) -> List<U>`

返回将每个元素以给定函数转换后的新列表。输出元素类型可以与输入不同。原始列表不会被修改。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
ys = xs.map((x: int) => x * 2)
print(ys)    # [2, 4, 6]
```

sort

签名: `sort(list: List<T>) -> List<T> / sort(list: List<T>, comp: function(T, T) -> bool) -> List<T>`

返回排序后的新列表。默认为升序。可提供自定义比较函数（第一参数应排在第二参数之前时返回 `true`）。原始列表不会被修改。排序是稳定的（相等元素保持原始顺序）。也可使用 UFCS 记法。

```
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# 降序排序
desc = xs.sort((a: int, b: int) => a > b)
print(desc)    # [3, 2, 1]
```

sort!

签名: `sort!(list: List<T>) / sort!(list: List<T>, comp: function(T, T) -> bool)`

就地排序列表。排序算法与 `sort()` 相同，但修改原始列表而非创建新列表。也可使用 UFCS 记法。

```
xs = [3, 1, 2]
xs.sort!()
print(xs)    # [1, 2, 3]
```

reverse!

签名: `reverse!(list: List<T>)`

就地反转列表。也可使用 UFCS 记法。

```
xs = [1, 2, 3]
xs.reverse!()
print(xs)    # [3, 2, 1]
```

appended

签名: `appended(list: List<T>, value: T) -> List<T>`

返回添加元素后的新列表。原始列表不会被修改。也可使用 UFCS 记法。

```
xs = [1, 2]
ys = xs.appended(3)
print(xs)    # [1, 2] (未修改)
print(ys)    # [1, 2, 3]
```

append!

签名: `append!(list: List<T>, value: T)`

`append()` 的别名。就地向列表末尾添加元素。为配合 ! 命名约定而提供。

first

签名: `first(list: List<T>) -> Option<T>`

返回列表的第一个元素 (`Option<T>`)。列表为空时返回 `None`。

```
print(first([10, 20, 30])) # Some(10)
```

last

签名: `last(list: List<T>) -> Option<T>`

返回列表的最后一个元素 (`Option<T>`)。列表为空时返回 `None`。

```
print(last([10, 20, 30])) # Some(30)
```

get (映射)

签名: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

双参数形式返回键的值 (`Option<V>`)。三参数形式返回键的值，若不存在则返回默认值。

```
m = {"a": 1, "b": 2}
print(get(m, "a"))      # Some(1)
print(get(m, "z"))      # None
print(get(m, "z", 0))   # 0
```

iter

签名: `iter(collection: List<T> | Set<T>) -> Iterator<T>` / `iter(collection: Map<K, V>) -> Iterator<(K, V)>`

从集合创建惰性迭代器。迭代器不复制数据；它引用原始集合。也可使用 UFCS 记法。

- 对于 `List<T>` 和 `Set<T>`，元素类型为 `T`。

- 对于 `Map<K, V>`，元素类型为元组 `(K, V)`。

```
xs = [1, 2, 3]
it = xs.iter()           # Iterator<int>
ys = it.to_list()        # [1, 2, 3]

m = {"a": 1, "b": 2}
for k, v in m.iter():     # Iterator<(str, int)>
    print(k)
```

next

签名：`next(iter: Iterator<T>) -> Option<T>`

返回迭代器的下一个元素 (`Option<T>`)。当迭代器耗尽时返回 `None`。每次调用时迭代器会推进其内部状态。也可使用 UFCS 记法。

```
it = [10, 20].iter()
print(it.next())      # Some(10)
print(it.next())      # Some(20)
print(it.next())      # None
```

to_list

签名：`to_list(iter: Iterator<T>) -> List<T>`

将迭代器剩余的所有元素收集到新列表中。也可使用 UFCS 记法。

```
xs = [1, 2, 3, 4, 5]
ys = xs.iter().filter((x: int) => x > 2).to_list()
print(ys)             # [3, 4, 5]
```

type_of

签名：`type_of(expr: T) -> Type`

返回一个表达式的类型，作为 `Type` 值。每个不同的类型定义（基本类型、集合、record、enum、`Option`、`Result`、`function` 等）在编译时都会获得唯一的标识，因此可以使用 `==` 比较 `type_of` 值，以检查两个表达式是否共享同一类型。

- 参数会因副作用被求值，但只使用其静态类型。
- 通过 `print` 或 `to_str` 输出 `Type` 值会产生人类可读的名称（例如 `"int"`、`"List"`、`"Point"`）。
- 具有相同规范类型的两个表达式返回相等的 `Type` 值；不同的 `record`（或恰好同名的 `record` 与 `enum`）始终可区分。
- 字面 `none` 报告为 `"None"`。一个有类型的 `Option<T>` 值（无论是通过 `Some(...)` 构造还是从 `none` 赋值）报告为 `"Option"`。

```
record Point:
  x: int
  y: int
```

```
enum Color:
  Red
  Green
```

Blue

```
print(to_str(type_of(42)))      # int
print(to_str(type_of(3.14)))    # float
print(to_str(type_of("hello"))) # str
print(to_str(type_of([1, 2, 3]))) # List
print(to_str(type_of({"a": 1}))) # Map
print(to_str(type_of({1, 2})))  # Set

p = Point(1, 2)
print(to_str(type_of(p)))      # Point

c = Color::Red
print(to_str(type_of(c)))      # Color

# 标识比较
print(type_of(42) == type_of(100)) # true
print(type_of(42) == type_of(3.14)) # false
print(type_of(p) != type_of(c))    # true

# 低层数值类型与 `int` 区分
x: i32 = 1
print(to_str(type_of(x)))      # i32
print(type_of(x) == type_of(42)) # false

# type_of 是反射性的: Type 值的类型是 Type
print(to_str(type_of(type_of(42)))) # Type
```

type_of 返回的类型类别

输入	to_str(type_of(...))
42	int
3.14	float
true / false	bool
"hello"	str
[1, 2]	List
{"a": 1}	Map
{1, 2}	Set
x: i32 = 1	i32 (同样适用于 u8、i16、...、f32)
record 值	record 名称 (例如 Point)
enum 值	enum 名称 (例如 Color)
none 字面	None
Some(1)	Option
x: Option<int> = none	Option
Ok(1) / Err(e)	Result
lambda / closure	function
type_of(x)	Type

字面 none 报告为 "None" 以与有类型的 Option 值区分。任何 Option<T> 容器 — 无论是通过 Some(...) 构造，还是通过将 none 赋值给 Option<T> 类型的绑定 — 都报告为 "Option"。

[English](#) | [日本語](#) | [简体中文](#)

集合参考（元组、列表、映射、集合）

元组

概述

固定长度、异质类型的值组合。以 LLVM literal StructType 实现，是栈上的值类型。

语法

```
t = (42,)                # 单元素元组（需要尾随逗号）
t = (1, 3.14)
t: (int, float) = (1, 3.14)
```

类型注解

```
single: (int,) = (42,)    # 单元素需要尾随逗号
pair: (int, str) = (42, "hello")
triple: (int, float, bool) = (1, 2.0, true)
```

比较

元组通过逐元素比较支持 == 和 !=。

```
print((1, 2) == (1, 2))    # true
print((1, 2) != (3, 4))    # true
print(("a", 1) == ("b", 1)) # false
```

元素访问

使用 .0、.1、... 的数值索引访问。

```
t = (10, 3.14)
print(t.0)    # 10
print(t.1)    # 3.14
```

函数返回值

```
function swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
result = swap(1, 2)
print(result.0)    # 2
print(result.1)    # 1
```

约束与错误

约束	详细
超出范围的索引	编译错误
比较运算符	仅支持 == 和 !=；不支持 <、<=、>、>=

列表

概述

相同类型的可变长度序列。分配在堆上。

语法

```
xs = [1, 2, 3]
xs: List<int> = [1, 2, 3]
```

空列表

空列表需要类型注解以确定元素类型：

```
xs: List<int> = []
xs: List<str> = []
```

连接

列表可以使用 + 和 += 进行连接：

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b          # [1, 2, 3, 4, 5, 6]
a += [7, 8]        # a 现在是 [1, 2, 3, 7, 8]
```

两个操作数必须具有相同的元素类型。

支持的元素类型

int, float, bool, str

相等性

列表支持 == 和 !=。当两个列表长度相同且所有对应元素相等时，它们相等。

```
[1, 2, 3] == [1, 2, 3]    # true
[1, 2, 3] != [1, 2, 4]    # true
[1, 2]     != [1, 2, 3]    # true (不同长度)
```

索引访问

```
xs = [1, 2, 3]
print(xs[0])    # 1
print(xs[2])    # 3
```

负数索引从末尾开始计算（Python 风格）：

```
xs = [10, 20, 30]
print(xs[-1])   # 30 (最后一个元素)
print(xs[-2])   # 20
print(xs[-3])   # 10
```

超出范围的访问（包括超出列表长度的负数索引）会产生运行时错误。

索引赋值

```
xs = [1, 2, 3]
xs[0] = 99
print(xs[0])    # 99
xs[-1] = 42      # 赋值给最后一个元素
print(xs[2])    # 42
```

链式索引和字段赋值

索引和字段赋值可以组合：`=` / `+=` / `--` / `*` / `/` / `//` / `%` / `**` / `&` / `|` / `^` / `<<` / `>>` 的左侧可以是任何根植于可变变量的后缀链。

```
record Point:
    x: int
    y: int

pts = [Point(1, 2), Point(3, 4)]
pts[0].x = 99          # 列表中记录的字段更新
pts[0].x += 1
print(pts[0].x)        # 100

grid = [[1, 2], [3, 4]]
grid[0][1] = 99        # 嵌套列表元素
print(grid[0])         # [1, 99]

m: Map<str, Map<str, int>> = {"a": {"x": 1}}
m["a"]["x"] = 42        # 嵌套映射元素
```

复合赋值对每个索引恰好求值一次，因此 `xs[f()] += 1` 只调用 `f()` 一次。对缺失映射键的复合赋值是运行时错误——如果想要累加，请先插入键：

```
m = {"a": 1}
m["a"] += 10            # OK → {"a": 11}
m["b"] += 10            # 运行时错误：对缺失映射键的复合赋值
```

嵌套写入是隔离的：像 `grid[i][j] = v` 和 `r.items[i] = v`（通过包含列表的记录字段）这样的链式写入会在变更发生之前私有化 LHS 路径上引用计数 > 1 的每一层，因此外部容器的别名——或路径上任何内部容器——会观察到写入前的状态。详细信息请参阅下面的 [Path Copy-on-Write](#)。

length

```
xs = [1, 2, 3]
print(length(xs))      # 3
```

print

```
xs = [1, 2, 3]
print(xs)              # [1, 2, 3]
```

for 遍历

```
xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

append

向列表末尾添加元素。此为就地修改操作。

```
xs = [1, 2]
xs.append(3)
print(xs)              # [1, 2, 3]
```

pop

移除并返回列表的最后一个元素。列表为空时返回 `None`。

```
xs = [1, 2, 3]
v = xs.pop()
print(v)      # Some(3)
print(xs)     # [1, 2]
```

reverse

返回元素顺序反转的新列表。原始列表不会被修改。也适用于字符串。

```
xs = [1, 2, 3]
print(reverse(xs))  # [3, 2, 1]
print(xs)           # [1, 2, 3] (unchanged)
```

slice

返回从 `start`（含）到 `end`（不含）的新子列表。索引会被钳制在有效范围内。

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))  # [2, 3]
print(slice(xs, 0, 100)) # [1, 2, 3, 4, 5] (clamped)
```

take

返回包含前 `count` 个元素的新列表。若 `count` 超过列表长度，返回整个列表的副本。若 `count` ≤ 0 ，返回空列表。原始列表不会被修改。

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)  # [1, 2, 3]
print(xs.take(10))  # [1, 2, 3, 4, 5] (clamped)
print(xs.take(0))   # []
```

tap

对每个元素调用给定函数（忽略返回值），然后返回原始列表。适用于方法链中的调试或插入副作用。

```
xs = [1, 2, 3]
ys = xs.tap((x: int) => print(x)).map((x: int) => x * 2)
# prints 1, 2, 3, then ys = [2, 4, 6]
```

filter

返回仅包含满足谓词的元素的新列表。原始列表不会被修改。

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter((x: int) => x > 3)
print(ys)  # [4, 5]
```

map

返回将每个元素以给定函数转换后的新列表。输出元素类型可以与输入不同。原始列表不会被修改。

```
xs = [1, 2, 3]
ys = xs.map((x: int) => x * 2)
print(ys)  # [2, 4, 6]
```

sort

返回排序后的新列表。默认为升序。可提供自定义比较函数。原始列表不会被修改。排序是稳定的（相等元素保持原始顺序）。内部使用 TimSort。

```
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# 使用比较函数降序排序
desc = xs.sort((a: int, b: int) => a > b)
print(desc)        # [3, 2, 1]
```

filter、map、sort 的链式调用

这些函数返回新列表，因此可通过 UFCS 进行链式调用。

```
xs = [5, 3, 1, 4, 2]
result = xs.filter((x: int) => x > 1).map((x: int) => x * 10).sort()
print(result)      # [20, 30, 40, 50]
```

reduce

使用累加函数将列表归约为单个值，以第一个元素作为初始值。

```
xs = [1, 2, 3, 4, 5]
total = reduce(xs, (a: int, b: int) => a + b)
print(total)       # 15
```

fold

使用明确的初始值和累加函数将列表折叠为单个值。

```
xs = [1, 2, 3, 4, 5]
total = fold(xs, 0, (a: int, b: int) => a + b)
print(total)       # 15
```

any

如果至少有一个元素满足谓词，则返回 true。

```
xs = [1, 2, 3, 4, 5]
print(any(xs, (x: int) => x > 4))    # true
print(any(xs, (x: int) => x > 9))    # false
```

all

如果所有元素都满足谓词，则返回 true。

```
xs = [2, 4, 6]
print(all(xs, (x: int) => x > 0))    # true
print(all(xs, (x: int) => x > 3))    # false
```

sum

返回所有元素的总和。

```
xs = [1, 2, 3, 4, 5]
print(sum(xs))    # 15
```

min

返回最小的元素。

```
xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```

max

返回最大的元素。

```
xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

first

返回第一个元素。列表为空时返回 None。

```
xs = [10, 20, 30]
print(first(xs))   # Some(10)
```

last

返回最后一个元素。列表为空时返回 None。

```
xs = [10, 20, 30]
print(last(xs))    # Some(30)
```

is_empty

如果容器没有元素则返回 true。接受列表、映射、集合和字符串。

```
xs = [1, 2, 3]
print(is_empty(xs))           # false
print(is_empty([]))           # true (requires type annotation in practice)
print(is_empty(""))           # true (str support, #831)
print(is_empty("hello"))      # false
```

enumerate

返回 (index, element) 元组的列表。

```
xs = [10, 20, 30]
pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# for 循环中的元组解构
for i, x in enumerate(xs):
    print(f"{i}: {x}")        # 0: 10, 1: 20, 2: 30
```

zip

将两个列表合并为 (elem1, elem2) 元组的列表。结果长度等于较短的列表。

```
xs = [1, 2, 3]
ys = ["a", "b", "c"]
pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# for 循环中的元组解构
```



```
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c
```

insert

在指定索引处插入元素。该索引及之后的元素向右移动。

```
xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)    # [1, 99, 2, 3]
```

remove_at

移除并返回指定索引处的元素。该索引之后的元素向左移动。

```
xs = [1, 2, 3, 4]
v = remove_at(xs, 1)
print(v)    # 2
print(xs)    # [1, 3, 4]
```

remove

从列表中移除第一个匹配的指定值。若找不到该值，则不做任何操作。这是一个可变操作。

```
xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)    # [1, 3, 2, 4]
```

distinct

返回一个移除重复元素的新列表。保持原始顺序（保留第一次出现）。原始列表不会被修改。

```
xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))    # [1, 2, 3, 4]
print(xs)    # [1, 2, 3, 2, 1, 4] (unchanged)
```

flatten

将嵌套列表（列表的列表）展开一层。返回新列表。原始列表不会被修改。

```
xs = [[1, 2], [3, 4]]
print(flatten(xs))    # [1, 2, 3, 4]
print(xs)    # [[1, 2], [3, 4]] (unchanged)
```

就地修改变体

某些列表操作有两种形式：返回新列表的非变更版本，以及名称以 **!** 结尾的就地变体。当您打算变更接收器并希望调用点明确此意图时，请使用 **!** 形式；当您希望保留原始值时，请使用非变更形式。

就地 (!)	非变更等价	区别
<code>append!(xs, v) / append(xs, v)</code>	<code>appended(xs, v)</code>	<code>append!</code> / <code>append</code> 就地变更并返回 <code>Unit</code> ； <code>appended</code> 返回新列表
<code>sort!(xs) / sort!(xs, cmp)</code>	<code>sort(xs) / sort(xs, cmp)</code>	<code>sort!</code> 就地变更 <code>xs</code> ； <code>sort</code> 返回新的排序列表
<code>reverse!(xs)</code>	<code>reverse(xs)</code>	<code>reverse!</code> 就地变更 <code>xs</code> ； <code>reverse</code> 返回新的反转列表

所有！变体都参与 Copy-on-Write：如果 `xs` 是共享的（引用计数 > 1 ），外部缓冲区会在就地变更前克隆一次，以使别名不受影响（请参阅 [Copy-on-Write \(CoW\)](#) 语意）。

```
xs = [3, 1, 2]
sort!(xs)
print(xs)           # [1, 2, 3]

ys = [1, 2, 3]
reverse!(ys)
print(ys)           # [3, 2, 1]

zs = [1, 2]
append!(zs, 3)
print(zs)           # [1, 2, 3]

# 非变更变体：appended 返回新列表
a = [1, 2]
b = appended(a, 3)
print(a)            # [1, 2]（未变更）
print(b)            # [1, 2, 3]
```

何时使用哪个：

- 当读者受益于看到原始值被保留时（例如函数式管道，或当后续仍然需要原始值时），优先使用 `sort`、`reverse`、`appended`。
- 当原始值真的被替换时，优先使用 `sort!`、`reverse!`、`append!`，以避免分配中间副本，并在调用点明确变更意图。

操作复杂度

操作	复杂度
<code>xs[i]</code> 索引访问	$O(1)$
<code>append</code> / <code>append!</code>	均摊 $O(1)$
<code>pop</code>	$O(1)$
<code>first</code> , <code>last</code>	$O(1)$
<code>insert</code> , <code>remove_at</code>	$O(n)$
<code>sort</code> / <code>sort!</code>	$O(n \log n)$
<code>take</code>	$O(n)$
<code>tap</code>	$O(n)$
<code>filter</code> , <code>map</code> , <code>reduce</code> , <code>fold</code>	$O(n)$
<code>reverse</code> / <code>reverse!</code>	$O(n)$
<code>distinct</code>	$O(n)$
<code>length</code>	$O(1)$

约束与错误

约束	详细
所有元素必须为相同类型	混合不同类型会产生编译错误
空列表 <code>[]</code>	需要类型注解（例如 <code>xs: List<int> = []</code> ）
超出范围的访问	运行时错误（ <code>exit(1)</code> ）

Copy-on-Write (CoW) 语义

所有集合类型 (List、Map、Set) 在 ARC 管理下使用 **Copy-on-Write** 语义。这意味着：

- 赋值共享数据：b = a 不会复制集合——两个变量引用相同的数据。引用计数递增。
- 变更触发路径遍历复制：当共享集合被变更时，LHS 路径上引用计数 > 1 的每一层都会在变更发生之前被克隆。顶层写入 (b.append(...)、b[i] = v，当 b 是共享时) 仅克隆最外层容器。链式写入 (b[i][j] = v、r.items[i] = v、m[k1][k2] = v) 从根遍历到底，并克隆仍然共享的每个中间容器，因此外部容器的别名— 或路径上的任何内部容器 — 都会与变更隔离。
- 唯一所有者就地修改：当集合 (或某一跳的容器) 只有一个引用 (strong_count == 1) 时，CoW 检查会跳过该层级的克隆并就地修改，零复制开销。

```
a = [1, 2, 3]           # strong_count = 1
b = a                   # strong_count = 2 (共享)
append(b, 4)            # strong_count > 1 → 复制外部缓冲区，然后修改
                        # a = [1, 2, 3] (strong_count = 1)
                        # b = [1, 2, 3, 4] (strong_count = 1, 新分配)

c = [10, 20]            # strong_count = 1
append(c, 30)           # strong_count == 1 → 就地修改 (无复制)
```

Path Copy-on-Write

Path CoW 处理通过嵌套集合的链式写入 — 写入器从 LHS 根变量 (或记录字段) 遍历到叶写入点，并私有化引用计数大于一的每一层。这在共享外部容器和路径上任何内部容器的别名之间提供严格的别名隔离。

```
a = [[1, 2], [3, 4]]
b = a                   # 外部列表共享: strong_count = 2
b[0][0] = 99           # 遍历: 克隆外部 (a 和 b 现在有自己的
                        # 外部); 克隆 b 的 inner[0] (a 的 inner[0] 保持 [1, 2]);
                        # 将 99 写入克隆

# a = [[1, 2], [3, 4]]
# b = [[99, 2], [3, 4]]
```

带有 ARC 管理的集合值的记录字段以与直接索引跳转相同的方式参与 path CoW。记录到记录的赋值 (r2 = r1) 保留每个 ARC 字段，因此后续的 r2.items[i] = v 会在字段槽观察到 refcount > 1 并在变更前克隆：

```
record Box:
  items: List<int>

r1 = Box([1, 2, 3])
r2 = r1
r2.items[0] = 99
# r1.items[0] == 1
# r2.items[0] == 99
```

不支持作为 path-CoW 链的根：方法调用 lvalue (f().x[i] = v) 和在记录字段遍历内穿插索引跳转的链 (rec.arr[0].items[i] = v)。这些形状会产生编译时错误；请先将中间值赋给局部变量。

触发 CoW 的操作

类型	可变操作
List	append()、pop()、insert()、remove()、remove_at()、sort!()、reverse!()、索引赋值 (xs[i] = val)
Map	remove()、索引赋值 (m[key] = val)
Set	add()、remove()

非可变操作（`appended`、`slice`、`take`、`filter`、`map`、`sort`、`reverse`、`get`、`items` 等）创建新集合，不触发 CoW。

映射

概述

键与值的对应表。分配在堆上。

语法

```
m = {"a": 1, "b": 2}
m: Map<str, int> = {"a": 1, "b": 2}
```

相等性

映射支持 `==` 和 `!=`。当两个映射拥有相同数量的条目，且一个映射中的每个键值对在另一个映射中都存在且值相等时，它们相等。

```
{"a": 1, "b": 2} == {"a": 1, "b": 2}    # true
{"a": 1}           != {"a": 2}           # true (不同值)
{"a": 1}           != {"b": 1}           # true (不同键)
```

键访问

```
m = {"a": 1, "b": 2}
print(m["a"])    # 1
```

插入与更新

```
m = {"a": 1}
m["b"] = 2    # 插入新条目
m["a"] = 99   # 更新已有条目
```

length

```
m = {"a": 1, "b": 2, "c": 3}
print(length(m))    # 3
```

print

```
m = {"a": 1, "b": 2}
print(m)    # {a: 1, b: 2}
```

has_key

```
m = {"a": 1, "b": 2}
print(m.has_key("a"))    # true
print(m.has_key("z"))    # false
```

keys

返回映射中所有键的列表。

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
```

values

返回映射中所有值的列表。

```
m = {"a": 1, "b": 2, "c": 3}
print(values(m))    # [1, 2, 3]
```

items

返回映射中所有条目的 (key, value) 元组列表。

```
m = {"a": 1, "b": 2}
pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

remove (Map)

从映射中删除指定键的条目。若键不存在则不做任何操作。

```
m = {"a": 1, "b": 2}
remove(m, "a")
print(m)    # {b: 2}
```

get

返回指定键的值，若键不存在则返回默认值。

```
m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

merge

返回一个合并两个映射的新映射。当键重复时，第二个映射的值优先。原始映射不会被修改。

```
m1 = {"a": 1, "b": 2}
m2 = {"b": 99, "c": 3}
m3 = merge(m1, m2)
print(m3["a"])    # 1
print(m3["b"])    # 99
print(m3["c"])    # 3
```

约束与错误

约束	详细
所有键必须为相同类型	混合不同类型的键会产生编译错误
所有值必须为相同类型	混合不同类型的值会产生编译错误
空映射	需要类型注解（例如 <code>m: Map<str, int> = {}</code> ）
访问不存在的键	运行时错误（ <code>exit(1)</code> ）
键查找	哈希表（平均 $O(1)$ ）
容量溢出	自动扩展为 2 倍

集合

概述

持有相同类型的元素且不重复的集合。分配在堆上。

语法

```
s = {1, 2, 3}
s: Set<int> = {1, 2, 3}
```

支持的元素类型

int, float, bool, str

相等性

集合支持 == 和 !=。相等性与顺序无关 — 当两个集合包含完全相同的元素时，它们相等。

```
{1, 2, 3} == {3, 2, 1}    # true (顺序无关紧要)
{1, 2}    != {1, 2, 3}    # true (不同大小)
```

in 运算符 (成员检查)

```
s = {1, 2, 3}
print(2 in s)    # true
print(5 in s)    # false
```

length

```
s = {1, 2, 3}
print(length(s))    # 3
```

print

```
s = {1, 2, 3}
print(s)    # {1, 2, 3}
```

add (添加元素)

添加重复的元素时会被忽略。

```
s = {1, 2, 3}
s.add(4)    # 添加
s.add(1)    # 已存在，因此忽略
print(length(s))    # 4
```

remove (删除元素)

```
s = {1, 2, 3}
s.remove(2)
print(2 in s)    # false
```

for 遍历

```
s = {10, 20, 30}
for x in s:
    print(x)
```

空集合

空集合需要类型注解。

```
s: Set<int> = {}
```

函数参数

```
function has_value(s: Set<int>, v: int) -> bool:  
    return v in s
```

union

返回包含两个集合所有元素的新集合。

```
a = {1, 2, 3}  
b = {3, 4, 5}  
print(union(a, b))    # {1, 2, 3, 4, 5}
```

intersection

返回仅包含两个集合中都存在的元素的新集合。

```
a = {1, 2, 3}  
b = {2, 3, 4}  
print(intersection(a, b))    # {2, 3}
```

difference

返回包含在第一个集合中但不在第二个集合中的元素的新集合。

```
a = {1, 2, 3}  
b = {2, 3, 4}  
print(difference(a, b))    # {1}
```

symmetric_difference

返回包含在任一集合中但不同时在两个集合中的元素的新集合。

```
a = {1, 2, 3}  
b = {2, 3, 4}  
print(symmetric_difference(a, b))    # {1, 4}
```

is_subset

如果第一个集合的所有元素都包含在第二个集合中，则返回 true。

```
a = {1, 2}  
b = {1, 2, 3}  
print(is_subset(a, b))    # true  
print(is_subset(b, a))    # false
```

is_superset

如果第一个集合包含第二个集合的所有元素，则返回 true。

```
a = {1, 2, 3}  
b = {1, 2}  
print(is_superset(a, b))    # true  
print(is_superset(b, a))    # false
```

约束与错误

约束	详细
所有元素必须为相同类型	混合不同类型会产生编译错误
空集合 {}	需要类型注解
元素查找	哈希表 (平均 $O(1)$)
容量溢出	自动扩展为 2 倍

迭代器

概述

惰性迭代器抽象，支持高效的数据转换管道。迭代器不会复制或具体化中间结果——每个元素按需处理。

创建迭代器

使用 `iter()` 从任何集合创建迭代器:

```
xs = [1, 2, 3, 4, 5]
it = xs.iter()           # Iterator<int>

s = {10, 20, 30}
sit = s.iter()           # Iterator<int>

m = {"a": 1, "b": 2}
mit = m.iter()           # Iterator<(str, int)>
```

惰性方法链

迭代器方法返回新的迭代器，形成一个只在消费时才执行的管道:

方法	说明
<code>.filter(function)</code>	仅产出谓词返回 <code>true</code> 的元素
<code>.map(function)</code>	使用给定函数转换每个元素
<code>.take(count)</code>	最多产出 <code>count</code> 个元素

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter((x: int) => x > 2)
    .map((x: int) => x * 2)
    .take(2)
    .to_list()  # [6, 8]
```

消费迭代器

方法	说明
<code>.to_list()</code>	将所有元素收集到 <code>List<T></code>
<code>.next()</code>	返回下一个元素，类型为 <code>Option<T></code>

```
it = [10, 20].iter()
print(it.next())  # Some(10)
print(it.next())  # Some(20)
print(it.next())  # None
```


For 循环支持

迭代器可以直接在 for 循环中使用:

```
for x in [1, 2, 3].iter().filter((x: int) => x > 1):
    print(x)
# 2
# 3

for k, v in {"a": 1, "b": 2}.iter():
    print(k)
```

[English](#) | [日本語](#) | [繁體中文](#)

契约式设计 (Design by Contract)

Ry 支持 Eiffel 风格的契约式设计，包含前置条件 (require)、后置条件 (ensure) 和结构体不变量 (invariant)。
契约违反时，程序将以 `exit(1)` 终止。

前置条件 (require)

前置条件在函数入口处检查。它们指定函数被正确调用所需满足的条件。

```
function deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    new_balance: int = balance + amount
    return new_balance
```

若前置条件未满足，程序将输出以下消息并终止：

```
Contract violation: require failed in deposit()
```

后置条件 (ensure)

后置条件在每个 `return` 之前检查。它们指定函数对其返回值的保证。

变量绑定

`ensure` 需要一个变量名来绑定返回值。此变量可在后置条件表达式中使用。

```
function abs(x: int) -> int:
    ensure v:
        v >= 0
    if x < 0:
        return -x
    return x
```

由于 Ry 的函数参数是不可变的，可以在 `ensure` 块中直接引用参数来与入口值比较：

```
function increment(x: int) -> int:
    ensure v:
        v == x + 1
    return x + 1
```

元组解构

对于返回元组的函数，可以用逗号分隔指定多个变量名：

```
function divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

绑定变量的数量必须与元组元素数量一致。

组合示例

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

结构体不变量 (invariant)

不变量是结构体实例必须始终满足的条件。在以下时机检查：- 构造时 - 每次字段赋值后

```
record BankAccount:
  balance: int
  min_balance: int
  invariant:
    balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
a.balance = -1            # Contract violation: invariant failed
```

规则

- `require` 和 `ensure` 块为可选，写在函数体之前。
- 同时使用时，`require` 必须在 `ensure` 之前。
- `ensure` 需要变量绑定来命名返回值（例：`ensure v:`）。
- 对于元组返回值，可指定多个绑定变量（例：`ensure q, r:`）。
- `invariant` 写在 `record` 定义的末尾，所有字段声明之后。
- 所有契约违反以 `exit(1)` 终止程序并输出诊断消息。

[English](#) | [日本語](#) | [简体中文](#)

控制流参考

if / else

语句语法

```
if condition:
    # then 代码块
else:
    # else 代码块 (可省略)
```

表达式形式

if 也可以用作产生值的表达式。支持两种形式：

单表达式形式 (=>)：

```
x = if condition => true_value else false_value
```

示例：

```
abs_val = if x > 0 => x else -x
label = if score >= 90 => "A" else "B"
```

单表达式形式中的 else 分支直接接受一个值 (不需要=>)。两个分支必须产生相同的类型，且 else 是必需的。

块形式 (:)：

```
x = if condition:
    compute_something()
else:
    compute_other()
```

块形式中，每个块必须以表达式语句结尾 (尾表达式语义)。else 分支是必需的，且两个分支必须产生相同的类型。

对于带值的多分支条件式，请改用 case: (见下文)。

条件式的类型

类型	为 false 的值	为 true 的值
bool	false	true
int	0	非 0
float	0.0	非 0

只有 bool、整数和 float 类型可以出现在条件式中。str、List、Map、Set、迭代器、闭包、record、Option 和 Result 不能直接用作条件式。对于集合和字符串，请显式编写长度检查：

```
xs = [1, 2, 3]
# □ 错误：此类型的值不能用作布尔条件
# if xs:
#     print("non-empty")
# □ 显式长度检查
if length(xs) > 0:
    print("non-empty")
# □ 等价的 is_empty 写法
if not is_empty(xs):
    print("non-empty")
```

对于 Option 和 Result，请使用 case 显式对变体进行模式匹配，而不是将它们用作条件式。这些规则同样适用于 while、case 分支和一元 not 运算符。

示例

```
x = 10

if x > 5:
    print("big")
else:
    print("small or equal")
```

作用域规则

- if / else 的各个代码块分别拥有独立的块作用域。
- 在代码块内声明的变量无法从代码块外访问。

```
if true:
    y = 42
# y 在此处无法访问
```

while

语法

```
while condition:
    # 循环体
```

当条件式为 true 时，重复执行循环体。

示例

```
i = 0
while i < 5:
    print(i)
    i += 1
```

搭配 break / continue

```
i = 0
while true:
    if i >= 3:
        break
    i += 1
```

for

语法

```
# 列表 / 集合遍历
for x in iterable_expr:
    # 各元素依次赋值给 x

# range (从 0 开始)
for i in range(n):
    # i = 0, 1, ..., n-1

# range (指定起始与结束)
```

```

for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (指定步长)
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...

```

字符串迭代

对 `str` 进行 `for` 循环会以单字符 `str` 的形式产生每个 **Unicode** 码位。多字节 UTF-8 序列（包括 CJK 字符和表情符号）会被正确解码；多字节字符串内的字节绝不会被分割。

这是码位迭代，不是字素簇迭代：跨多个码位的用户感知字符—组合标记序列（例如基础字母 + U+0301）和 ZWJ 表情符号序列（例如家庭或肤色组合）—会作为多次迭代产生，每个码位一次。如果需要字素簇感知的迭代，请使用未来的分段助手将字符串分解，而不是依赖 `for c in s:`。

```

for c in "hello":
    print(c)          # h, e, l, l, o

for c in "こんにちは":
    print(c)          # こ, ん, に, ち, は （不是单个字节）

for c in "a b":
    print(c)          # a,  , b

```

循环变量的类型是 `str`，因此可以将其传递给其他字符串函数：

```

for c in "abc":
    print(to_upper(c)) # A, B, C

```

迭代空字符串不会执行循环体。`enumerate` 和 `zip` 也接受 `str` 参数并产生相同的码位单位：

```

for i, c in enumerate("abc"):
    print(i, c)

for a, b in zip("abc", "xyz"):
    print(a + b)      # ax, by, cz

```

映射键值遍历

```

for k, v in map_expr:
    # k 为键，v 为各条目的值

```

元组解构

遍历元组列表时，可以将元组解构为 `N` 个变量（需与元组元素数量匹配）。使用 `_` 丢弃值。

```

xs = [10, 20, 30]

for i, x in enumerate(xs):
    print(f"{i}: {x}") # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)      # 11, 22

for _, x in enumerate(xs):
    print(x)          # 丢弃索引

# N 元素解构（3 个以上的变量）

```

```
triples = [(1, 2, 3), (4, 5, 6)]
for a, b, c in triples:
    print(a + b + c)      # 6, 15

for a, _, c in triples:
    print(a + c)          # 4, 10 (丢弃中间元素)
```

范围运算符 (..)

.. 运算符创建包含两端的整数范围。1 .. 5 产生 [1, 2, 3, 4, 5]。

```
for i in 1 .. 5:
    print(i)              # 1 2 3 4 5
```

示例

```
xs = [10, 20, 30]
for x in xs:
    print(x)

s = {1, 2, 3}
for x in s:
    print(x)

for i in range(5):
    print(i)              # 0 1 2 3 4

for i in range(2, 6):
    print(i)              # 2 3 4 5

for i in range(0, 10, 2):
    print(i)              # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)              # 10 7 4 1
```

映射遍历

```
m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)
```

范围运算符

```
for i in 1 .. 3:
    print(i)              # 1 2 3
```

async / await

async function 声明一个并发运行的函数。调用 async function 返回 Task<T>。在另一个 async function 内部使用 await，或从同步上下文中使用 block_on() 等待结果。

```
async function add(a: int, b: int) -> int:
    return a + b
```

```

# 从同步上下文中，使用 block_on()
t: Task<int> = add(20, 22)
print(block_on(t))                # 42
print(block_on(add(1, 2)))        # 3

# 在 async function 内部，使用 await
async function double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2

```

规则

- `async function name(...) -> T`: 使用等待结果类型 `T` 声明。
 - 调用 `async function` 会立即返回 `Task<T>`。
 - `await expr` 要求 `expr` 为 `Task<T>` 并产生 `T`。
 - `await` 只能在 `async function` 内部使用。从同步上下文中使用 `block_on(task)`。
 - `block_on(task)` 阻塞当前线程直到任务完成并返回结果。
 - 支持 `async function ... -> Unit`；当不产生值时，`block_on(task)` 是等待的主要方式。
 - 任务在运行时工作线程池上运行；不是每个任务一个操作系统线程。
 - `v1` 不支持 `async lambda` 和 `async @native function`。
-

@parallel for

`@parallel` 只能附加到使用 `range(...)` 或整数 `..` 范围的计数 `for` 循环上。循环体在运行时工作线程池上以并行块运行。

```

@parallel
for i in range(8):
    print(i)

```

约束

- 仅支持 `range(...)` 和整数 `..` 循环。
- 不支持解构迭代。
- 拒绝对外层可变绑定的赋值。
- 拒绝 `break` 和 `continue`。
- `v1` 中拒绝循环体内的索引赋值和字段赋值。

使用 `available_parallelism()` 查看运行时工作线程数。

break

- 立即跳出最内层的循环 (`while` 或 `for`)。
- 在循环外使用会产生编译错误。

```

for i in range(10):
    if i == 5:
        break    # 在 i == 5 时跳出
    print(i)     # 0 1 2 3 4

```

错误示例

```

# 在循环外使用 break 会产生编译错误
break    # Error: break outside loop

```

continue

- 结束最内层循环的当前迭代，跳至下一次迭代。
- 在循环外使用会产生编译错误。

```
for i in range(5):
    if i == 2:
        continue    # 跳过 i == 2
    print(i)         # 0 1 3 4
```

... (Ellipsis)

- 不执行任何操作的语句 (no-op)。用作空代码块的占位符。
- 可在任何代码块中使用：函数体、if/else、while、for、case 分支等。

```
function not_yet():
    ...

if true:
    ...
else:
    ...
```

case

case 将多分支条件流程（原 when）和模式匹配（原 match）统一为一个构造。支持两种形式：

- case: — 无主题，每个分支是一个条件表达式（取代 when:）
- case <expr>: — 有主题，每个分支是一个模式（取代 match）

两种形式都支持块主体 (:) 和表达式主体 (=>)。

注意：when 和 match 关键字已被移除，转而使用统一的 case 构造。使用 when / match 的旧 Ry 代码必须迁移。

case 无主题

使用 case: 进行无主题值的多分支条件流程。

语法

```
case:
    condition:
        # 主体
    condition:
        # 主体
    _:
        # 兜底主体
```

示例


```

x = 0

case:
  x > 0:
    print("positive")
  x < 0:
    print("negative")
  _:
    print("zero")

```

各分支自上而下求值，仅执行第一个条件为真的分支。通配符分支 `_:` 对于语句形式是可选的。

`case:` 的表达式形式请参见下文的「表达式形式」章节。

case 带主题（模式匹配）

语法

```

case expression:
  pattern:
    # 主体
  pattern if guard_condition:
    # 带守卫的主体
  _:
    # 通配符（匹配任何值）

```

模式的种类

模式	示例	说明
通配符	<code>_</code>	匹配任何值
字面值	<code>0, "hello", true</code>	值的相等比较
变量绑定	<code>n</code>	匹配任何值并绑定到变量
enum 变体	<code>Color::Red</code>	enum 标签的比较（简单 enum）
ADT enum 变体	<code>Shape::Circle(r)</code>	匹配带有关联数据的 enum 变体并绑定
<code>Some(x)</code>	<code>Some(v)</code>	当 Option 有值时，绑定其内容
<code>None</code>	<code>None</code>	当 Option 无值时
<code>Ok(x)</code>	<code>Ok(v)</code>	当 Result 为 Ok 时，绑定其内容
<code>Err(x)</code>	<code>Err(e)</code>	当 Result 为 Err 时，绑定错误值
OR 模式	<code>1 \ 2 \ 3</code>	任一替代方案匹配时即匹配

guard 子句

可以使用 `pattern if condition:` 的形式指定守卫条件。只有当模式匹配且守卫条件为真时，该分支才会被执行。

OR 模式

可以使用 `|` 组合多个模式，任一模式匹配时即匹配。OR 模式中不允许使用变量绑定（`n`、`Some(x)`、`Ok(v)`、`Err(e)`）。

```

case x:
  1 | 2 | 3:
    print("small")
  -:
    print("other")

# enum OR 模式
case color:
  Color::Red | Color::Blue:
    print("warm or cool")
  Color::Green:
    print("green")

```

穷举性检查

- enum 类型：必须覆盖所有变体或包含 `_`。OR 模式中的各替代方案会分别计算。
- Option 类型：必须覆盖 `Some` 和 `None` 或包含 `_`。
- bool 类型：必须覆盖 `true` 和 `false` 或包含 `_`。
- int / float / str 字面值：`_` 为必需。
- 带守卫的分支不计入穷举性。

示例

```

# enum 模式匹配
enum Color:
  Red
  Green
  Blue

case color:
  Color::Red:
    print("red")
  Color::Green:
    print("green")
  Color::Blue:
    print("blue")

# Option 模式匹配
x: Option<int> = Some(42)
case x:
  Some(v):
    print(v)
  None:
    print("nothing")

# Result 模式匹配
function divide(a: int, b: int) -> Result<int, Error>:
  if b == 0:
    return Err(Error("division by zero"))
  return Ok(a // b)

case divide(10, 2):
  Ok(v):
    print(v)          # 5
  Err(e):

```

```

        print(e.message)

# 字面值模式匹配
case x:
    0:
        print("zero")
    1:
        print("one")
    -:
        print("other")

# guard 子句
case x:
    n if n > 0:
        print("positive")
    n if n < 0:
        print("negative")
    -:
        print("zero")

```

ADT enum 模式匹配

当 enum 变体携带关联数据时，使用绑定模式来提取值。

```

enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

s = Shape::Circle(3.14)
case s:
    Shape::Circle(r):
        print(r)          # 3.14
    Shape::Rectangle(w, h):
        print(w)
        print(h)
    Shape::Point:
        print("point")

```

多字段变体会按声明顺序将各字段绑定到不同的名称。

表达式形式

case: 和 case <expr>: 都可以作为表达式使用，将各分支中的 : 替换为=>。每个分支提供一个单一表达式，其值成为结果。

```

# case: 表达式（无主题）
label = case:
    x > 100 => "huge"
    x > 10  => "big"
    x > 0   => "small"
    -      => "non-positive"

```

模式匹配表达式形式：

语法

```

result = case expression:
    pattern => value_expression
    pattern if guard => value_expression
    _ => default_value

```

match 语句中支持的所有模式在 match 表达式中同样支持：字面值、变量绑定、enum、ADT enum、Some/None、Ok/Err、OR 模式、守卫和通配符。

match 表达式必须是穷举的（与 match 语句的规则相同）。

示例

```

# Option
value = case opt:
    Some(v) => v
    None    => 0

# Enum
label = case direction:
    Direction::North => "N"
    Direction::South => "S"
    Direction::East  => "E"
    Direction::West  => "W"

# Guard
grade = case score:
    n if n >= 90 => "A"
    n if n >= 80 => "B"
    -            => "F"

# OR 模式
kind = case x:
    1 | 2 | 3 => "small"
    -        => "large"

# ADT enum
area = case shape:
    Shape::Circle(r) => 3.14 * r * r
    Shape::Rect(w, h) => w * h
    Shape::Point      => 0.0

```

作用域规则

- 各 case 分支拥有块作用域。
- 通过变量绑定模式 (n)、Some(x)、Ok(v) 或 Err(e) 绑定的变量仅在该分支内有效。

作用域规则

块作用域

- if / else / while / for / case 的各代码块拥有块作用域。
- 在代码块内声明的变量会在代码块结束时离开作用域。

```

for i in range(3):
    tmp = i * 2

```

```
# tmp 在此处无法访问
```

```
if true:
```

```
    a = 1
```

```
# a 在此处无法访问
```

内层作用域的重新赋值

- 在内层作用域中对变量赋值会修改外层的变量（Python 风格的作用域）。
- 不会产生遮蔽——内层的赋值会修改同一个变量。

```
x = 10
```

```
if true:
```

```
    x = 99    # 修改外层的 x
```

```
    print(x)    # 99
```

```
print(x)    # 99
```

[English](#) | [日本語](#) | [繁體中文](#)

指令

指令是可以附加到声明上的编译时元数据注解。使用 `@name` 语法，类似于 Java 注解。

语法

```
@name
```

```
@name(key=value, ...)
```

指令放置在目标声明之前。可以堆叠多个指令。

支持的目标

指令可以应用到以下声明:

- `function` - 函数定义（包括用 `@it` / `@describe` 装饰的具名测试函数）
- `record` - 记录定义
- 变量声明（使用 `@const` 或普通赋值）
- `record` 定义内的字段
- `for` - 仅限计数循环，用于 `@parallel`
- `it` / `describe` 调用（旧 `lambda` 形式）- 用于 `@each` 和 `@property` 的测试用例和测试组

内建指令

`@deprecated`

将声明标记为已弃用。当已弃用的实体被使用（调用、引用或访问）时，会发出编译时警告。

应用于函数:

```
@deprecated
```

```
function old_function() -> int:
```

```
    return 42
```

```
print(old_function())    # warning: 'old_function' is deprecated
```

应用于类型:

```
@deprecated
record OldPoint:
    x: int
    y: int
```

```
@const
p = OldPoint(1, 2) # warning: 'OldPoint' is deprecated
```

应用于变量:

```
@deprecated
@const
old_value = 99
```

```
print(old_value) # warning: 'old_value' is deprecated
```

应用于字段:

```
record Config:
    @deprecated
    old_setting: int
    new_setting: int
```

```
@const
c = Config(1, 2)
print(c.old_setting) # warning: 'Config.old_setting' is deprecated
print(c.new_setting) # no warning
```

@const

将变量标记为不可变。使用 @const 声明的变量在初始化后无法重新赋值。未使用 @const 时，变量默认为可变。

```
@const
x = 42
# x = 10 # Error: cannot reassign @const variable
```

搭配类型注解:

```
@const
name: str = "hello"
```

元组解构:

```
@const
a, b = (1, 2)
```

顶层 @const 和函数。顶层 @const 声明对同一源文件中之后定义的任何顶层函数都可见，并且不可变性会对每次引用强制执行——包括通过顶层 @const 记录进行的字段变更。详细信息请参阅 [functions.md](#) 中的「函数体内的顶层变量与 @const」章节。

@native

声明由运行时提供实现的函数。该函数不能有函数体。

可选的字符串参数指定共享库模块名称。当调用 @native("libname") 函数时，JIT 会动态加载相应的共享库（macOS 上为 `library_<libname>.dylib`，Linux 上为 `library_<libname>.so`）并从中解析运行时符号：

```
@native # 内建（静态链接到进程中）
@native("base64") # 从 library_base64.dylib/.so 动态加载
```

基本语法:

```
@native
function contains(string: str, substring: str) -> bool

print(contains("hello world", "world")) # true
```

运算符重载:

```
@native
function operator+(a: str, b: str) -> str

print("hello" + " world") # hello world
```

与 UFCS 搭配使用:

```
@native
function to_upper(string: str) -> str

print("hello".to_upper()) # HELLO
```

参数数量验证:

当 @native 声明包含类型签名时，编译器会在调用处验证参数数量。支持重载函数（例如：1、2、3 个参数的 range），只要任一重载匹配即通过验证。

```
@native
function range(n: int) -> List<int>
@native
function range(start: int, end: int) -> List<int>

print(length(range(5)))      # OK: matches 1-arg overload
print(length(range(1, 10)))  # OK: matches 2-arg overload
print(length(range()))       # Error: expects 1 or 2 argument(s), but got 0
```

标准库声明 (core/):

core/ 目录包含所有内建函数的 @native 声明，按类别组织：

文件	内容
core/builtins.ry	print, length, range, enumerate, zip, exit, args, available_parallelism, sleep
core/str.ry	contains, starts_with, ends_with, find, substring, char_at, replace, to_upper, to_lower, trim, trim_start, trim_end, repeat, reverse, split, join
core/convert.ry	to_int, to_float, to_str
core/list.ry	append, pop, insert, remove_at, slice, distinct, flatten, sort, first, last, is_empty
core/map.ry	keys, values, items, has_key, get, merge
core/set.ry	add, remove, union, intersection, difference, symmetric_difference, is_subset, is_superset
core/higher_order.ry	filter, map, reduce, fold, any, all, sum, min, max

当 ry 可执行文件附近存在 core/ 目录时，这些文件会作为前置自动加载。前置机制使得内建函数调用时的参数数量验证生效。

约束: - @native 函数不能有函数体（签名后不能加:）- 提供函数体会导致解析错误: @native function must not have a body - 对于裸@native，声明的函数必须对应到现有的内建函数，否则调用时会在编译时失败。对于@native("libname")，函数会基于声明的签名编译，如果无法从加载的库中解析符号，将在 JIT 链接时失败。

库规范: - @native("libname") 指定原生函数位于名为 libry_<libname>.dylib(macOS)或 libry_<libname>.so (Linux) 的共享库中。在 JIT 启动时, 会按以下搜索路径顺序加载所需的共享库: 1. exe/./lib/ — 已安装布局 2. exe/lib/ — 开发/构建布局 3. \$RY_HOME/lib/ — 用户安装环境 - @native (静态) 和 @native("libname") (动态) 声明都会注册以进行参数数量验证和调用解析。区别仅在于运行时符号如何提供给 JIT。运行时函数名称遵循 __ry_<libname>_<fn_name> 约定 (例如 @native("base64") fn encode(...) → __ry_base64_encode)。这适用于标准库包和用户定义的原生库。

@parallel

将计数 for 循环标记为并行执行。

```
@parallel
for i in range(8):
    work(i)
```

支持的目标:

- 仅限 for 语句

约束:

- 一个 for 语句上只允许使用一个 @parallel 指令。
- 可迭代对象必须是 range(...) 或整数 .. 范围。
- 不支持解构迭代。
- 禁止对外部可变量赋值。
- v1 中禁止在循环体内使用 break、continue、索引赋值和字段赋值。

@each

启用参数化测试, 以不同参数多次运行测试。

语法 (具名函数, 推荐):

```
@each([(arg1, arg2, ...), ...])
@it("should handle {0} and {1}")
function test_handle(param1: type, param2: type):
    # test body
```

语法 (旧 it lambda):

```
@each([(arg1, arg2, ...), ...])
it("should handle {0} and {1}", (param1: type, param2: type):
    # test body
)
```

参数可以是任何求值为元组列表的表达式, 包括函数调用:

```
@each(make_inputs())
@it("should handle {0}")
function test_handle(x: int):
    # test body
```

支持的目标: 用 @it 装饰的函数, 或旧的 it 调用。

约束: - 参数必须求值为元组列表 - 元组的元素数量必须与函数参数数量匹配 - 描述字符串中的 {0}, {1}, ... 会被替换为参数的字符串表示

@property

启用基于属性的测试, 为测试生成随机输入。

语法 (具名函数, 推荐):


```
@property(count=100)
@it("should verify property name")
function test_property(a: int, b: int):
    # test body with random values
```

语法 (旧 `it lambda`) :

```
@property(count=100)
it("should verify property name", (a: int, b: int):
    # test body with random values
)
```

支持的目标: 用 `@it` 装饰的函数, 或旧的 `it` 调用。

参数:

参数	类型	默认值	说明
<code>count</code>	<code>int</code>	100	随机试验次数

支持的参数类型:

类型	范围
<code>int</code>	-1000 到 1000
<code>float</code>	-1000.0 到 1000.0
<code>bool</code>	true 或 false
<code>str</code>	随机 ASCII、0-20 字符

失败时会显示反例 (导致失败的参数值)。

@it

通过装饰具名函数来声明测试用例。函数体成为测试体, 由 `ry test` 执行。完整规范请参阅 [测试参考](#)。

语法:

```
@it("description")
function test_name():
    # 断言
```

基本示例:

```
@it("should add 1 + 2 = 3")
function test_add():
    expect(1 + 2).to_eq(3)
```

与 `@each` 或 `@property` 组合:

```
@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
@it("should add {0} + {1} = {2}")
function test_add_each(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
```

```
@property(count=100)
@it("should verify addition is commutative")
function test_commutative(a: int, b: int):
    expect(a + b).to_eq(b + a)
```

支持的目标: 仅 function 声明。函数不能有返回类型注解。

约束: - 仅在用 `ry test` 执行的 `*.test.ry` 文件中有效 - 与 `@each` 组合时, 函数的参数列表必须与元组元素数量匹配 - 与 `@property` 组合时, 每个参数类型必须是支持的生成器类型之一 (`int`、`float`、`bool`、`str`)

`@describe`

通过装饰具名函数将一组相关测试分组。在主体中声明的内部 `@it` 函数属于该组, 主体中直接声明的变量充当被每个内部 `@it` 捕获的共享设置。与旧 `lambda` 形式不同, `@describe` 组可以嵌套; 输出按嵌套深度成比例缩进。

语法:

```
@describe("group name")
function group_name():
    @it("nested test")
    function test_nested():
        # 断言
```

基本示例:

```
@describe("arithmetic")
function arithmetic_tests():
    @it("should subtract")
    function test_sub():
        expect(10 - 3).to_eq(7)

    @it("should multiply")
    function test_mul():
        expect(4 * 5).to_eq(20)
```

共享设置:

外层 `@describe` 主体中声明的变量会被每个内部 `@it` 函数自动捕获。

```
@describe("shared setup")
function shared_setup_tests():
    base = 100
    offset = 5

    @it("should use base")
    function test_base():
        expect(base).to_eq(100)

    @it("should use base and offset")
    function test_combined():
        expect(base + offset).to_eq(105)
```

嵌套组:

```
@describe("outer")
function outer():
    @describe("inner")
    function inner():
        @it("should pass deeply nested test")
        function test_deep():
            expect(1 + 1).to_eq(2)
```

支持的目标: 仅 function 声明。函数不能有参数或返回类型注解。

@inline

为 LLVM 优化器提供内联提示。默认情况下，标记函数进行积极内联。

基本用法（始终内联）：

```
@inline
function add(a: int, b: int) -> int:
    return a + b
```

带 mode 参数：

```
@inline(mode="always")
function hot_path(x: int) -> int:
    return x * 2 + 1
```

```
@inline(mode="hint")
function medium_path(x: int) -> int:
    return x + 1
```

```
@inline(mode="never")
function cold_error_handler(msg: str):
    print("ERROR: " + msg)
```

模式：

模式	LLVM 属性	说明
always（默认）	AlwaysInline	始终内联此函数
hint	InlineHint	向优化器建议内联
never	NoInline	禁止内联此函数

约束：- @inline 不能与 @native 一起使用（native 函数没有可内联的函数体）。- 未知的 mode 值会导致编译错误。

参数（未来扩展）

指令支持可选的参数语法，为未来扩展做准备：

```
@deprecated(reason="use new_api instead")
function old_api() -> int:
    return 0
```

目前，参数会被解析但不会被 @deprecated 指令使用。

注意事项

- 已弃用的实体仍然正常运作，仅会发出警告。
- 警告在使用点发出，不在定义点发出。
- 定义已弃用的实体但不使用它，不会产生警告。
- 未知的指令名称会导致解析错误。
- 在不支持的目标（如 if、while）上使用指令会导致解析错误。@parallel 是唯一支持在 for 上使用的指令。

[English](#) | [日本語](#) | [简体中文](#)

错误参考

错误格式

ry 以 Rust 风格的丰富格式显示编译错误，显示错误的确切位置及源代码上下文：

```
error: cannot reassign @const variable: x
  --> main.ry:5:1
  |
5 | x = 10
  | ^ cannot reassign @const variable: x
```

每条错误消息包含：- 错误等级和消息 (error: ...) - 文件位置 (--> 文件: 行: 列) - 源代码行 (相关代码) - 插入符号指示器 (^) 指向确切的列位置

编译错误

错误内容	原因	示例
使用未声明的变量 对 @const 变量重新赋值	引用了未声明的变量 对以 @const 声明的变量重新赋值	print(x) (x 未声明) @const x = 1 -> x = 2
重新声明同名变量	在同一作用域重新声明同名变量	x = 1 -> 同名再声明
变量类型变更赋值 类型标注不一致	对变量赋值不同类型的值 声明时的类型与赋值的类型不同	x = 1 -> x = 3.14 x: int = 3.14
重载返回类型冲突	定义了参数类型相同但仅返回类型不同的重载	参数 (int, int) 但返回类型分别为 int 和 float 的两个函数
重载解析失败	不存在与参数类型匹配的重载	function add(a: int, b: int) 却调用 add(1.0, 2.0)
对位运算传入 float	对&、\ 、^、~、<<、>> 传入 float 类型	3.14 & 1
空列表	无法从空列表面值 [] 推断类型	xs = []
空映射	无法从空映射面值 {} 推断类型	m = {}
元组超出范围的索引 在循环外使用 break/continue	访问元组中不存在的索引 在 for/while 循环外使用 break 或 continue	t = (1, 2) -> t.2 在函数顶层使用 break
在块内导入模块	在函数或条件块内使用 from 语句	在函数内使用 from math
循环导入	模块之间互相导入	a.ry 导入 b.ry, b.ry 导入 a.ry
相同字段名重复	在结构体内定义了两次相同的字段名	在 type T: x: int 中定义了两个 x
非穷尽 match	case 未覆盖所有模式	enum 的部分变体未覆盖、Option 缺少 None、Result 缺少 Ok/Err、面值缺少 _
? 用于非 Result 类型	对非 Result 类型的表达式使用?	x = 42 -> x?

错误内容	原因	示例
? 用于非 Result 函数	在不返回 Result 的函数中使用?	function foo() -> int: -> bar()?

编译错误示例

```
# 对 @const 变量重新赋值
@const
x = 10
x = 20  # 错误

# 类型变更赋值
n = 1
n = "hello"  # 错误：对 int 类型赋值 str 类型

# 空列表
xs = []  # 错误：无法推断类型

# 在循环外使用 break
break  # 错误：在循环外

# 在块内导入
function foo():
    from math  # 错误：仅能在顶层导入

# 相同字段名重复
record Bad:
    x: int
    x: float  # 错误：x 重复
```

运行时错误

错误内容	原因	示例
列表超出范围的访问	列表的索引超出范围	xs = [1, 2, 3] -> xs[5]
映射不存在的键访问	引用映射中不存在的键	m = {"a": 1} -> m["b"]
契约违反	require、ensure 或 invariant 条件评估为 false	参阅 契约式设计

所有运行时错误都会以 `exit(1)` 终止程序。

运行时错误示例

```
# 列表超出范围的访问
xs = [1, 2, 3]
print(xs[10])  # 运行时错误：exit(1)

# 映射不存在的键访问
m = {"a": 1}
print(m["z"])  # 运行时错误：exit(1)
```

文件系统函数参考

文件和目录操作。所有函数需要从 `filesystem` 显式导入。

`filesystem` 包处理文件和目录本身的操作（复制、移动、删除等），而 `io` 包处理文件内容的读写。

```
from filesystem import list_dir, walk, glob_files, copy, move, remove, remove_all
from filesystem import make_dir, make_dir_all, file_size, is_file, is_dir, is_symlink
from filesystem import chmod, symlink, read_link
```

函数一览

函数	签名	说明
<code>list_dir</code>	<code>(str) -> Result<List<str>, Error></code>	列出目录中的条目（非递归）
<code>walk</code>	<code>(str) -> Result<List<str>, Error></code>	递归列出所有文件和目录
<code>glob_files</code>	<code>(str) -> Result<List<str>, Error></code>	查找匹配 <code>glob</code> 模式的文件
<code>copy</code>	<code>(str, str) -> Result<Unit, Error></code>	复制文件
<code>move</code>	<code>(str, str) -> Result<Unit, Error></code>	移动或重命名文件或目录
<code>remove</code>	<code>(str) -> Result<Unit, Error></code>	删除文件或空目录
<code>remove_all</code>	<code>(str) -> Result<Unit, Error></code>	递归删除文件或目录树
<code>make_dir</code>	<code>(str) -> Result<Unit, Error></code>	创建单个目录
<code>make_dir_all</code>	<code>(str) -> Result<Unit, Error></code>	创建目录及所有缺失的父目录
<code>file_size</code>	<code>(str) -> Result<int, Error></code>	返回文件大小（字节）
<code>is_file</code>	<code>(str) -> bool</code>	检查路径是否为普通文件
<code>is_dir</code>	<code>(str) -> bool</code>	检查路径是否为目录
<code>is_symlink</code>	<code>(str) -> bool</code>	检查路径是否为符号链接
<code>chmod</code>	<code>(str, int) -> Result<Unit, Error></code>	更改文件权限（POSIX 模式）
<code>symlink</code>	<code>(str, str) -> Result<Unit, Error></code>	创建符号链接
<code>read_link</code>	<code>(str) -> Result<str, Error></code>	读取符号链接的目标

示例

目录操作

```
from filesystem import make_dir, make_dir_all, list_dir, remove_all
```

```
# 创建单个目录
```

```
case make_dir("/tmp/myapp"):
```

```
    Ok(_):
```

```
        print("created")
```

```
    Err(e):
```

```

    print("error: " + e.message)

# 创建嵌套目录 (类似 mkdir -p)
make_dir_all("/tmp/myapp/data/logs")

# 列出目录内容
case list_dir("/tmp/myapp"):
    Ok(entries):
        for entry in entries:
            print(entry)
    Err(e):
        print("error: " + e.message)

# 删除目录树 (类似 rm -rf)
remove_all("/tmp/myapp")

```

文件操作

```

from filesystem import copy, move, remove, file_size
from io import write_text

write_text("/tmp/hello.txt", "Hello, World!")

# 复制文件
copy("/tmp/hello.txt", "/tmp/hello_copy.txt")

# 获取文件大小
case file_size("/tmp/hello.txt"):
    Ok(sz):
        print("size: " + to_str(sz))
    Err(e):
        print("error: " + e.message)

# 移动/重命名文件
move("/tmp/hello_copy.txt", "/tmp/renamed.txt")

# 删除文件
remove("/tmp/renamed.txt")

```

递归遍历

```

from filesystem import walk, glob_files

# 递归遍历目录树 (类似 find)
case walk("/var/log"):
    Ok(files):
        for f in files:
            print(f)
    Err(e):
        print("error: " + e.message)

# Glob 模式匹配
case glob_files("/var/log/*.log"):
    Ok(matches):
        for m in matches:

```

```
    print(m)
Err(e):
    print("error: " + e.message)
```

路径类型检查

```
from filesystem import is_file, is_dir, is_symlink

if is_file("/etc/hosts"):
    print("regular file")

if is_dir("/tmp"):
    print("directory")

if is_symlink("/usr/local/bin/python"):
    print("symbolic link")
```

符号链接

```
from filesystem import symlink, read_link, is_symlink

# 创建符号链接
symlink("/usr/local/bin/ry", "/tmp/ry_link")

# 检查并读取符号链接
if is_symlink("/tmp/ry_link"):
    case read_link("/tmp/ry_link"):
        Ok(target):
            print("points to: " + target)
        Err(e):
            print("error: " + e.message)
```

权限

```
from filesystem import chmod

# chmod 755 (rwxr-xr-x) —— 使用十进制值: 0o755 = 493
chmod("/tmp/script.sh", 493)

# chmod 644 (rw-r--r--) —— 0o644 = 420
chmod("/tmp/data.txt", 420)
```

注意事项

- `is_file`、`is_dir` 和 `is_symlink` 在出错时（例如路径不存在）返回 `false`
- `is_file` 和 `is_dir` 会跟随符号链接；`is_symlink` 使用 `lstat` 检测链接
- `list_dir` 仅返回条目名称（不包含完整路径）
- `walk` 返回所有条目（文件和目录）的完整路径
- `glob_files` 在没有文件匹配模式时返回空列表（而非错误）
- `remove` 对非空目录会失败；使用 `remove_all` 进行递归删除

[English](#) | [日本語](#) | [简体中文](#)

函数参考

函数定义语法

```
function function_name(param_name: type, ...) -> return_type:
```

```
    # body
    return value
```

- 参数类型可省略。省略时视为 `any` 类型。
- 返回类型可省略。省略时会从函数体推断（命名函数和 `lambda` 均如此）。若无 `return` 语句则推断为 `Unit`。若要显式允许任意返回类型，请指定 `-> any`。
- 函数体为缩进的代码块。
- 具有显式返回类型（`Unit` 和 `any` 除外）的函数，必须在所有控制流路径中包含 `return` 语句。若缺少则会产生编译错误。
- 函数可以定义 `require`（前置条件）和 `ensure`（后置条件）子句。参阅[契约式设计](#)。

命名约定：函数名称和参数名称必须使用 `snake_case`（如 `add`、`get_value`、`map_list`）。编译器会强制执行此约定。

```
function add(a: int, b: int) -> int:
```

```
    return a + b
```

```
function greet(name: str) -> Unit:
```

```
    print("Hello, " + name)    # 返回类型为 Unit（显式）
```

参数与返回值的类型

项目	说明
参数类型	可省略。省略： <code>type</code> 时默认为 <code>any</code>
返回类型	可省略。省略时从函数体推断（若无 <code>return</code> 语句则为 <code>Unit</code> ）
<code>Unit</code>	不返回值的函数的返回类型

注意：函数参数是不可变的。不能在函数体内对参数重新赋值。这确保了入口时的参数值始终可用于后置条件检查（参阅[契约式设计](#)）。

```
function no_return(x: int) -> Unit:    # 返回类型 Unit（显式）
```

```
    print(x)
```

```
function get_value() -> int:          # 返回类型 int
```

```
    return 42
```

```
function identity(x) -> any:          # 参数类型 any（省略）
```

```
    return x
```

类型省略与 `any`

当参数的类型标注被省略时，该参数被视为 `any` ——一种在运行时接受任意基本值的动态类型。这类似于 Python 的无类型参数。

```
# 所有参数默认为 any
```

```
function add(a, b):
```

```
    return a + b
```

```
add(1, 2)           # 3 (int + int)
add("hello", " world") # "hello world" (str + str)
add(1, 2.0)         # 3.0 (int + float)
```

也可以在类型标注中显式使用 `any` :

```
function identity(x: any) -> any:
    return x
```

返回类型推断

当返回类型省略时，会从函数体中的 `return` 语句推断：

```
function double(x: int):      # 返回类型推断为 int
    return x * 2

function greet(name: str):    # 返回类型推断为 Unit (无 return)
    print("Hello, " + name)
```

若要显式允许任意返回类型，请使用 `-> any` :

```
function flexible(x: any) -> any:
    return x    # 可以返回 int、float、str 等
```

嵌套函数

函数可以在其他函数内部定义。嵌套函数仅在其外层函数的作用域内可见 — 不能从外部调用。

```
function outer() -> int:
    function helper() -> int:
        return 42
    return helper()
```

```
outer()    # 42
# helper() # 错误：未定义的函数
```

兄弟作用域中同名的嵌套函数不会冲突：

```
function foo() -> int:
    function helper() -> int:
        return 1
    return helper()
```

```
function bar() -> int:
    function helper() -> int:
        return 2
    return helper()
```

```
foo()    # 1
bar()    # 2
```

嵌套函数可以作为值使用并传递给高阶函数。同一作用域内嵌套函数之间的相互递归也能正常工作（编译器会前向声明它们）。

闭包捕获

嵌套的具名函数可以从外层作用域捕获变量，与 `lambda` 一样。当嵌套函数引用外层变量时，它会成为闭包：

```
function make_adder(base: int) -> function(int) -> int:
    function add(x: int) -> int:
        return x + base
    return add

add10 = make_adder(10)
add10(5)    # 15
```

捕获规则：

- 捕获是按值的（与 lambda 相同）。值在闭包创建时被复制。
- 捕获的变量不能在嵌套函数体内重新赋值。
- ARC 管理的值（字符串、列表等）会被正确地保留和释放。
- 如果嵌套函数没有捕获，它仍然是普通函数指针（无开销）。
- 多层捕获有效：深层嵌套的函数可以引用任何外层作用域的变量。

递归

函数可以调用自身。

```
function factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

互递归

函数可以相互调用，无论定义顺序如何。编译器在处理函数体之前会预先声明具有显式返回类型的顶层函数，前提是所有引用的类型已知（基本类型始终可用；record/enum 类型必须在文件中先定义）。

```
function is_even(n: int) -> bool:
    if n == 0:
        return true
    return is_odd(n - 1)    # 调用下方定义的 is_odd

function is_odd(n: int) -> bool:
    if n == 0:
        return false
    return is_even(n - 1)   # 调用上方定义的 is_even
```

前向引用的要求：

- 函数必须具有显式返回类型标注（-> type）。返回类型推断的函数不能被前向引用。
- 函数必须在顶层或在另一个函数体内部定义。前向引用在同一作用域级别内有效。
- 所有参数和返回类型必须在前向声明点可解析（例如，record 类型必须在使用它们的函数之前定义）。

函数体内的顶层变量与@const

顶层 let 绑定和 @const 声明对任何顶层函数 —包括这些函数内的嵌套函数和 lambda— 都可见，只要声明在同一源文件中文本上先于引用函数出现。

```
@const
PI: float = 3.14

@const
MAX_RETRIES: int = 5
```

```

counter: int = 0

function area(radius: float) -> float:
    return PI * radius * radius          # 读取顶层 @const

function clamp_retries(n: int) -> int:
    if n > MAX_RETRIES:
        return MAX_RETRIES
    return n

function bump():
    counter = counter + 1                # 写入顶层可变 `let`

```

规则:

- 严格按源代码顺序。函数体不能引用同一文件中之后声明的顶层绑定。请将绑定移到函数上方，或将绑定包装在延迟调用的辅助函数中。
- `@const` 是只读的。重新赋值或字段变更（顶层 `@const P: Point` 的 `P.x = 99`）会在编译时被拒绝。
- 可变 `let` 写入是写穿透的。从函数内部对顶层可变量赋值实际上会变更顶层绑定——它不会创建一个同名的本地变量。
- 嵌套块不是模块级的。在顶层 `if`、`while` 或 `for` 块内的 `let` 是该块的本地变量，从函数中不可见。

限制 (v0.0.8) :

- 并行 `for` 块不能对顶层可变量赋值（避免数据竞争）。
- 顶层 `weak` 引用和资源类型绑定（文件/正则句柄）尚不能从函数体内访问——如果你需要它们，请在后续 issue 中跟踪这些用例。

尾调用优化

编译器会自动检测自递归尾调用——即函数的最后一个操作是调用自身——并应用 LLVM 的 `musttail` 优化。这保证了尾递归函数使用常量栈空间，防止深度递归时的栈溢出。

```

function sum_to(n: int, acc: int) -> int:
    if n <= 0:
        return acc
    return sum_to(n - 1, acc + n)      # 尾调用 → 被优化

```

```
sum_to(1000000, 0)      # 不会栈溢出
```

尾调用优化的条件:

- 函数在 `return` 语句中直接调用自身 (`return f(args)`)
- 调用结果不经过任何进一步计算直接返回 (`return n * f(n-1)` 不是尾调用)
- 函数没有 `ensure` (后置条件) 子句

互递归 (A 调用 B, B 调用 A) 目前不会被优化为尾调用。

重载

可以定义参数数量或类型不同的同名函数。

规则

- 参数的数量或类型不同即可定义同名函数。
- 调用时会根据参数的类型和数量选择适当的函数。
- 仅返回类型不同的重载是不允许的。

```
function area(side: int) -> int:
    return side * side

function area(w: int, h: int) -> int:
    return w * h

a = area(5)          # 25
b = area(3, 4)       # 12
```

解析优先级

当多个重载匹配一个调用时，编译器使用以下优先级（从高到低）选择最具体的重载：

1. 精确类型匹配 — 参数类型与形参类型完全匹配
2. 隐式拓宽 — 安全的拓宽转换 ($u8 \rightarrow int$ 、 $u8 \rightarrow float$ 、 $int \rightarrow float$)
3. **union** 类型匹配 — 参数类型是 union 形参类型的成员
4. **any** 类型匹配 — 形参类型是 any（接受任何值）

精确匹配数量最多的重载胜出。如果两个或更多重载具有相同的具体程度，编译器会报告歧义错误。

低级数值类型（i8、i16、i32、i64、u8、u64、f32）不参与隐式拓宽——需要显式 as 转换。

```
function process(x: int) -> str:
    return "int"

function process(x) -> str:          # x: any
    return "any"

process(42)          # "int" — 精确匹配 (int) 优于 any
process("hello")     # "any" — str 没有精确匹配，回退到 any

function double(x: float) -> float:
    return x * 2.0

double(5)            # OK — int 隐式拓宽为 float，返回 10.0
```

默认参数

参数可以有默认值，允许调用者省略尾部的参数。

语法

```
function connect(host: str, port: int = 8080, timeout: int = 30):
    # ...

connect("localhost")          # port=8080, timeout=30
connect("localhost", 3000)    # port=3000, timeout=30
connect("localhost", 3000, 10000) # port=3000, timeout=10000
```

规则

- 默认参数必须放在所有非默认参数之后。
- 有默认值的参数必须有显式的类型标注（例如：`x: int = 10`；`x = 10` 是编译错误）。
- 默认值必须是编译时常量表达式（字面量和 `@const` 变量）。
- 如果默认参数导致模糊的重载（参数数量范围重叠且类型匹配），编译器会报告错误。

```
# 错误：模糊的重载
function calc(x: int, y: int = 0) -> int:
    return x + y
function calc(x: int) -> int:          # 与上面的 calc(int) 冲突
    return x * 2
```

限制

- 泛型函数和 lambda 表达式不支持默认参数。

Unit 类型函数

不返回值的函数会返回 Unit。返回类型可以省略（推断为 Unit）或用 `-> Unit` 显式指定。

```
function log(msg: str) -> Unit:
    print(msg)
```

Task 与异步函数

`Task<T>` 是用于并发工作的内置句柄类型。`async function` 返回 `Task<T>`，`await` 在另一个 `async function` 内部提取 `T`，`block_on(task)` 从同步上下文中阻塞直到任务完成。

```
async function add(a: int, b: int) -> int:
    return a + b

# 从同步上下文中，使用 block_on()
t: Task<int> = add(20, 22)
print(block_on(t))          # 42
block_on(add(1, 2))         # 等待并丢弃结果

# 在 async function 内部，使用 await
async function double_add(a: int, b: int) -> int:
    return (await add(a, b)) * 2
```

规则

- `async function name(...) -> T`: 使用等待结果类型 `T` 声明。
- 调用 `async function` 会立即返回 `Task<T>`。
- `await expr` 要求 `expr` 为 `Task<T>` 并产生 `T`。
- `await` 只能在 `async function` 内部使用。从同步上下文中使用 `block_on(task)`。
- `block_on(task)` 阻塞当前线程直到任务完成并返回结果。
- 支持 `async function ... -> Unit`；当不产生值时，`block_on(task)` 是等待的主要方式。
- 任务在运行时工作线程池上运行；不是每个任务一个操作系统线程。
- v1 不支持 `async lambda` 和 `async @native function`。

Lambda 函数

可以内联定义匿名函数。

语法

```
# 单一表达式（返回类型从表达式推断）
(param_name: type, ...) => expression

# 参数类型可省略（默认为 any）
(param_name, ...) => expression

# 多行代码块
(param_name: type, ...):
    # 多个语句
    return value

# 带显式返回类型（可选）
(param_name: type, ...) -> return_type => expression
```

示例

```
double = (x: int) => x * 2
result = double(5)    # 10

add = (a: int, b: int) => a + b
sum = add(3, 4)       # 7

# 多行 lambda
abs = (x: int):
    if x < 0:
        return -x
    return x
```

闭包

Lambda 函数会以值捕获定义时外层作用域的变量。闭包在捕获时获得自己的独立副本，并且捕获的变量不能在闭包内重新赋值。

外层更改不影响闭包

由于闭包持有副本，定义闭包后重新赋值原始变量不会影响捕获的值：

```
base = 10
add_base = (x: int) => x + base    # 以值捕获 base (10 的副本)

base = 99                        # 不影响已捕获的值
r = add_base(5)                  # 15 (使用捕获时的 base = 10)
```

捕获的变量实际上是 final 的

捕获的变量不能在闭包内重新赋值。尝试这样做会产生编译错误：

```
counter = 0
inc = ():
    counter += 1    # 编译错误：不能在闭包内修改捕获的变量 'counter'

inc()
```

捕获的记录上的字段赋值是允许的，因为它修改的是副本的内部状态，而不是重新赋值变量本身：

```
record Point:
    x: int
    y: int

p = Point(0, 0)
move = ():
    p.x = p.x + 1    # OK — 修改捕获副本的字段
```

注意：字段修改仅适用于闭包的副本 — 外层变量不受影响。

捕获规则

项目	内容
捕获方式	值捕获（复制）
捕获时机	Lambda 定义时
捕获变量的重新赋值	不允许（编译错误）
捕获记录上的字段赋值	允许（仅修改副本）
外层变量修改的影响	无（闭包持有自己的副本）

Python/JavaScript 用户注意：在 JavaScript 中，闭包以引用捕获变量，因此对捕获变量的更改会反映在闭包外部。在 Python 中，闭包可以访问外层变量，重新绑定外层名称（如 `counter += x`）需要声明 `nonlocal`。在 Ry 中，闭包始终以值捕获，且捕获的变量实际上是 `final` 的 — 它们不能在闭包内重新赋值。这是有意为之——确保安全性和可预测性，尤其在并发或高阶上下文中。

函数类型

用于将函数作为值处理的类型。

语法

```
function(param_type1, param_type2, ...) -> return_type
```

示例

```
f: function(int) -> int = (x: int) => x * 2
g: function(int, int) -> int = (a: int, b: int) => a + b

function apply(func: function(int) -> int, x: int) -> int:
    return func(x)

result = apply(f, 5)    # 10
```

字符串表示

`print()`、`to_str()` 和 f-string 内插都会为函数值产生 "`<closure>`"：

```
f = (x: int) => x + 1
print(f)                # <closure>
s = to_str(f)            # "<closure>"
msg = f"fn={f}"          # "fn=<closure>"
```

注意：闭包之间的相等比较 (`==` / `!=`) 不被支持，会产生编译时错误。

高阶函数

可以接收函数作为参数，或将函数作为返回值返回。

```
function map_list(xs: List<int>, f: function(int) -> int) -> List<int>:
    result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

doubled = map_list([1, 2, 3], (x: int) => x * 2)
# [2, 4, 6]
```

泛型函数

函数可以有类型参数，实现类型安全的复用而无需代码重复。

语法

```
function name<T, U>(param1: T, param2: U) -> T:
    # 使用 T、U 作为类型的函数体
```

示例

```
function identity<T>(x: T) -> T:
    return x

# 显式类型参数
result = identity[int](42)      # 42
result = identity[str]("hello") # "hello"

# 类型推断（从实际参数推导类型参数）
result = identity(42)           # T = int, result = 42
result = identity("hello")      # T = str, result = "hello"
```

多个类型参数

```
function pick_first<T, U>(a: T, b: U) -> T:
    return a

result = pick_first(1, "x")      # T = int, U = str, result = 1
result = pick_first("hello", 42) # T = str, U = int, result = "hello"
```

容器类型内的类型参数

类型参数可以出现在泛型容器类型（List<T>、Map<K, V>、Set<T>）、元组（T, T）和函数类型 function(T) -> T 内部。推断会针对实际参数对声明的参数类型进行结构化遍历，因此当形状明确时不需要显式类型注解。

```
function first_of<T>(xs: List<T>) -> T:
    return xs[0]

first_of([1, 2, 3])           # T = int  -> 1
first_of(["hello", "world"])  # T = str  -> "hello"
first_of([[1, 2], [3, 4]])    # T = List<int> -> [1, 2]
```

```
function map_lookup<K, V>(m: Map<K, V>, k: K) -> V:
    return m[k]

map_lookup({1: "a", 2: "b"}, 1)      # K = int, V = str -> "a"
map_lookup({"x": 10, "y": 20}, "y") # K = str, V = int -> 20
```

```
function pair_first<T>(p: (T, T)) -> T:
    return p.0

pair_first((42, 7))      # T = int -> 42
pair_first(("a", "b"))   # T = str -> "a"
```

跨多个参数位置引用的类型参数会被统一 —— 两次出现必须解析为同一具体类型：

```
function apply_list<T>(xs: List<T>, f: function(T) -> T) -> T:
    return f(xs[0])

apply_list([10, 20, 30], (x: int) => x + 1) # T = int -> 11
```

如果推断无法确定类型参数（例如，从空容器字面值），请使用显式 `name[Type](args)` 语法：

```
first_of[int]([]) # 空列表：明确告诉编译器 T = int
```

跨参数的冲突推断会产生清晰的编译错误，命名类型参数和函数，而不是不透明的类型不匹配：

```
function same<T>(a: T, b: T) -> T:
    return a

same(1, "x") # 错误：在调用 'same' 时 'T' 的类型推断冲突
```

类型约束（边界）

类型参数可以使用 `RecordName` 语法以 `record` 类型进行约束。具体类型必须是绑定类型本身或其子类型。

```
record Animal:
    name: str
    legs: int

record Dog < Animal:
    breed: str

function get_name<T: Animal>(a: T) -> str:
    return a.name

get_name(Dog("Rex", 4, "Lab")) # OK — Dog 是 Animal 的子类型
get_name(Animal("Cat", 4))     # OK — 精确类型匹配
```

有约束和无约束的类型参数可以混合使用：

```
function pair_name<T: Animal, U>(a: T, x: U) -> str:
    return a.name
```

工作原理

泛型函数使用单态化：为每个唯一的类型参数组合生成函数的特化版本。相同的实例化会被缓存并在多次调用间复用。当存在类型约束时，会在实例化时进行验证。

UFCS（统一函数调用语法）

可以使用 `a.f(b)` 的形式调用 `f(a, b)`。方便用于方法链。

语法

```
# 普通调用
f(a, b)

# UFCS 调用（等价）
a.f(b)
```

链接

```
function double(x: int) -> int:
    return x * 2

function add_one(x: int) -> int:
    return x + 1

result = 5.double().add_one()  # double(5) -> 10, add_one(10) -> 11
```

与字段访问混用

字段访问 (`.field`) 和 UFCS (`.method()`) 使用相同的点号记法，但通过是否有参数来区分。

```
p = Point(3, 4)
length = p.x.to_float()  # 字段访问 + UFCS
```

运算符重载

可以为用户定义类型定义运算符的行为。

语法

```
# 二元运算符（2 个参数）
function operator<op>(a: type, b: type) -> return_type:
    ...

# 一元运算符（1 个参数）
function operator<op>(a: type) -> return_type:
    ...
```

可重载的运算符

类别	运算符
算术（二元）	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code> <code>//</code>
比较（二元）	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>
位运算（二元）	<code>&</code> <code> </code> <code>^</code> <code><<</code> <code>>></code>
逻辑（二元）	<code>and</code> <code>or</code>
成员测试	<code>in</code>
下标	<code>[]</code> （读取）、 <code>[]=</code> （写入）
调用	<code>()</code>
转换	<code>as</code>

类别	运算符
一元	- ~ not

返回类型约束

比较、逻辑和成员测试运算符必须返回 bool：

类别	运算符	必需返回类型
比较	== != < <= > >=	bool
逻辑	and or not	bool
成员测试	in	bool
转换	as	必需（目标类型）

OK

```
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

Error: comparison operator '=' must return 'bool', but returns 'int'

```
function operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

算术运算符和位运算符没有返回类型约束。

二元 / 一元的区别

依参数个数区分。

```
record Vec2:
    x: float
    y: float
```

二元 +

```
function operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)
```

一元 -

```
function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

比较

```
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

```
v1 = Vec2(1.0, 2.0)
v2 = Vec2(3.0, 4.0)
v3 = v1 + v2      # Vec2(4.0, 6.0)
v4 = -v1          # Vec2(-1.0, -2.0)
```

检查/饱和算术

用于低级整数类型（i8、i16、i32、i64、u8、u16、u32、u64）的显式溢出控制内置函数。两个参数必须是相同类型。

函数	返回值	行为
checked_add(a, b)	Result<T, Error>	溢出时返回 Err
checked_sub(a, b)	Result<T, Error>	下溢时返回 Err
checked_mul(a, b)	Result<T, Error>	溢出时返回 Err
saturating_add(a, b)	T	溢出时钳制到类型最小/最大值
saturating_sub(a, b)	T	下溢时钳制到类型最小/最大值
saturating_mul(a, b)	T	溢出时钳制到类型最小/最大值
wrapping_add(a, b)	T	显式回绕（与 + 相同）
wrapping_sub(a, b)	T	显式回绕（与 - 相同）
wrapping_mul(a, b)	T	显式回绕（与 * 相同）

```
# Checked: 返回 Result, 使用 case 或 ? 处理
r = checked_add(2147483647i32, 1i32)
case r:
  Ok(v):
    print(v)
  Err(e):
    print("overflow!")    # 输出 "overflow!"
```

```
# Saturating: 钳制到边界
v = saturating_add(2147483647i32, 100i32)
print(v as int)    # 2147483647
```

```
# Wrapping: 自文档化的回绕行为
v = wrapping_add(2147483647i32, 1i32)
print(v as int)    # -2147483648
```

注意：这些函数不支持浮点类型（f32）或高级 int 类型。低级整数的默认 +、-、* 运算符使用回绕行为（有符号为补码，无符号为模运算）。

[English](#) | [日本語](#) | [简体中文](#)

GC 参考

gc 包提供对循环收集器的控制。循环收集器是与 ARC（自动引用计数）协同工作的安全网，用于检测和回收循环引用链。

概述

Ry 使用 ARC 进行内存管理，大多数释放操作是即时且确定性的。然而，ARC 本身无法回收循环引用（例如 A -> B -> A）。循环收集器使用类似 CPython 的试探性删除算法，定期查找并释放这类不可达的循环。

使用 weak 引用仍然是避免循环的推荐方式，但循环收集器可以捕获用户遗漏的情况。

导入

```
from gc import collect, enable, disable, set_threshold
```

函数

函数	签名	说明
collect	() -> int	执行一次完整的收集周期。返回收集的对象数量。

函数	签名	说明
<code>enable</code>	<code>() -> Unit</code>	启用自动收集（默认已启用）。
<code>disable</code>	<code>() -> Unit</code>	禁用自动收集。适用于性能关键的代码段。
<code>set_threshold</code>	<code>(n: int) -> Unit</code>	设置自动收集的候选计数阈值（默认值：700）。

工作原理

1. 候选跟踪：当 `arc_release` 减少对象的引用计数但计数仍大于零时，该对象会被添加到候选集合中（仅限于可能形成循环的类型）。
2. 试探性删除：收集器暂时减少候选对象引用的所有对象的引用计数。如果某个对象的计数仅通过试探性删除就降为零，则说明它只在候选集合内可达——它是循环的一部分。
3. 复活检查：仍可从候选集合外部访问的对象会恢复其引用计数。
4. 收集：剩余的不可达对象被释放。

静态分析优化

编译器在编译时执行静态分析，以识别哪些类型可能形成引用循环（例如递归 ADT `enum`）。只有这些类型参与 GC 候选跟踪。非循环类型的 GC 开销为零。

示例

```
from gc import collect, disable, enable

# 在性能关键区段禁用自动回收
disable()

# ... 性能关键代码 ...

# 重新启用并强制执行一次回收
enable()
collected = collect()
print(f"Collected {collected} cyclic objects")
```

[English](#) | [简体中文](#)

HTTP 参考

类型

类型	说明
<code>HttpRequest</code>	传入 HTTP 请求的不透明句柄
<code>HttpResponse</code>	传出 HTTP 响应的不透明句柄
<code>HttpClientResponse</code>	HTTP 客户端响应的不透明句柄

`HttpRequest` 由服务器框架提供。`HttpResponse` 通过 `response()` 创建。`HttpClientResponse` 由客户端函数 (`http_get`、`http_post`、`http_request`) 返回。

函数（来自 http）

这些函数需要显式导入：

```
from http import listen, method, path, header, body, body_bytes, query, query_all, cookie, cookies, form
```

服务器

函数	签名	说明
listen	(host: str, port: int, handler: function(HttpRequest) -> HttpResponse) -> Unit	在指定地址启动 HTTP 服务器。在接受循环中阻塞，对每个请求调用 handler。
listen	(host: str, port: int, handler: function(HttpRequest) -> HttpResponse, max_requests: int) -> Unit	启动一个在处理 max_requests 个请求后停止的 HTTP 服务器。支持 async function + block_on() 的生命周期管理。
listen	(host: str, port: int, handler: function(HttpRequest) -> HttpResponse, max_requests: int, port_callback: function(int) -> Unit) -> Unit	与上述相同，但在 bind + listen 成功后调用 port_callback 并传入实际绑定的端口。可与端口 0 配合使用以获取操作系统分配的临时端口。

请求访问器

函数	签名	说明
method	(req: HttpRequest) -> str	返回 HTTP 方法（例如 "GET"、"POST"）。
path	(req: HttpRequest) -> str	返回不含查询字符串的请求路径（例如对于 "/search?q=hello" 返回 "/search"）。
header	(req: HttpRequest, key: str) -> Option<str>	返回请求头的值（不区分大小写查找）。未找到时返回 None。
body	(req: HttpRequest) -> str	以字符串形式返回请求体。在第一个 NUL 字节处截断；对于二进制数据请使用 body_bytes。
body_bytes	(req: HttpRequest) -> List<u8>	以字节列表形式返回请求体。二进制安全——保留所有字节包括 NUL。
query	(req: HttpRequest, key: str) -> Option<str>	返回查询参数的值。未找到时返回 None。值会自动进行 URL 解码。
query_all	(req: HttpRequest) -> Map<str, str>	以映射形式返回所有查询参数。键和值会自动进行 URL 解码。
cookie	(req: HttpRequest, name: str) -> Option<str>	按名称返回 cookie 的值。未找到时返回 None。
cookies	(req: HttpRequest) -> Map<str, str>	以映射形式返回所有 cookie。
form_field	(req: HttpRequest, name: str) -> Option<str>	返回 multipart 表单文本字段的值。未找到时返回 None。

函数	签名	说明
form_file	(req: HttpRequest, name: str) -> Option<Map<str, str>>	以 Option 形式返回文件上传信息。Some(map) 包含 "filename" "content_type" "data" 键。未找到时返回 None。
form_fields	(req: HttpRequest) -> Map<str, str>	以映射形式返回所有 multipart 表单文本字段。

响应构建器

函数	签名	说明
response	(status: int, headers: Map<str, str>, body: str) -> HttpResponse	使用给定的状态码、响应头和响应体创建 HTTP 响应。

使用示例

基本 HTTP 服务器

```
from http import listen, method, path, header, body, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    m = method(req)
    p = path(req)
    if p == "/hello":
        return response(200, {"Content-Type": "text/plain"}, "Hello, World!")
    if p == "/echo":
        b = body(req)
        return response(200, {"Content-Type": "text/plain"}, b)
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)
```

使用 async function 的非阻塞服务器

```
from http import listen, path, response

async function start_server(port: int) -> str:
    listen("127.0.0.1", port, (req: HttpRequest) -> HttpResponse:
        p = path(req)
        if p == "/api/health":
            return response(200, {"Content-Type": "application/json"}, "{\"status\": \"ok\"}")
        return response(404, {"Content-Type": "text/plain"}, "Not Found")
    )
    return "done"

t = start_server(8080)
# 服务器在后台任务中运行
```

带请求限制的服务器 (max_requests)

```
from http import listen, path, response, http_get, status, body

port_holder = [0]
```



```

function on_port(p: int) -> Unit:
    port_holder[0] = p

async function start_server() -> str:
    listen("127.0.0.1", 0, (req: HttpRequest) -> HttpResponse:
        return response(200, {"Content-Type": "text/plain"}, "Hello!")
    , 1, on_port) # 处理 1 个请求后停止；调用 on_port 传入绑定的端口
    return "done"

t = start_server()
sleep(100) # 等待服务器启动
port = port_holder[0]

case http_get("http://127.0.0.1:" + to_str(port) + "/"):
    Ok(resp):
        print(body(resp)) # "Hello!"
    Err(e):
        print("error")

result = block_on(t) # 服务器在处理 1 个请求后退出；block_on 完成

```

读取查询参数

```

from http import listen, path, query, query_all, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    p = path(req)
    if p == "/search":
        case query(req, "q"):
            Some(q):
                return response(200, {"Content-Type": "text/plain"}, "Search: " + q)
            None:
                return response(400, {"Content-Type": "text/plain"}, "Missing query parameter: q")
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)

```

读取请求头

```

from http import listen, header, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case header(req, "Authorization"):
        Some(token):
            return response(200, {"Content-Type": "text/plain"}, "Authenticated: " + token)
        None:
            return response(401, {"Content-Type": "text/plain"}, "Unauthorized")
)

```

处理表单提交

```

from http import listen, form_field, form_file, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case form_field(req, "username"):
        Some(name):

```

```

        case form_file(req, "avatar"):
            Some(file_info):
                filename = file_info["filename"]
                return response(200, {"Content-Type": "text/plain"}, "Hello " + name + ", file: " +
                                filename)
            None:
                return response(400, {"Content-Type": "text/plain"}, "No file uploaded")
    None:
        return response(400, {"Content-Type": "text/plain"}, "Missing username")
)

```

读取 Cookie

```

from http import listen, cookie, cookies, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case cookie(req, "session_id"):
        Some(sid):
            return response(200, {"Content-Type": "text/plain"}, "Session: " + sid)
        None:
            return response(401, {"Content-Type": "text/plain"}, "No session")
)

```

行为

- `listen()` 绑定到地址，开始监听，并进入接受循环。
- 使用 3 个参数调用时，接受循环无限运行。
- 使用 4 个参数调用时 (`max_requests`)，服务器在处理指定数量的请求后停止。`max_requests` 必须是正整数。这支持 `async function + block_on()` 的生命周期管理。格式错误的请求（静默跳过）不计入限制。
- 使用 5 个参数调用时 (`max_requests port_callback`)，在 `bind + listen` 成功后同步调用 `port_callback` 并传入实际绑定的端口。这允许安全使用端口 0（操作系统分配的临时端口）以避免并行测试中的端口冲突。
- 服务器默认支持 HTTP/1.1 keep-alive。单个连接上可以处理多个请求。服务器在每个请求上检查 `Connection` 头：如果发送了 `Connection: close`，则在响应后关闭连接；否则连接保持打开以处理后续请求。空闲连接在 5 秒超时后关闭。
- 如果响应头映射中未提供 `Content-Length`，则会自动添加。
- 服务器支持基于 `Content-Length` 的请求体读取和 `Transfer-Encoding: chunked` 解码的 HTTP/1.1。
- 当请求中存在 `Transfer-Encoding: chunked` 时，请求体会自动解码并拼接——Ry 代码透明地接收完整的请求体。
- 如果同时存在 `Transfer-Encoding: chunked` 和 `Content-Length`，请求将被视为格式错误而拒绝（依据 RFC 9112）。
- 要发送分块响应，请在传给 `response()` 的响应头映射中包含 `"Transfer-Encoding": "chunked"`。响应体将自动以分块格式编码。
- 通过 `header()` 的头部查找不区分大小写。
- `path()` 返回不含查询字符串的路径。查询参数通过 `query()` 或 `query_all()` 单独访问。
- 查询参数值会自动进行 URL 解码（`%20` → 空格，`+` → 空格）。
- 对于重复的查询参数键，返回第一个值。
- `cookie()` 和 `cookies()` 通过按 `;` 分割来解析 Cookie 头，然后按第一个 `=` 分割每个键值对。名称和值的前后空白会被去除。
- 对于重复的 cookie 名称，返回第一个值。
- Cookie 值可以包含 `=` 字符（只有第一个 `=` 用于分隔名称和值）。
- `form_field()`、`form_file()` 和 `form_fields()` 解析 `multipart/form-data` 请求体。解析是惰性的——在第一次调用时解析请求体并缓存。
- `boundary` 参数从 `Content-Type` 头中提取，支持带引号和不带引号的值。
- `Content-Disposition` 中带有 `filename` 的部分被视为文件上传；没有的则被视为文本字段。
- 对于重复的字段/文件名称，返回第一个值。

- `form_file()` 返回 `Some(map)`，包含 `"filename"`、`"content_type"` 和 `"data"` 键；如果未找到字段则返回 `None`。如果部分未指定 `Content-Type`，默认为 `"application/octet-stream"`。
- 对于非 `multipart` 请求，表单函数返回 `None` (对于 `form_field`、`form_file`) 或空映射 (对于 `form_fields`)。

支持的状态码

以下状态码具有标准原因短语 (RFC 9110)：

代码	原因
100	Continue
101	Switching Protocols
200	OK
201	Created
202	Accepted
203	Non-Authoritative Information
204	No Content
205	Reset Content
206	Partial Content
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
307	Temporary Redirect
308	Permanent Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
408	Request Timeout
409	Conflict
410	Gone
411	Length Required
413	Content Too Large
414	URI Too Long
415	Unsupported Media Type
416	Range Not Satisfiable
417	Expectation Failed
422	Unprocessable Content
426	Upgrade Required
429	Too Many Requests
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported

其他状态码使用 `"Unknown"` 作为原因短语。

HTTP 客户端

客户端函数

函数	签名	说明
<code>http_get</code>	<code>(url: str) -> Result<HttpClientResponse, Error></code>	向给定 URL 发送 HTTP GET 请求。
<code>http_post</code>	<code>(url: str, body: str, headers: Map<str, str>) -> Result<HttpClientResponse, Error></code>	发送带有请求体和请求头的 HTTP POST 请求。
<code>http_request</code>	<code>(method: str, url: str, headers: Map<str, str>, body: str) -> Result<HttpClientResponse, Error></code>	使用自定义方法发送 HTTP 请求。

响应访问器

函数	签名	说明
<code>status</code>	<code>(resp: HttpClientResponse) -> int</code>	返回 HTTP 状态码。
<code>body</code>	<code>(resp: HttpClientResponse) -> str</code>	以字符串形式返回响应体。在第一个 NUL 字节处截断；对于二进制数据请使用 <code>body_bytes</code> 。
<code>body_bytes</code>	<code>(resp: HttpClientResponse) -> List<u8></code>	以字节列表形式返回响应体。二进制安全——保留所有字节包括 NUL。
<code>header</code>	<code>(resp: HttpClientResponse, key: str) -> Option<str></code>	返回响应头的值（不区分大小写）。未找到时返回 <code>None</code> 。
<code>http_client_response_free</code>	<code>(resp: HttpClientResponse) -> Unit</code>	释放响应及其关联的内存。使用完响应后调用。

客户端使用示例

```
from http import http_get, http_post, status, body, header
```

```
# 简单的 GET 请求
```

```
case http_get("http://example.com/api/data"):
    Ok(resp):
        s = status(resp)
        b = body(resp)
        print(to_str(s) + ": " + b)
    Err(e):
        print("Request failed")
```

```
# 带请求体和请求头的 POST 请求
```

```
headers: Map<str, str> = {"Content-Type": "application/json"}
case http_post("http://example.com/api/data", "{\"key\": \"value\"}", headers):
    Ok(resp):
        print(body(resp))
```

```
Err(e):
    print("Request failed")
```

客户端行为

- 支持 `http://` 和 `https://` URL。HTTPS 使用 TLS 和系统 CA 证书包进行证书验证。
- Host 头会根据 URL 自动添加。
- 始终发送 `Connection: close`；每个请求使用单独的 TCP 连接。
- Content-Length 始终自动添加（包括空请求体的 0）。用户提供的 Content-Length 头会被正确的值覆盖。
- 响应体读取支持 Content-Length、Transfer-Encoding: chunked 或读取到连接关闭。
- 响应头查找不区分大小写。
- 连接超时为 5 秒；接收超时为 30 秒。
- 连接失败、无效 URL 或格式错误的响应时返回 Err。
- HttpClientResponse 拥有已分配的内存（头部、响应体）。使用完毕后调用 `http_client_response_free()` 以避免内存泄漏。

重定向行为

HTTP 客户端函数会自动跟随重定向响应（带有 Location 头的 3xx 响应）。

- 支持的状态码：301、302、303、307、308
- 最大重定向次数：10（超过时返回 Err）
- 方法转换（依据 RFC 9110）：
 - 301、302：POST 会被更改为 GET（请求体被丢弃）；其他方法保持不变
 - 303：方法始终更改为 GET（请求体被丢弃）
 - 307、308：方法和请求体保持不变
- URL 解析：支持 Location 头中的绝对 URL、协议相对 URL (`//...`)、绝对路径 (`/...`) 和相对路径
- 用户提供的头部在每次重定向跳转时重新发送，但敏感头部（Authorization、Proxy-Authorization、Cookie）在跨源重定向（不同主机或端口）时会被移除
- 如果 Location 头缺失或为空，则按原样返回重定向响应
- 调用者仅接收所有重定向完成后的最终响应

错误处理

- 如果 `bind()` 失败（例如端口已被占用），`listen()` 会引发运行时错误。
- 格式错误的请求或 keep-alive 连接上的空闲超时会导致连接关闭。服务器随后继续接受新连接。
- 处理函数必须始终返回 `HttpResponse`——没有默认响应。
- 客户端函数返回 `Result<HttpClientResponse, Error>`——使用 case 处理成功和失败情况。

[English](#) | [日本語](#) | [繁體中文](#)

I/O 函数参考手册

标准输入输出与文件操作。所有函数均需从 `io` 明确导入。

```
from io import read_text, write_text, exists
```

函数列表

标准输入

函数	签名	说明
<code>read_line</code>	<code>() -> str</code>	从 stdin 读取一行（移除末尾换行）
<code>read_all</code>	<code>() -> str</code>	读取 stdin 直到 EOF

文件 I/O

函数	签名	说明
read_text	(str) -> Result<str, Error>	将整个文件作为字符串读取
write_text	(str, str) -> Result<Unit, Error>	将字符串写入文件（覆盖）
append_text	(str, str) -> Result<Unit, Error>	在文件末尾追加字符串
exists	(str) -> bool	检查文件是否存在
delete_file	(str) -> Result<Unit, Error>	删除文件
read_bytes	(str) -> Result<List<u8>, Error>	将文件作为字节列表读取
write_bytes	(str, List<u8>) -> Result<Unit, Error>	将字节列表写入文件

字节转换

函数	签名	说明
to_bytes	(str) -> List<u8>	将字符串转换为 UTF-8 字节
bytes_to_str	(List<u8>) -> Result<str, Error>	将字节列表转换为字符串

使用示例

读写文件

```
from io import read_text, write_text, append_text, exists, delete_file

case write_text("hello.txt", "Hello, World!"):
    Ok(_):
        case read_text("hello.txt"):
            Ok(content):
                print(content)    # Hello, World!
            Err(e):
                print(e.message)
    Err(e):
        print(e.message)

print(exists("hello.txt"))    # true

case delete_file("hello.txt"):
    Ok(_):
        print(exists("hello.txt"))    # false
    Err(e):
        print(e.message)
```

字节操作

```
from io import to_bytes, bytes_to_str, write_bytes, read_bytes

bs = to_bytes("ABC")
print(length(bs))    # 3
```

```

case write_bytes("data.bin", bs):
    Ok(_):
        case read_bytes("data.bin"):
            Ok(rb):
                case bytes_to_str(rb):
                    Ok(s):
                        print(s)          # ABC
                    Err(e):
                        print(e.message)
            Err(e):
                print(e.message)
        Err(e):
            print(e.message)

```

从标准输入读取

```

from io import read_line

name = read_line()
print(f"Hello, {name}!")

```

错误处理

文件操作返回 `Result<T, Error>` 而不是在失败时终止程序。使用 `case` 配合 `Ok/Err` 模式来处理错误:

```

case read_text("missing.txt"):
    Ok(content):
        print(content)
    Err(e):
        print(e.message)    # cannot open file 'missing.txt' for reading

```

操作	错误条件
<code>read_text</code> / <code>read_bytes</code>	文件不存在或无法打开
<code>write_text</code> / <code>write_bytes</code> / <code>append_text</code>	无法打开文件进行写入
<code>delete_file</code>	无法删除文件
<code>bytes_to_str</code>	输入包含 NUL 字节

备注

- 使用 `List<u8>` 作为缓冲区类型。标准列表操作 (`length()`、`append()`、`slice()`、索引访问) 均可用于字节列表。
- 文件路径若未指定绝对路径, 则为相对于当前工作目录的相对路径。
- `write_text` 与 `write_bytes` 会覆盖现有文件。若要追加内容, 请使用 `append_text`。

[English](#) | [日本語](#) | [繁體中文](#)

JSON 函数参考

JSON 解析与序列化。所有函数需要从 `json` 明确导入。

```

from json import parse, stringify, kind, get, at, to_str, to_int, to_float, to_bool, length, keys, json

```

概述

json 包提供将 JSON 文本解析为不透明的 JsonValue 类型、通过访问函数读取其内容、以及重新序列化为文本的功能。由于 JSON 值可以是异构的（对象可以包含字符串、数字、布尔值、数组和嵌套对象），本包使用不透明指针类型和类型化的访问函数。

函数列表

解析 / 序列化

函数	签名	说明
parse	(str) -> Result<JsonValue, Error>	将 JSON 字符串解析为 JsonValue
stringify	(JsonValue) -> str	将 JsonValue 序列化为紧凑的 JSON 文本
stringify	(JsonValue, int) -> str	以缩进格式输出（参数为空格数）

类型查询

函数	签名	说明
kind	(JsonValue) -> str	返回 JSON 类型: "object", "array", "string", "number", "boolean", "null"

对象 / 数组访问

函数	签名	说明
get	(JsonValue, str) -> Result<JsonValue, Error>	以键从对象获取字段
at	(JsonValue, int) -> Result<JsonValue, Error>	以索引从数组获取元素

值提取

函数	签名	说明
to_str	(JsonValue) -> Result<str, Error>	提取字符串值
to_int	(JsonValue) -> Result<int, Error>	提取整数值
to_float	(JsonValue) -> Result<float, Error>	提取浮点数值
to_bool	(JsonValue) -> Result<bool, Error>	提取布尔值

集合信息

函数	签名	说明
length	(JsonValue) -> int	返回数组长度或对象键数
keys	(JsonValue) -> Result<List<str>, Error>	返回对象的键列表，若值不是对象则返回错误

内存管理

函数	签名	说明
<code>json_free</code>	<code>(JsonValue) -> Unit</code>	释放 <code>JsonValue</code> 及其所有子元素

解开 `Result<JsonValue, Error>`

`parse`、`get` 和 `at` 返回 `Result<JsonValue, Error>`。在将内部值传递给其他 `json` 函数之前，您必须先解开 `Result` — 直接传递 `Result` 在编译期会被拒绝：

```
case parse(text):
  Ok(doc):
    # □ 错误: kind() 需要 JsonValue 参数
    # kind(get(doc, "name"))
    # □ 先解开
    case get(doc, "name"):
      Ok(name_val):
        print(kind(name_val))
      Err(e):
        print("no name")
  Err(e):
    print("parse error")
```

对 `Result` 的通用字符串化 (`to_str(result)`、`print(result)`、`f-string` 内插) 仍然可用，并会格式化为 `Ok(...)` / `Err(...)`，与其他任何 `Result` 值的行为一致。

使用示例

解析与访问字段

```
from json import parse, get, to_str, to_int, json_free

case parse("{\"name\": \"Alice\", \"age\": 30}"):
  Ok(data):
    case get(data, "name"):
      Ok(val):
        case to_str(val):
          Ok(name):
            print(name)    # "Alice"
          Err(e):
            print("error")
      Err(e):
        print("error")
    json_free(data)
  Err(e):
    print("parse error: " + e.message)
```

处理数组

```
from json import parse, at, to_int, length, json_free

case parse("[10, 20, 30]"):
  Ok(data):
    print(to_str(length(data)))    # 3
    case at(data, 0):
```

```

Ok(elem):
    case to_int(elem):
        Ok(n):
            print(to_str(n))    # 10
        Err(e):
            print("error")
    Err(e):
        print("error")
json_free(data)
Err(e):
    print("parse error")

```

带缩进的序列化

```

from json import parse, stringify, json_free

case parse("{\"key\":\"value\", \"count\":42}"):
    Ok(data):
        print(stringify(data, 2))
        # {
        #   "key": "value",
        #   "count": 42
        # }
        json_free(data)
    Err(e):
        print("error")

```

注意事项

- to_int 接受整数和整数值的浮点数（例如 42.0 -> 42）
- to_float 接受浮点数和整数（例如 42 -> 42.0）
- get 和 at 返回解析树中子元素的引用——不要对子元素调用 json_free，只对 parse 返回的根值调用
- kind 对整数和浮点数都返回 "number"

[English](#) | [日本語](#) | [繁體中文](#)

數學函式 (math)

概述

math 套件提供數學常數與函式。與 std 套件不同，它不會自動匯入。需要使用明確匯入來存取函式。

```
from math import sqrt, PI, sin
```

常數

常數	型別	說明
PI	float	圓周率 (3.141592653589793)
E	float	尤拉數 (2.718281828459045)
Inf	float	正無窮大
NaN	float	非數值 (Not a Number)

```
from math import PI, E, Inf, NaN

circumference = 2.0 * PI * radius
```

絕對值

函式	簽章	說明
abs(x)	(int) -> int	整數的絕對值
abs(x)	(float) -> float	浮點數的絕對值

```
from math import abs

abs(-5)      # 5
abs(-3.14)   # 3.14
```

捨入

函式	簽章	說明
floor(x)	(float) -> int	向下取整
floor(x, digits)	(float, int) -> float	向下取整至指定的小數位數
ceil(x)	(float) -> int	向上取整
ceil(x, digits)	(float, int) -> float	向上取整至指定的小數位數
round(x)	(float) -> int	四舍五入（離零較遠的方向）
round(x, digits)	(float, int) -> float	四舍五入至指定的小數位數（離零較遠的方向）

```
from math import floor, ceil, round

floor(3.7)      # 3
ceil(3.2)       # 4
round(3.5)      # 4

round(3.14159, 2) # 3.14
floor(3.789, 1)  # 3.7
ceil(3.123, 1)   # 3.2
```

双参数形式接受负数 digits 以四舍五入至 10 的幂：

```
round(1234.5, -2) # 1200.0
round(1750.0, -3) # 2000.0
```

舍入采用 C99 的「離零較遠」語意（透過 $\text{round}(x * 10^{\text{digits}}) / 10^{\text{digits}}$ ），与单参数形式一致。这与 Python 的银行家舍入不同——例如， $\text{round}(2.675, 2) == 2.68$ ，而非 2.67。NaN 和 $\pm\text{Inf}$ 会原样传递。

幂次與平方根

函式	簽章	說明
<code>sqrt(x)</code>	<code>(float) -> float</code>	平方根
<code>pow(x, y)</code>	<code>(float, float) -> float</code>	x 的 y 次方
<code>pow(x, y)</code>	<code>(int, int) -> int</code>	透過快速幂算法的整数幂

```
from math import sqrt, pow
```

```
sqrt(9.0)      # 3.0
pow(2.0, 3.0)  # 8.0
pow(2, 10)     # 1024
pow(-2, 3)     # -8
```

当指数为负时，整数重载会引发运行时错误（`pow(2, -1)` 会以 `pow() integer exponent must be non-negative` 中止）。溢位会静默回绕，与 Ry 现有的整数算术模型一致。

對數

函式	簽章	說明
<code>log(x)</code>	<code>(float) -> float</code>	自然對數 (底數 e)
<code>log(x, base)</code>	<code>(float, float) -> float</code>	任意底数的对数
<code>log2(x)</code>	<code>(float) -> float</code>	底數 2 的對數
<code>log10(x)</code>	<code>(float) -> float</code>	常用對數 (底數 10)

```
from math import log
```

```
log(8.0, 2.0)    # 3.0
log(100.0, 10.0) # 2.0
```

`log(x, base)` 计算为 `log(x) / log(base)`，因此任一参数的定义域错误都会传播为 NaN 或 -Inf。

指數

函式	簽章	說明
<code>exp(x)</code>	<code>(float) -> float</code>	e 的 x 次方

三角函式

函式	簽章	說明
<code>sin(x)</code>	<code>(float) -> float</code>	正弦 (x 為弧度)
<code>cos(x)</code>	<code>(float) -> float</code>	餘弦 (x 為弧度)
<code>tan(x)</code>	<code>(float) -> float</code>	正切 (x 為弧度)

反三角函数

函数	簽章	說明
<code>asin(x)</code>	<code>(float) -> float</code>	反正弦 (結果為弧度)
<code>acos(x)</code>	<code>(float) -> float</code>	反餘弦 (結果為弧度)
<code>atan(x)</code>	<code>(float) -> float</code>	反正切 (結果為弧度)

雙參數函数

函数	簽章	說明
<code>atan2(y, x)</code>	<code>(float, float) -> float</code>	y/x 的反正切 (結果為弧度)
<code>hypot(x, y)</code>	<code>(float, float) -> float</code>	斜邊長度: $\sqrt{x^2 + y^2}$

判定函数

函数	簽章	說明
<code>is_nan(x)</code>	<code>(float) -> bool</code>	若 x 為 NaN 則回傳 true
<code>is_inf(x)</code>	<code>(float) -> bool</code>	若 x 為正或負無窮大則回傳 true

```
from math import is_nan, is_inf, NaN, Inf
```

```
is_nan(NaN)    # true
is_inf(Inf)    # true
is_nan(1.0)    # false
```

[English](#)

网络 (TCP) 参考手册

类型

类型	说明
<code>TcpListener</code>	TCP 服务器套接字的不透明句柄
<code>TcpStream</code>	TCP 连接的不透明句柄
<code>TlsStream</code>	TLS 加密 TCP 连接的不透明句柄

所有类型都是由 ARC (自动引用计数) 管理的不透明指针。不能直接构造; 使用 `bind()` 或 `connect()` 获取 `TcpListener`/`TcpStream`, 使用 `tls_connect()` 获取 `TlsStream`。当不再被引用时, 资源会自动关闭——显式调用 `close()` 是可选的, 但支持立即清理。

函数 (来自 `net`)

这些函数需要显式导入:

```
from net import bind, listen, accept, connect, listener_port, shutdown, set_timeout, set_receive_timeout
```

函数	签名	说明
bind	(host: str, port: int) -> Result<TcpListener, Error>	创建绑定到指定地址的 TCP 服务器套接字。失败时返回 Err。使用端口 0 进行动态分配。
listen	(listener: TcpListener, backlog: int) -> Result<Unit, Error>	开始监听传入连接。失败时返回 Err。
accept	(listener: TcpListener) -> Result<TcpStream, Error>	接受新连接。等待客户端连接最多 1 秒。超时或失败时返回 Err。
connect	(host: str, port: int) -> Result<TcpStream, Error>	连接到远程 TCP 服务器。5 秒后超时。超时或失败时返回 Err。
listener_port	(listener: TcpListener) -> int	返回监听器绑定的实际端口号。在绑定到端口 0（操作系统分配的端口）时很有用。
shutdown	(listener: TcpListener) -> Unit	通知监听器停止接受连接。使任何挂起的 accept() 在最多 1 秒内返回。
tls_connect	(host: str, port: int) -> Result<TlsStream, Error>	使用 TLS 加密连接到远程服务器。根据系统 CA 证书包验证服务器证书。连接或握手失败时返回 Err。
set_timeout	(stream: TcpStream TlsStream, ms: int) -> Unit	设置接收和发送超时（毫秒）。
set_receive_timeout	(stream: TcpStream TlsStream, ms: int) -> Unit	设置接收超时（毫秒）。如果在此时间内没有数据到达，receive() 返回 Err。
set_send_timeout	(stream: TcpStream TlsStream, ms: int) -> Unit	设置发送超时（毫秒）。

内建重载函数

这些函数是内建的，可与 TCP 套接字类型配合使用。无需导入。

函数	签名	说明
send	(stream: TcpStream TlsStream, data: List<u8>) -> Result<int, Error>	通过 TCP 或 TLS 连接发送字节。成功时返回 Ok 和已发送的字节数，失败时返回 Err。
receive	(stream: TcpStream TlsStream, max: int) -> Result<List<u8>, Error>	接收最多 max 个字节。连接关闭时返回 Ok 和空列表，出错时返回 Err。
close	(handle: TcpStream TlsStream) -> Unit	关闭 TCP 或 TLS 流。
close	(handle: TcpListener) -> Unit	关闭 TCP 监听器。

使用示例

回显服务器

```
from net import bind, listen, accept, connect
from io import to_bytes, bytes_to_str

# 服务器
case bind("127.0.0.1", 8080):
    Ok(server):
        case listen(server, 128):
            Ok(_):
                case accept(server):
                    Ok(conn):
                        case receive(conn, 4096):
                            Ok(data):
                                case send(conn, data):
                                    Ok(_):
                                        ...
                                    Err(e):
                                        print(e.message)
                                Err(e):
                                    print(e.message)
                            close(conn)
                        Err(e):
                            ...
                    Err(e):
                        print("listen failed")
                close(server)
            Err(e):
                print("bind failed")
```

客户端

```
case connect("127.0.0.1", 8080):
    Ok(conn):
        case send(conn, to_bytes("hello")):
            Ok(_):
                ...
            Err(e):
                print(e.message)
        case receive(conn, 4096):
            Ok(resp):
                case bytes_to_str(resp):
                    Ok(s):
                        print(s)
                    Err(e):
                        print(e.message)
            Err(e):
                print(e.message)
        close(conn)
    Err(e):
        print("connect failed")
```

使用 `async function` 的并发回显服务器

```
from net import bind, listen, accept, connect, listener_port
from io import to_bytes, bytes_to_str
```

```
async function echo_server(server: TcpListener) -> str:
```

```
    case accept(server):
```

```
        Ok(conn):
```

```
            case receive(conn, 4096):
```

```
                Ok(data):
```

```
                    case send(conn, data):
```

```
                        Ok(_):
```

```
                            ...
```

```
                        Err(e):
```

```
                            ...
```

```
                    Err(e):
```

```
                        ...
```

```
                close(conn)
```

```
            Err(e):
```

```
                ...
```

```
        close(server)
```

```
    return "done"
```

```
case bind("127.0.0.1", 0):
```

```
    Ok(server):
```

```
        case listen(server, 1):
```

```
            Ok(_):
```

```
                port = listener_port(server)
```

```
                t = echo_server(server)
```

```
                # ... 使用 port 的客户端代码 ...
```

```
                block_on(t)
```

```
            Err(e):
```

```
                ...
```

```
        Err(e):
```

```
            ...
```

超时配置

默认情况下，如果未设置自定义超时，`receive()` 使用 30 秒超时。使用 `set_timeout()`、`set_receive_timeout()` 或 `set_send_timeout()` 来覆盖默认值：

```
from net import connect, set_receive_timeout
```

```
case connect("127.0.0.1", 8080):
```

```
    Ok(conn):
```

```
        set_receive_timeout(conn, 5000) # 5 秒超时
```

```
        case receive(conn, 4096):
```

```
            Ok(data):
```

```
                ...
```

```
            Err(e):
```

```
                print("timeout or error")
```

```
        close(conn)
```

```
    Err(e):
```

```
        print("connect failed")
```

传入 0 可禁用超时（无限等待）。

错误处理

- 除 close() 外，所有 TCP 函数都返回 Result<T, Error> — 使用 case 配合 Ok/Err 来处理失败情况。
- 当对端关闭连接时，receive() 返回 Ok 和空的 List<u8>；实际错误（超时、套接字错误）时返回 Err。
- close() 关闭套接字并释放句柄。在关闭后使用句柄是未定义行为。

字节转换

TCP 操作使用 List<u8>。使用 io 中的 to_bytes() 和 bytes_to_str() 在字符串和字节列表之间进行转换。

[English](#) | [日本語](#) | [简体中文](#)

运算符参考

优先级表

优先级数字越小越高（越先被求值）。

优先级	运算符	说明	结合性
0	? !!	错误传播（后缀）	左
1	()	分组	—
2	+x -x ~x	一元正号、负号、位 NOT	右
3	**	幂运算	右
3.5	as	类型转换	左
4	* / % //	乘法、除法、取余、整数除法	左
5	+ -	加法、减法	左
6	<< >> >>>	位移	左
7	&	位 AND	左
8	^	位 XOR	左
9	\	位 OR	左
10	== != < <= > >= in not in	比较、成员测试	左
11	not	逻辑 NOT	右
12	and	逻辑 AND	左
13	or	逻辑 OR	左
13.5	??	空值合并	左

算术运算符

运算符	说明	示例
+	加法 / 字符串拼接	1 + 2 -> 3、"a" + "b" -> "ab"、"x" + 1 -> "x1"
-	减法	5 - 3 -> 2
*	乘法 / 字符串重复	4 * 3 -> 12、"ab" * 3 -> "ababab"
/	除法（始终为 float）	7 / 2 -> 3.5
//	向下取整除法（向 -∞）	7 // 2 -> 3、-7 // 2 -> -4
%	取余	7 % 3 -> 1
**	幂运算（始终为 float）	2 ** 10 -> 1024.0
-x	一元负号	-5、-3.14
+x	一元正号	+5（不改变正负号）

```
a = 10 // 3    # 3 (int)
b = 10 / 3     # 3.3333... (float)
```

```
c = 2 ** 8      # 256.0 (float)
s = "foo" + "bar" # "foobar"
t = "val=" + 42   # "val=42"
u = 3.14 + "!"    # "3.14!"
```

当 + 的一个操作数为 str、另一个为 int、float 或 bool 时，非 str 操作数会自动转换为字符串表示并拼接。

比较运算符

全部返回 bool。

运算符	说明
==	等于
!=	不等于
<	小于
<=	小于等于
>	大于
>=	大于等于

- 可用于数值类型 (int / float) 和 bool。
- str 之间以字典序 (字节顺序) 比较。
- 记录类型支持 == 和 !=，自动生成逐字段比较 (参见[结构体参考](#))。
- in 运算符用于集合、列表、映射的成员测试 (x in s)。
- not in 运算符为 in 的否定 (x not in s)。
- 对于映射，in 检查键是否存在。

```
x = 3 < 5      # true
y = "abc" < "abd" # true (字典序)
s = {1, 2, 3}
z = 2 in s     # true
w = 4 not in s # true
xs = [1, 2, 3]
a = 2 in xs    # true (列表线性搜索)
m = {"a": 1}
b = "a" in m   # true (映射键搜索)
```

逻辑运算符

运算符	说明	类型
and	逻辑 AND	bool x bool -> bool
or	逻辑 OR	bool x bool -> bool
not	逻辑 NOT	bool -> bool

```
a = true and false # false
b = true or false  # true
c = not true        # false
```

位运算符

仅可用于 int 类型。对 float 或 bool 使用会产生编译错误。

运算符	说明	示例
&	位 AND	0b1100 & 0b1010 -> 0b1000
\	位 OR	0b1100 \ 0b1010 -> 0b1110
^	位 XOR	0b1100 ^ 0b1010 -> 0b0110
~	位 NOT (一元)	~0 -> -1
<<	左移	1 << 4 -> 16
>>	算术右移	16 >> 2 -> 4
>>>	逻辑右移	-1 >>> 1 -> 9223372036854775807

```
flags = 0b0001 | 0b0010 # 3
masked = flags & 0b0011 # 3
shifted = 1 << 8 # 256
```

错误传播运算符 (? / !!)

后缀 ? 运算符在快乐路径上解包 Result 或 Option 值，并在不快乐路径上短路。!! 运算符是? 的别名，语义完全相同。

操作数	快乐路径	不快乐路径
Result<T, E>	求值为 Ok 内部值 v	外层函数返回 Err(e)
Option<T>	求值为 Some 内部值 v	外层函数返回 None

在函数内使用时，操作数类型必须与外层函数的返回类型匹配：

- 对 Result 值使用 ? 要求外层函数返回 Result。
- 对 Option 值使用 ? 要求外层函数返回 Option。

```
function safe_divide(a: int, b: int) -> Result<int, Error>:
  if b == 0:
    return Err(Error("division by zero"))
  return Ok(a // b)

function compute(a: int, b: int, c: int) -> Result<int, Error>:
  x = safe_divide(a, b)? # 若 b == 0 则提前返回 Err
  y = safe_divide(x, c)!!
  return Ok(y + 1)

function safe_get(xs: List<int>, i: int) -> Option<int>:
  if i < 0 or i >= xs.length():
    return none
  return Some(xs[i])

function first_plus_second(xs: List<int>) -> Option<int>:
  a = safe_get(xs, 0)? # 越界时提前返回 None
  b = safe_get(xs, 1)?
  return Some(a + b)
```

在顶层使用

? 和 !! 也可以直接在脚本的顶层使用。在那里，Err(e) 和 None 被视为致命错误：错误消息会写入 stderr，进程以状态 1 退出。

```
function mk() -> Result<int, Error>:
  return Err(Error("something broke"))
```

```
v = mk()?    # 向 stderr 输出 "error: something broke" 并以 1 退出
```

```
x: int? = none
```

```
y = x?      # 向 stderr 输出 "error: unexpected None" 并以 1 退出
```

在顶层，Result 的 Err 类型必须是 Error（这样它的 message 字段才能被打印）。

case: 条件表达式

```
x = case:
    condition => true_value
    _ => false_value
```

自上而下求值条件，返回第一个为真的分支表达式。所有结果表达式必须具有相同的类型。else => 为必需，因此该表达式总会产生一个值。

```
x = case:
    3 > 2 => 10
    _ => 20    # 10
```

```
s = case:
    false => "yes"
    _ => "no"  # "no"
```

嵌套三元运算可展平为多个分支

```
score = 85
y = case:
    score >= 90 => 3
    score >= 80 => 2
    _ => 1        # 2
```

范围运算符

.. 运算符创建包含两端的整数范围。

```
xs = 1 .. 5    # [1, 2, 3, 4, 5]
```

```
for i in 1 .. 3:
    print(i)    # 1 2 3
```

结果是包含从左操作数到右操作数（两端皆含）的所有整数的 List<int>。

空值合并运算符 (??)

```
x = optional_val ?? default_val
```

?? 运算符接受左侧的 Option<T> 或 Result<T, E> :

左侧	结果
Some(v)	v
None	default_val

左侧	结果
Ok(v)	v
Err(_)	default_val (错误值会被丢弃)

右操作数必须与 Option 的内部类型（或 Result 的 Ok 类型）相同。

```

a: int? = Some(10)
b: int? = none

print(a ?? 0)    # 10
print(b ?? 0)    # 0

function parse_int(s: str) -> Result<int, Error>:
    # ...

i = parse_int("42") ?? -1    # 成功时为 42, Err 时为 -1
j = parse_int("nope") ?? -1  # -1 — Err 值被丢弃

```

复合赋值运算符

更新变量的简写形式。x op= y 等价于 x = x op y。

运算符	等价表达式
x += y	x = x + y
x -= y	x = x - y
x *= y	x = x * y
x /= y	x = x / y
x %= y	x = x % y
x //= y	x = x // y
x **= y	x = x ** y
x &= y	x = x & y
x = y	x = x y
x ^= y	x = x ^ y
x <<= y	x = x << y
x >>= y	x = x >> y

```

x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 3    # x = 8
x &= 6    # x = 0

```

复合赋值允许用于任何 lvalue — 普通变量、列表或映射元素、记录字段以及任意嵌套链：

```

xs = [1, 2, 3]
xs[0] += 10    # 列表元素

record Point:
    x: int
    y: int
p = Point(1, 2)

```

```
p.x *= 5 # 记录字段

pts = [Point(1, 2), Point(3, 4)]
pts[0].x -= 1 # 链式：列表中记录字段
```

链式 LHS 上的每个索引表达式恰好被求值一次。对缺失的映射键进行复合赋值 (`m["absent"] += 1`) 是运行时错误。

递增 / 递减运算符

仅后缀、仅语句级别的运算符，用于将变量增减 1。内部分别被转换为 `x = x + 1` 和 `x = x - 1`。

运算符	等价表达式
<code>x++</code>	<code>x = x + 1</code>
<code>x--</code>	<code>x = x - 1</code>

```
count = 0
count++ # count = 1
count++ # count = 2
count-- # count = 1

f = 1.5
f++ # f = 2.5 (int 1 会提升为 float)
```

注意：`++` / `--` 只能作为语句使用，不能在表达式中使用。`@const` 变量不能使用递增/递减（不可变性会被强制执行）。

运算的类型规则

运算	左操作数类型	右操作数类型	结果类型
<code>+ - *</code>	int	int	int
<code>+ - *</code>	float	int / float	float
<code>+ - *</code>	int	float	float
<code>/</code>	任意数值	任意数值	float
<code>//</code>	int	int	int
<code>//</code>	float 或 int (其中一方为 float)	-	float
<code>**</code>	任意数值	任意数值	float
<code>%</code>	int	int	int
<code>%</code>	float 或 int (其中一方为 float)	-	float
<code>+</code>	str	str	str
<code>+</code>	str	int / float / bool	str
<code>+</code>	int / float / bool	str	str
<code>== != < <= > >=</code>	数值 / bool / str	同类型	bool
<code>*</code>	str	int	str
<code>in</code>	任意	Set / List / Map<K, V>	bool
<code>not in</code>	任意	Set / List / Map<K, V>	bool
<code>& \ ^ ~ << >> >>></code>	int	int	int
<code>and or not</code>	bool	bool	bool

运算符重载

可以为用户定义类型定义运算符的行为。

语法

```
# 二元运算符 (2 个参数)
function operator+(a: MyType, b: MyType) -> MyType:
    ...

# 一元运算符 (1 个参数)
function operator-(a: MyType) -> MyType:
    ...
```

可重载的运算符一览

类别	运算符
算术 (二元)	+ - * / % ** //
比较 (二元)	== != < <= > >=
位运算 (二元)	& \ ^ << >> >>>
逻辑 (二元)	and or
成员测试	in
下标	[] (读取)、[] = (写入)
调用	()
转换	as
一元	- ~ not
复合赋值	+= -= *= /= %= //= **= &= \ = ^= <<= >>=

返回类型约束

比较运算符和逻辑运算符必须返回 bool :

类别	运算符	必需返回类型
比较	== != < <= > >=	bool
逻辑	and or not	bool
成员测试	in	bool
转换	as	必需 (目标类型)

```
# OK
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

# Error: comparison operator '=' must return 'bool', but returns 'int'
function operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

算术运算符和位运算符没有返回类型约束。

二元 / 一元的区别

依参数个数区分。

```
# 二元 -
function operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)

# 一元 -
function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

复合赋值运算符重载

复合赋值运算符（+=、-= 等）可以独立重载。这使得大数据结构可以进行就地优化。

```
record Matrix:
    data: List
    rows: int
    cols: int

function operator+=(a: Matrix, b: Matrix) -> Matrix:
    for i in range(len(a.data)):
        a.data[i] = a.data[i] + b.data[i]
    return a
```

解析优先级 当 $x += y$ 被求值时：

1. 如果定义了 $\text{operator}+=$ $\rightarrow$ 直接调用
2. 如果未定义 $\text{operator}+=$ 但定义了 $\text{operator}+$ $\rightarrow$ 回退到 $x = x + y$
3. 如果都未定义（非内置类型） $\rightarrow$ 编译错误

```
record Vec2:
    x: float
    y: float

function operator+=(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)
```

```
v = Vec2(1.0, 2.0)
v += Vec2(3.0, 4.0) # 直接调用 operator+=
# v.x == 4.0, v.y == 6.0
```

复合赋值运算符要求恰好 2 个参数，没有返回类型约束。

下标运算符重载

$[]$ （读取）和 $[]=$ （写入）运算符可以为用户定义类型启用自定义下标行为。支持多索引访问（例如 $m[\text{row}, \text{col}]$ ）。

```
record Grid:
    a: int
    b: int
    c: int
    d: int

# 读取：需要 2+ 个参数（对象 + 索引）
function operator[](g: Grid, row: int, col: int) -> int:
    if row == 0 and col == 0:
        return g.a
    if row == 0 and col == 1:
        return g.b
    if row == 1 and col == 0:
        return g.c
    return g.d

# 写入：需要 3+ 个参数（对象 + 索引 + 值）
function operator[]=(g: Grid, row: int, col: int, value: int):
    ...
```



```
g = Grid(1, 2, 3, 4)
print(g[0, 1])    # 2
g[1, 0] = 99
```

优先尝试用户定义的下标运算符；如果没有匹配，则回退到内置下标行为（用于列表、映射和数组）。

成员测试运算符重载

`in` 运算符可以被重载以定义自定义成员测试。必须返回 `bool`。

```
record Range:
  lo: int
  hi: int

function operator in(value: int, r: Range) -> bool:
  return value >= r.lo and value < r.hi

r = Range(1, 10)
print(5 in r)      # true
print(15 not in r) # true
```

优先尝试用户定义的 `in` 运算符；如果没有匹配，则回退到内置行为（用于集合、映射和列表）。定义了 `in` 后，`not in` 自动支持。

调用运算符重载

() 运算符使记录可以作为可调用对象使用。至少需要 2 个参数（对象+ 参数）。

```
record Adder:
  base: int

function operator()(a: Adder, x: int) -> int:
  return a.base + x

add5 = Adder(5)
print(add5(10))    # 15
```

当持有记录值的变量被像函数一样调用时，编译器首先尝试 `operator()` 重载。如果没有匹配，其他调用解析策略（函数、构造函数、`lambda`）优先。

转换运算符重载

`as` 运算符可以被重载以定义自定义类型转换。接受恰好 1 个参数（源值），必须指定返回类型（目标类型）。分派按源类型和返回类型匹配。

```
record Celsius:
  value: int

record Fahrenheit:
  value: int

function operator as(c: Celsius) -> Fahrenheit:
  return Fahrenheit(c.value * 9 // 5 + 32)

c = Celsius(100)
f = c as Fahrenheit    # Fahrenheit(212)
```

目标类型可以是编译器能解析的任何类型，包括泛型类型：

```
record Temperature:
  value: int

function operator as(t: Temperature) -> int?:
  return Some(t.value)

t = Temperature(42)
result: int? = t as int?  # Some(42)
```

优先尝试用户定义的 `as` 运算符；如果没有匹配，则回退到内置转换（`int`、`float`、`bool`、`str` 等）。

[English](#) | [日本語](#) | [简体中文](#)

包参考

概述

Ry 使用包系统来组织代码。包可以是单个 `.ry` 文件，或是包含多个 `.ry` 文件的目录。使用 `from` 语句导入包。`std` 包（标准库）会自动导入到每个程序中。

导入语法

导入全部定义

```
from math
```

导入包内的所有函数与类型。

选择性导入

```
from math import sqrt
```

仅导入指定的定义。

多重选择性导入

```
from math import sqrt, PI
```

以逗号分隔选择性导入多个定义。

相对导入

```
from .helper import greet
```

从当前文件目录的相对位置导入模块。`.` 前缀将解析限制为仅当前目录（不搜索标准库和其他搜索路径）。

从子目录相对导入

```
from .utils import helper_fn
from .utils.calc import add
```

从当前文件目录的子目录导入。

从当前目录相对导入全部

```
from . import add, sub
```

从当前目录包中导入指定符号（目录中的所有 .ry 文件，排除 _ 前缀和 .test.ry 文件）。

包解析

以点号分隔的包名称按以下方式解析：

导入语句	解析结果
from math	math/ 目录（包）或 math.ry 文件
from utils.math	utils/math/ 目录或 utils/math.ry 文件
from str	str/ 目录或 str.ry 文件

解析顺序

对于每个搜索路径：1. 目录 ({path}/) — 若存在，加载目录内的所有 .ry 文件（包）2. 文件 ({path}.ry) — 单个文件（向后兼容）

目录包

当包解析为目录时：- 目录内的所有 .ry 文件会自动加载 - 以 _ 开头的文件会被排除 - 测试文件 (.test.ry) 会被排除 - 不需要特殊的入口文件（如 __init__.py）- 目录内文件中定义的所有函数与类型都会被导出

私有符号

名称以 _（下划线）开头的定义为包内部的私有符号，无法被导入：

- 通配导入（from pkg）会自动排除 _ 前缀的符号
- 具名导入（from pkg import _helper）会产生编译错误

```
# mylib/internal.ry
function _helper() -> int:      # 私有 — 无法导入
    return 42
function public_api() -> int:   # 公开 — 可导入
    return _helper()

mypackage/
  calc.ry      # function add(), function sub()
  string.ry    # function concat()

from mypackage      # 导入 add, sub, concat
from mypackage import add  # 仅导入 add
```

标准库 (std)

std 包会自动导入到每个程序中。提供的功能：- 内建函数（print、length、range 等）- 字符串函数（contains、find、replace 等）- 类型转换函数（to_int、to_float、to_str）- 集合函数（map、filter、sort 等）

子包

以下子包需要明确导入：

包	说明
<code>math</code>	数学常量与函数
<code>io</code>	文件 I/O、标准输入、字节转换
<code>path</code>	文件路径操作（ <code>join</code> 、 <code>basename</code> 、 <code>dirname</code> 等）

```
from math import sqrt, PI, sin
```

也可以直接从标准库的包中明确导入特定定义：

```
from str import contains
```

RY_HOME

标准库安装于 `$RY_HOME/share/std/`。RY_HOME 的默认值为 `~/.ry`。

```
export RY_HOME="$HOME/.ry" # default
```

RY_ENV

RY_ENV 环境变量控制运行时环境模式。也可以使用 `--env=<value>` CLI 标志。

值	别名	.env 加载	lib 搜索
prod	production	禁用	仓库构建的项目覆盖 → \$RY_HOME/share (回退至 lib) → exe/./share (回退至 lib) → exe/share (回退至 lib)
dev	development	.env.dev → .env	与 prod 相同
test	—	.env.test → .env	与 prod 相同
staging	—	.env.staging → .env	与 prod 相同
internal	—	.env.internal → .env	仓库构建的项目覆盖 → exe/./share (回退至 lib) → exe/share (回退至 lib) (跳过 \$RY_HOME)
(未设置) (默认)	—	仅 .env	与 prod 相同

别名会自动解析为规范形式。例如 `RY_ENV=production` 会被规范化为 `prod`。

在 `prod` 模式下，出于安全考虑不会加载 `.env` 文件——生产环境的机密信息应通过基础设施级别的环境变量管理（CI/CD、密钥管理等）。

其他模式下，先加载 `.env.<env>`（若存在），再加载 `.env`。由于不会覆盖已存在的环境变量，环境专属的值会优先生效。

简写形式（推荐）

```
RY_ENV=dev ./build/ry app.ry
```

完整名称（向后兼容）

```
RY_ENV=development ./build/ry app.ry
```

CLI 标志

```
./build/ry --env=dev test
```

prod 模式：不加载 .env

```
RY_ENV=prod ./build/ry app.ry
```

```
# 开发 Ry 自身时的额外隔离
RY_ENV=internal ./build/ry test
```

当在 Ry 源码树中构建 ry 可执行文件时，它可以使用项目 `package.toml` 中的仓库本地 `stdlib` 覆盖。这使得仓库构建与签出的 `share/std` 保持一致，即使 `~/ry/share/std` 版本较旧。已安装的 ry 二进制文件会忽略该覆盖，继续使用 `$RY_HOME/share/std`。

搜索路径优先级

1. 导入来源文件所在的目录
2. 使用仓库构建的 ry 时，来自当前 Ry 签出的仓库本地 `stdlib` 覆盖
3. `$RY_HOME/share`（标准库位置，对于旧版安装会回退至 `$RY_HOME/lib`）
4. 可执行文件相对的 `share/` 目录（对于旧版布局会回退至 `lib/`）
5. `RY_PATH` 环境变量中包含的路径（以冒号分隔）

RY_PATH 环境变量

在 `RY_PATH` 中以冒号分隔指定目录，即可添加到包搜索路径。

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

约束

约束	详细
可使用的位置	仅限顶层（函数或块内不可）
重复导入	自动跳过（不会产生错误）
循环导入	编译错误
相对导入	<code>from .</code> 和 <code>from .pkg</code> 仅在当前文件目录中解析
父目录导入	不支持 <code>from ..</code>
包名称	仅允许字母、数字和下划线（不允许连字符）

```
# 错误示例：在块内导入
function main():
    from math    # Error: imports only allowed at top level
```

```
# OK：多次导入相同包不会产生错误
from math
from math    # Skipped
```

创建包文件

单文件包

```
# calc.ry
function add(a: int, b: int) -> int:
    return a + b
```

```
function sub(a: int, b: int) -> int:
    return a - b

# main.ry
from calc import add, sub

print(add(1, 2))    # 3
print(sub(5, 3))    # 2
```

目录包

```
mylib/
  calc.ry
  string.ry

# main.ry
from mylib import add, concat
```

原生函数命名约定

实作为 C 执行时函数的标准库包函数遵循 `__ry_<package>_<function_name>` 约定。

注意：此约定适用于标准库包函数（例如 `base64`、`filesystem`、`path`）。内建函数（例如 `print`、`length`）和 `math` 函数使用不同的实作（内联 LLVM IR、`libc` 呼叫），不遵循此命名模式。

格式

`__ry_<package>_<function_name>`

规则

1. 前缀：`__ry_`
2. 包：包名称（例如 `from base64 import encode` 中的 `base64`）
3. 函数名称：在 Ry 中宣告的 `snake_case` 函数名称
4. 重载：当函数有多个不同 `arity` 的重载时，将参数数量附加为后缀（例如 `__ry_path_join2`、`__ry_path_join3`）
5. 错误获取器：每个返回 `Result` 类型的包都提供 `__ry_<pkg>_get_last_error`

范例

Ry 宣告	C 执行时函数名称
<code>base64::encode(data: str) -> str</code>	<code>__ry_base64_encode</code>
<code>filesystem::list_dir(path: str)</code> <code>-> Result<List<str>, Error></code>	<code>__ry_filesystem_list_dir</code>
<code>path::join(a: str, b: str) -> str</code>	<code>__ry_path_join2</code>
<code>path::join(a: str, b: str, c: str) -> str</code>	<code>__ry_path_join3</code>

[English](#) | [日本語](#) | [简体中文](#)

路径函数参考

文件路径操作。所有函数需要从 `path` 显式导入。

```
from path import join, basename, dirname, extension, resolve, is_absolute
```

函数一览

函数	签名	说明
join	(str, str) -> str	连接两个路径段
join	(str, str, str) -> str	连接三个路径段
join	(str, str, str, str) -> str	连接四个路径段
basename	(str) -> str	提取文件名部分
dirname	(str) -> str	提取目录部分
extension	(str) -> str	提取文件扩展名（包含点号）
resolve	(str) -> Result<str, Error>	将路径解析为绝对规范路径
is_absolute	(str) -> bool	返回路径是否为绝对路径

示例

连接路径

```
from path import join

p = join("/tmp", "data", "file.txt")
print(p)  # /tmp/data/file.txt

# 绝对路径的第二参数会替换第一参数
print(join("/tmp", "/usr"))  # /usr
```

提取路径组成部分

```
from path import basename, dirname, extension

p = "/home/user/docs/report.pdf"

print(basename(p))  # report.pdf
print(dirname(p))   # /home/user/docs
print(extension(p)) # .pdf
```

扩展名的边界情况

```
from path import extension

print(extension("archive.tar.gz"))  # .gz
print(extension(".gitignore"))      # (空字符串 — 没有扩展名的隐藏文件)
print(extension(".config.json"))    # .json
print(extension("Makefile"))        # (空字符串)
```

检查绝对路径

```
from path import is_absolute

print(is_absolute("/usr/local"))  # true
print(is_absolute("src/main.ry")) # false
```

解析路径

```
from path import resolve

case resolve("/tmp"):
```

```

Ok(p):
    print(p) # /private/tmp (on macOS) or /tmp
Err(e):
    print(e.message)

case resolve("/nonexistent"):
    Ok(p):
        print(p)
    Err(e):
        print(e.message) # cannot resolve path '/nonexistent': No such file or directory

```

[English](#) | [日本語](#) | [简体中文](#)

项目管理

CLI 概述

```

ry <file.ry> [args...]           # 运行 Ry 脚本
echo '<code>' | ry -c             # 从标准输入运行代码
ry test [options] [<file> | <dir>] # 运行测试
ry init                          # 初始化项目
ry new <project-name>            # 创建新项目
ry run [<script-name>]           # 运行项目脚本
ry fmt [options] [<file> | <dir>] # 格式化源文件
ry self-update [options]         # 更新 ry 本身

```

全局选项

选项	说明
-c	从标准输入读取并执行代码
-h, --help	显示帮助
-v, --version	显示版本
--env=<env>	设置环境 (production development internal)。覆盖 RY_ENV 环境变量。

入口点执行

当没有给出文件参数时，ry 在当前目录（或父目录）中查找 package.toml，并运行 entry 字段指定的文件：

```

ry           # 运行入口文件（例如 src/main.ry）
ry -- arg1 arg2 # 运行入口文件并传入参数

```

如果未找到 package.toml 或未设置 entry 字段，ry 会打印帮助信息并退出。

裸文件名

如果第一个参数是单一路径组件且其名称以 .ry 结尾（例如 main.ry），并且当前工作目录中不存在该名称的文件，ry 会按以下顺序在最近的 package.toml 项目中搜索：首先是项目根目录（包含 package.toml 的目录），然后是 [paths] 下列出的每个目录（按键名字母顺序排列；以 _ 开头的键被保留并在此搜索中忽略，_dev_stdlib 单独处理）。第一个存在的常规文件优先（例如，package.toml 旁的 foo.ry 优先于 src/foo.ry，当两者同时存在时）。如果都没有匹配，ry 会报告该文件不存在并列出尝试过的路径。不带 .ry 后缀的标记不会以这种方式解析（因此误输入的子命令仍会显示 “unknown command”）。

如果参数是包含多个组件的路径（例如 src/foo.ry 或 ./foo.ry），并且该路径不存在，ry 会报告文件不存在，而不是将其视为未知子命令。

当 `<file.ry>` 是基本文件名且不存在于当前目录中时，相同的规则同样适用于 `ry test <file.ry>`。

标准输入执行

使用 `-c` 标志从标准输入读取并执行代码：

```
echo 'print("hello")' | ry -c
```

ry init - 项目初始化

将当前目录初始化为 Ry 项目。

```
ry init
```

生成的文件与目录

```
my-project/
  package.toml      # 项目配置文件
  src/
    main.ry         # 入口点（示例代码）
```

行为

1. 若 `package.toml` 已存在则错误退出
 2. 创建 `src/` 目录（若不存在）
 3. 生成 `package.toml` (`name` 为当前目录名称)
 4. 生成 `src/main.ry`（若已存在则跳过）
-

ry new - 创建新项目

创建新目录并将其初始化为 Ry 项目。

```
ry new my-project
```

生成的文件与目录

```
my-project/
  package.toml      # 项目配置文件
  src/
    main.ry         # 入口点（示例代码）
```

行为

1. 若未指定项目名称则错误退出
 2. 若同名目录已存在则错误退出
 3. 创建 `<project-name>/` 目录
 4. 在其中创建 `src/` 目录
 5. 生成 `package.toml` (`name` 为指定的项目名称)
 6. 生成 `src/main.ry`
-

ry run - 运行项目脚本

执行 `package.toml` 的 `[scripts]` 部分中定义的脚本。

```
ry run          # 列出所有可用脚本
ry run build    # 运行 "build" 脚本
ry run test     # 运行 "test" 脚本
```

行为

1. 从当前目录向上搜索 `package.toml`
2. 不带参数时，列出所有可用脚本及其命令
3. 带脚本名称时，通过 `/bin/sh -c` 执行对应的 shell 命令
4. 所执行命令的退出码会被传播
5. 如果脚本名称未找到，显示错误并列出可用脚本

注意事项

- 不需要 LLVM 初始化（快速启动）
 - 命令在当前工作目录中执行
 - 由于命令通过 shell 运行，因此支持 shell 功能（管道、重定向等）
-

ry fmt - 代码格式化工具

以一致的 2 空格缩进和规范风格格式化 `.ry` 源代码文件。

```
ry fmt          # 格式化项目中所有 .ry 文件（需要 package.toml）
ry fmt src/main.ry # 格式化单个文件
ry fmt src/      # 递归格式化目录中所有 .ry 文件
ry fmt --check   # 检查文件是否已格式化（未格式化则 exit 1）
ry fmt --check src/ # 检查指定目录
```

格式化规则

- 每个代码块层级使用 2 空格缩进
- 二元运算符前后加空格（`a + b`，而非 `a+b`）
- 逗号后加空格（`f(a, b)`，而非 `f(a,b)`）
- 顶层定义之间（函数、记录、枚举）加空行
- 注释会被保留

行为

1. 读取源代码文件，解析为 AST，并以规范格式重新输出
2. 将格式化结果写回文件（就地修改）
3. 使用 `--check` 时仅报告未格式化的文件，若存在则以代码 1 退出（适用于 CI）
4. 递归格式化时跳过 `.git/`、`build/`、`node_modules/` 目录

注意事项

- 不需要 LLVM 初始化（快速启动）
 - 复合赋值运算符（`+=`、`-=` 等）因解析器会进行脱糖，格式化后会变成展开形式（`x = x + expr`）
 - 十六进制（`0xFF`）和二进制（`0b1010`）数字字面量会被转换为十进制表示
-

ry test - 运行测试

发现并运行测试文件 (*.test.ry)。完整的测试语法文档请参阅[测试](#)。

```
ry test                # 自动发现并运行所有 *.test.ry 文件
ry test tests/spec     # 运行目录下所有测试
ry test test_file.ry   # 运行指定的测试文件
ry test -p             # 并行运行测试
ry test -w             # 监视模式：文件变更时重新运行
ry test --coverage     # 收集行覆盖率信息
```

选项

选项	说明
-p, --parallel	并行运行测试
-w, --watch	监视变更并重新运行
--coverage, --cov	收集覆盖率信息
-h, --help	显示帮助

行为

1. 无参数时，搜索 package.toml 以找到项目根目录，并递归发现*.test.ry 文件（跳过.git、build、node_modules）
2. 所有测试通过时退出码为 0，有失败时为 1
3. --coverage 与 --parallel 一起使用时会回退到顺序执行

ry self-update - 自我更新

将 ry 本身更新至最新版本。从 GitHub Releases 下载二进制文件并替换当前的可执行文件。

```
ry self-update         # 更新至最新稳定版
ry self-update --nightly # 更新至最新 nightly 预发布版
ry self-update v0.0.1  # 更新至指定版本
```

行为

1. 显示当前版本
2. 根据参数解析更新目标版本
 - 无参数：GitHub 的最新稳定发行版 (/releases/latest)
 - --nightly：最新的预发布版
 - 指定版本：指定标签的发行版
3. 若与当前版本相同，则以 "Already up to date." 退出
4. 下载二进制文件并替换当前的可执行文件

安全性

发行版归档文件通过两个步骤进行验证：

1. 真实性：使用内嵌的 Ed25519 公钥验证 checksums.txt.sig 文件。
 - 如果签名文件不存在，除非设置了 RY_SKIP_SIGNATURE=1，否则更新将被中止。
 - 如果签名文件存在但无效，无论是否设置 RY_SKIP_SIGNATURE，更新都将被中止。
2. 完整性：将归档文件的 SHA-256 哈希值与 checksums.txt 进行对比。

若签名文件缺失且仍希望继续更新（不推荐），可以设置 RY_SKIP_SIGNATURE=1。但如果签名文件存在且签名无效，则无论该变量是否设置，更新都会被中止。

注意事项

- 执行需要 `curl` 和 `tar` 命令
- 若因权限不足导致二进制文件替换失败，会显示建议使用 `sudo` 的消息（不会自动执行 `sudo`）
- 下载会先在临时目录中进行；但若跨文件系统的 `cp` 回退操作被中断，目标二进制文件可能处于不完整状态

package.toml 配置文件

以 TOML 格式描述项目的元数据与路径设置。

```
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"

[paths]
src = "src"

[scripts]
build = "cmake --preset default && cmake --build build"
test = "./build/ry_tests"
clean = "rm -rf build"
```

[project] 部分

键	说明
name	项目名称（初始化时为目录名称）
version	版本字符串
entry	作为入口点的源代码文件

[paths] 部分

键	说明
src (其他键)	源代码目录 额外的项目相对目录。值不能是绝对路径，也不能包含 <code>..</code> 。这些目录与 <code>src</code> 一起，用于解析 <code>ry <file></code> 和 <code>ry test <file></code> 的裸文件名（参见上文的裸文件名）。
_dev_stdlib	可选；标准库位置的覆盖（参见工具文档）。不用于裸文件名解析。

[scripts] 部分

定义可通过 `ry run <name>` 执行的具名脚本。每个键是脚本名称，值是 `shell` 命令字符串。

键	说明
<name>	要执行的 <code>shell</code> 命令（通过 <code>ry run <name></code> 运行）

TOML 子集规范

package.toml 支持以下 TOML 子集。

- 部分头：[section]
- 键值对：key = "value"（仅字符串值）
- 注释：从 # 到行尾
- 空行会被忽略

[English](#) | [日本語](#) | [简体中文](#)

正则表达式参考

正则表达式字面量语法

正则表达式字面量使用 /pattern/ 语法，产生 Regex 类型的值：

```
from regex import is_match, split, replace
```

```
# 正则表达式字面量支持基于类型的重载
"hello".is_match(/[a-z]+)/          # true
"a1b2c".split(/[0-9]/)              # ["a", "b", "c"]
"abc123".replace(/[0-9]+/, "X")    # "abcX"
```

正则表达式字面量可以存储在变量中：

```
pat = /[a-z]+/
"hello".is_match(pat)  # true
```

正则表达式字面量中的 / 可以用 \/ 转义：

```
"a/b".is_match(/a\/b/)  # true
```

除法与正则表达式的区分

词法分析器使用上下文来区分正则表达式字面量和除法运算：

- 在产生值的标记（标识符、数字、字符串字面量、) 或]）之后，/ 被解析为除法
- 在运算符、关键字或期望表达式的定界符（(、[、,、=）之后，/ 开始一个正则表达式字面量

```
x = 10 / 2          # 除法：5
y = is_match("a", /a/) # 正则表达式字面量
```

函数一览

正则表达式字面量函数（文本在前，兼容 UFCS）

这些函数接受 Regex 类型的模式，使用文本在前的参数顺序以支持 UFCS：

函数	签名	说明
is_match	(str, Regex) -> bool	返回整个文本是否匹配模式
search	(str, Regex) -> int	返回第一个匹配的起始位置（未找到时返回 -1）
replace	(str, Regex, str) -> str	将所有匹配替换为替换字符串
split	(str, Regex) -> List<str>	按模式匹配进行分割
find_all	(str, Regex) -> List<str>	返回所有不重叠的匹配结果

```
from regex import is_match, search, replace, split, find_all
```

```
# 直接调用
print(is_match("hello", /[a-z]+/))      # true

# UFCS (text.function(pattern))
print("abc123".search(/[0-9]+/))        # 3
print("abc123".replace(/[0-9]+/, "X"))  # abcX
parts = "hello world".split(/\s+/)
nums = "a1b2c3".find_all(/[0-9]/)
```

遗留函数（文本在前）

原始的 `regex_*` 函数仍然可用以保持向后兼容。它们接受模式字符串（非正则表达式字面量），使用文本在前的参数顺序，与正则表达式字面量 API 一致：

函数	签名	说明
<code>regex_match</code>	<code>(text: str, pattern: str) -> bool</code>	返回整个文本是否匹配模式
<code>regex_search</code>	<code>(text: str, pattern: str) -> int</code>	返回第一个匹配的起始位置（未找到时返回 -1）
<code>regex_replace</code>	<code>(text: str, pattern: str, replacement: str) -> str</code>	将所有匹配替换为替换字符串
<code>regex_split</code>	<code>(text: str, pattern: str) -> List<str></code>	按模式匹配进行分割
<code>regex_find_all</code>	<code>(text: str, pattern: str) -> List<str></code>	返回所有不重叠的匹配结果

```
print(regex_match("hello", "[a-z]+"))    # true
pos = regex_search("abc123", "[0-9]+")  # 3
```

支持的模式语法

语法	说明	示例
<code>abc</code>	字面字符	<code>"hello"</code>
<code>.</code>	任意一个字符（换行除外）	<code>"a.c"</code> 匹配 <code>"abc"</code> 、 <code>"aXc"</code>
<code> </code>	选择	<code>"cat dog"</code> 匹配 <code>"cat"</code> 或 <code>"dog"</code>
<code>*</code>	零次或多次	<code>"a*"</code> 匹配 <code>"</code> 、 <code>"a"</code> 、 <code>"aaa"</code>
<code>+</code>	一次或多次	<code>"a+"</code> 匹配 <code>"a"</code> 、 <code>"aaa"</code>
<code>?</code>	零次或一次	<code>"a?"</code> 匹配 <code>"</code> 或 <code>"a"</code>
<code>{n}</code>	恰好 <code>n</code> 次	<code>"a{3}"</code> 匹配 <code>"aaa"</code>
<code>{n,m}</code>	<code>n</code> 到 <code>m</code> 次	<code>"a{2,4}"</code> 匹配 <code>"aa"</code> 到 <code>"aaaa"</code>
<code>{n,}</code>	至少 <code>n</code> 次	<code>"a{2,}"</code> 匹配 <code>"aa"</code> 、 <code>"aaa"</code> 、 <code>...</code>
<code>*?</code>	零次或多次（非贪婪）	<code>".*?"</code> 匹配最短
<code>+?</code>	一次或多次（非贪婪）	<code>".+?"</code> 匹配最短
<code>??</code>	零次或一次（非贪婪）	<code>"a??"</code> 优先零次
<code>{n,m}?</code>	范围（非贪婪）	<code>"a{2,4}?"</code> 优先 <code>n</code> 次
<code>(...)</code>	分组	<code>"(ab)+"</code> 匹配 <code>"abab"</code>
<code>[abc]</code>	字符类	<code>"[aeiou]"</code> 匹配元音字母
<code>[a-z]</code>	字符范围	<code>"[a-z]+"</code> 匹配小写单词
<code>[^...]</code>	否定字符类	<code>"[^0-9]"</code> 匹配非数字
<code>^</code>	字符串开头锚点	<code>"^hello"</code>
<code>\$</code>	字符串结尾锚点	<code>"world\$"</code>
<code>\d</code>	数字 <code>[0-9]</code>	<code>"\d+"</code> 匹配数字

语法	说明	示例
\D	非数字 [^0-9]	
\w	单词字符 [a-zA-Z0-9_]	"\w+" 匹配标识符
\W	非单词字符	
\s	空白字符	"\s+" 匹配空格与 Tab
\S	非空白字符	
\b	单词边界	"\bword\b" 匹配完整单词
\B	非单词边界	"\Bword" 匹配单词内部
(?i)	忽略大小写标志	"(?i)hello" 匹配"HELLO"
\.	转义特殊字符	"\." 匹配字面的 .

使用示例

范围量词

```
print(regex_match("123-4567", "\\d{3}-\\d{4}")) # true
print(regex_match("aaa", "a{2,4}"))           # true
print(regex_match("ababab", "(ab){2,}"))       # true
```

非贪婪（最短）匹配

```
# 贪婪：匹配最长
g = regex_replace("\"a\" and \"b\"", "\".*\"", "X")
print(g) # X
```

```
# 非贪婪：匹配最短
l = regex_replace("\"a\" and \"b\"", "\".*?\"", "X")
print(l) # X and X
```

```
# 查找各个 HTML 标签
tags = regex_find_all("<a> <bb> <ccc>", "<.*?>")
print(length(tags)) # 3
```

注意：非贪婪匹配控制整体匹配长度。在不支持提取括号分组的情况下，greedy/lazy 混合模式可能与 PCRE 引擎的行为不同。

单词边界

```
# 匹配完整单词
pos = regex_search("hello world", "\\bworld\\b")
print(pos) # 6

# 查找所有单词
words = regex_find_all("hello world foo", "\\b\\w+\\b")
print(length(words)) # 3
```

忽略大小写匹配

```
# (?i) 放在模式开头即可忽略大小写
print(regex_match("HELLO", "(?i)hello")) # true
print(regex_match("Hello", "(?i)hello")) # true
```

注意：(?i) 必须出现在模式的开头，并适用于整个模式。不支持部分忽略大小写匹配（例如 (?i:sub)pattern）。

[English](#) | [日本語](#) | [简体中文](#)

结构体参考

概述

结构体是栈上的值类型。使用 `record` 关键字定义。结构体可以使用 `invariant` 子句实现契约式设计。参阅[契约式设计](#)。

命名约定：结构体名称必须使用 PascalCase（如 `Point`、`Rectangle`）。字段名称必须使用 `snake_case`。编译器会强制执行这些约定。

定义语法

```
record TypeName:
    field_name: type
    field_name: type
```

示例

```
record Point:
    x: int
    y: int

record Rectangle:
    width: float
    height: float
```

构造函数

按照字段定义顺序传递参数。不支持命名参数。

```
p = Point(10, 20)
r = Rectangle(3.0, 4.5)
```

字段访问

使用点号记法读取字段。

```
p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

字段赋值

变量声明	字段赋值
可变（无 <code>@const</code> ）	可以
<code>@const</code>	编译错误

```
p = Point(10, 20)
p.x = 100    # OK: 可变量
```



```
@const
q = Point(10, 20)
q.x = 100    # 错误: @const 变量的字段不可变更
```

链式与深层字段赋值

字段赋值可组合应用于嵌套记录、集合元素和复合运算符。左侧可以是任何根植于可变变量的后缀链。

```
record Inner:
  val: int

record Outer:
  inner: Inner
  tag: str

o = Outer(Inner(1), "t")
o.inner.val = 42          # 深层字段写入
o.inner.val += 1          # 复合形式
print(o.inner.val)        # 43

pts = [Point(1, 2), Point(3, 4)]
pts[0].x = 99             # 列表中记录字段更新
pts[0].x *= 2
print(pts[0].x)           # 198
```

请参阅 [collections](#) → [链式索引与字段赋值](#) 了解完整矩阵以及记录内嵌套集合的别名注意事项。

作为函数参数与返回值使用

```
function distance(p: Point) -> float:
  return (p.x * p.x + p.y * p.y) as float

function make_point(x: int, y: int) -> Point:
  return Point(x, y)
```

嵌套结构体

可以将结构体作为另一个结构体的字段使用。

```
record Point:
  x: int
  y: int

record Circle:
  center: Point
  radius: float

c = Circle(Point(0, 0), 1.0)
print(c.center.x)    # 0
```

比较 (== / !=)

记录类型自动支持 == 和 != 运算符。比较是逐字段进行的（结构性相等）。

```
record Point:
  x: int
  y: int

p1 = Point(10, 20)
p2 = Point(10, 20)
p3 = Point(30, 40)

print(p1 == p2)  # true
print(p1 != p3)  # true
```

- 所有字段按顺序比较。对于 ==，所有字段必须相等。对于 !=，至少一个字段必须不同。
 - 嵌套记录会递归比较。
 - 如果提供了用户定义的 operator== 或 operator!=，则优先于自动生成的版本。
-

记录子类型（继承）

记录支持使用 < 语法的单继承。子记录继承父记录的所有字段。

语法

```
record ChildName < ParentName:
  child_field: type
```

示例

```
record HttpError < Error:
  status: int
  url: str
```

字段继承

- 子记录在其布局的开头继承所有父字段。
- 构造函数先接受父字段，然后是子特有的字段。

```
err = HttpError("not found", 404, 404, "/api")
print(err.message)  # "not found" (从 Error 继承)
print(err.status)   # 404 (自有字段)
```

子类型强制转换

子值可以传递给期望父类型的地方。子值会被自动切片以提取父前缀字段（值类型切片）。

```
function handle(e: Error) -> str:
  return e.message

err = HttpError("fail", 500, 500, "/api")
handle(err)  # OK — HttpError 强制转换为 Error
```

深层继承

记录可以形成继承链。每一层继承所有祖先字段。

```
record DetailedHttpError < HttpError:
    detail: str

# 构造函数: Error 字段 + HttpError 字段 + 自有字段
derr = DetailedHttpError("fail", 500, 500, "/x", "server crash")
handle(derr) # OK — 强制转换为 Error (祖父类型)
```

规则

规则	详细
仅支持单继承	record A < B: — 只能有一个父类
深层继承	record C < B: 其中 record B < A: — 允许
名称冲突	子字段与父字段同名 -> 编译错误
自动 == / to_str	包含所有继承字段
不变量继承	构造或修改子记录时检查父的 invariant: 子句
子类型强制转换	适用于: 函数参数、返回值、Err()、字段赋值、? 运算符
泛型边界	<T: RecordName> 将类型参数约束为记录的子类型
@const	适用于所有字段 (包括继承的字段)

约束与错误

约束	详细
相同字段名重复	编译错误
@const 变量的字段赋值	编译错误

```
# 错误示例: 相同字段名重复
record Bad:
    x: int
    x: int # 错误
```

枚举类型 (enum)

概述

枚举类型是具名常量的集合。默认以 i64 整数 (0, 1, 2, ...) 的连续编号表示。也可以指定显式的整数值。

定义语法

```
enum TypeName:
    VariantName
    VariantName
    ...
```

示例

```
enum Color:
    Red
    Green
    Blue
```

变体访问

使用 `::` 运算符访问变体。

```
c = Color::Red
print(c)    # Red
```

比较

enum 值为整数，因此可以直接使用 `==` / `!=` 进行比较。

```
print(Color::Red == Color::Red)    # true
print(Color::Red != Color::Green)  # true
```

在 if 语句中使用

```
c = Color::Green
case:
  c == Color::Red:
    print("red")
  c == Color::Green:
    print("green")
  -:
    print("blue")
```

函数参数

类型名称使用 enum 名称。

```
function is_red(c: Color) -> bool:
  return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

print

使用 `print()` 会输出变体名称。

```
c = Color::Blue
print(c)    # Blue
```

显式值指定

simple enum 的变体可以指定显式的整数值。适用于 HTTP 状态码或位掩码模式等用途。

```
enum HttpStatus:
  Ok = 200
  NotFound = 404
  InternalError = 500

s = HttpStatus::NotFound
print(s)                    # NotFound
print(s == HttpStatus::NotFound)  # true

enum Permission:
  Read = 1
  Write = 2
  Execute = 4
```

规则：- 仅支持 simple enum（不含关联数据的 ADT 变体）。- 值必须为整数字面量（允许负值）。- 若任一变体有显式值，则所有变体都必须有（不可混用自动和手动）。- 重复值会产生编译错误。- `print()` 显示变体名称，而非整数值。

ADT 变体的命名字段

ADT 变体字段可以选择性地包含名称以用于文档目的。命名字段使定义具有自描述性，而不改变构造或模式匹配的语义。

```
enum Shape:
  Circle(radius: float)
  Rect(width: float, height: float)
  Point
```

- 构造始终是位置性的：`Shape::Circle(3.14)`，而非 `Shape::Circle(radius: 3.14)`。
- 模式匹配绑定用户选择的变量名：`case Shape::Circle(r):`。
- 字段名必须为 `snake_case`。不允许在单个变体内混用命名和未命名字段。
- 未命名语法 (`Circle(float)`) 仍然有效。

约束与错误

约束	详细
变体访问为 <code>EnumName::VariantName</code>	必须使用 <code>::</code> 运算符
变体值	默认为自动分配 (0, 1, 2, ...)，可使用 <code>= value</code> 显式指定
比较为整数比较	可使用 <code>==</code> 、 <code>!=</code>
命名字段名	必须为 <code>snake_case</code> ；同一变体内不可重复；不可混用命名/未命名

[English](#) | [日本語](#) | [简体中文](#)

测试功能

Ry 内建 RSpec 风格的测试语法。使用 `ry test` 子命令执行测试文件。

运行测试

```
ry test           # 自动发现并运行项目内所有 *.test.ry 文件
ry test tests/spec # 递归运行指定目录下所有 *.test.ry 文件
ry test test_file.ry # 运行指定的测试文件
ry test -p         # 并行运行所有测试 (-p 或 --parallel)
ry test -p tests/  # 并行运行指定目录的测试
ry test -w         # 监视模式：文件变更时自动重新运行测试 (-w 或 --watch)
ry test -w -p      # 监视模式 + 并行运行
ry test -w tests/  # 监视指定目录
ry test --coverage # 所有测试 + 行覆盖率摘要
ry test --cov      # --coverage 的简写
ry test --outline  # 打印 describe/it 结构而不运行测试
```

退出码为 0 表示所有测试通过，1 表示有测试失败。

自动发现模式

不带参数运行 `ry test` 时：

1. 搜索 `package.toml` 以找到项目根目录
 2. 在项目根目录下递归发现所有 `*.test.ry` 文件（`.git`、`build`、`node_modules` 会被跳过）
 3. 逐一运行并汇总结果
-

语法

函数式语法（推荐）

使用 `@it` 和 `@describe` 指令将测试用例定义为普通的具名函数：

```
@it("test case name")
function test_add():
    expect(1 + 2).to_eq(3)
```

使用 `@describe` 对相关的测试进行分组：

```
@describe("Arithmetic")
function arithmetic_tests():
    @it("should add integers")
    function test_add():
        expect(1 + 2).to_eq(3)

    @it("should subtract integers")
    function test_sub():
        expect(5 - 3).to_eq(2)
```

- 函数名用于代码导航和符号标识
- 描述字符串（在指令中）用于测试输出和报告
- `@it` 函数除非与 `@each` 或 `@property` 组合，否则不能有参数
- `@it` 和 `@describe` 仅在 `ry test` 下可用

共享设置 `@describe` 函数主体中声明的变量会被内部 `@it` 函数自动捕获：

```
@describe("User validation")
function user_validation_tests():
    min_length = 8
    max_length = 64

    @it("should reject short passwords")
    function test_short():
        expect(min_length).to_be_greater_than(0)

    @it("should accept passwords within length limits")
    function test_range():
        expect(max_length).to_be_greater_than(min_length)
```

嵌套 `@describe` `@describe` 函数可以嵌套以创建多层分组。输出会缩进以反映嵌套深度：

```
@describe("API")
function api_tests():
    @describe("GET /users")
    function get_users_tests():
        @it("should return 200 OK")
```

```
function test_ok():
  expect(true).to_be_true()
```

输出：

```
API
GET /users
+ should return 200 OK
```

Lambda 语法（已弃用）

已弃用：describe() 和 it() lambda 调用语法已弃用。请改用具名函数上的 @describe 和 @it 指令。lambda 语法将在未来版本中移除。

迁移：

Lambda 语法	指令语法
it("name", (): ...)	@it("name") function name(): ...
describe("name", (): ...)	@describe("name") function name(): ...

```
describe("description", ():
  it("test case name", ():
    # test body
    expect(actual_value).to_eq(expected_value)
  )
)
```

- describe 和 it 接受描述字符串和 lambda 参数 ()：作为第二个参数
- describe 块内可以编写 it 块及其他语句（如变量声明等）
- 每个 it 块为独立的测试用例
- describe / expect 仅在 ry test 中可用（在普通的 ry 执行中会产生编译错误）

尾随块语法

任何函数调用（describe/it/mock 除外）都可以使用尾随块语法。在 () 后加上：会将缩进块作为无参数 lambda 传入最后的参数位置：

```
# 以下两者等价：
foo("arg"):
  bar()
```

```
foo("arg", ():
  bar()
)
```

expect / 匹配器

匹配器	说明	支持类型
to_eq(expected)	相等比较	int, float, bool, str
to_not_eq(expected)	不相等	int, float, bool, str
to_be_true()	为 true	bool
to_be_false()	为 false	bool
to_be_none()	为 None	Option
to_be_some()	Option 为 Some	Option
to_be_ok()	Result 为 Ok	Result

匹配器	说明	支持类型
<code>to_be_err()</code>	Result 为 Err	Result
<code>to_contain(val)</code>	容器包含值	List, Set, Map, str
<code>to_not_contain(val)</code>	容器不包含值	List, Set, Map, str
<code>to_be_greater_than(v)</code>	<code>actual > v</code>	int, float
<code>to_be_less_than(v)</code>	<code>actual < v</code>	int, float
<code>to_be_greater_than_or_eq(v)</code>	<code>actual >= v</code>	int, float
<code>to_be_less_than_or_eq(v)</code>	<code>actual <= v</code>	int, float
<code>to_have_length(n)</code>	长度为 n	List, Set, Map, str
<code>to_be_empty()</code>	长度为 0	List, Set, Map, str
<code>to_start_with(prefix)</code>	字符串以 prefix 开头	str
<code>to_end_with(suffix)</code>	字符串以 suffix 结尾	str

fail

立即将当前测试标记为失败。

```
it("should not reach here", ():
    fail("unexpected error")
)
```

- `fail()` — 使用通用消息标记测试失败
- `fail(msg)` — 使用自定义消息标记测试失败
- `fail()` 之后执行继续进行（不会中止测试）
- 仅在 `ry test` 模式下可用

输出格式

```
Calculator
+ adds numbers
+ subtracts
- fails test (red)
  line 10: expected 3, got 2
```

2 passed, 1 failed

- + 为成功（绿色），- 为失败（红色）
- 失败时显示行号与预期值/实际值

示例

```
describe("Arithmetic", ():
    it("adds integers", ():
        expect(1 + 2).to_eq(3)
    )
    it("compares strings", ():
        expect("hello").to_eq("hello")
    )
    it("checks booleans", ():
        expect(3 > 1).to_be_true()
```



```

    )
  )
  describe("Booleans", ():
    it("false check", ():
      expect(1 > 2).to_be_false()
    )
  )
)

```

模拟 (Mock)

mock(fn_name, replacement)

在当前 it 块中将函数替换为模拟实现。it 块结束时模拟会自动清除。

```

function fetch_data() -> str:
  return "real data"

describe("mocking", ():
  it("replaces function", ():
    mock(fetch_data, () => "fake")
    expect(fetch_data()).to_eq("fake")
  )
  it("auto-restores", ():
    expect(fetch_data()).to_eq("real data")
  )
)

```

- 第一个参数为函数名称（标识符，非字符串）
- 第二个参数为替换用 lambda
- 替换函数必须与原始函数具有相同的参数类型和返回类型
- 原始函数上的 require 和 ensure 契约在调用模拟时仍然生效
- it 块结束时模拟会自动恢复

verify(fn_name)

返回模拟函数被调用的次数 (int)。

```

describe("verify", ():
  it("counts calls", ():
    mock(fetch_data, () => "fake")
    fetch_data()
    fetch_data()
    expect(verify(fetch_data)).to_eq(2)
  )
)

```

限制

- 不支持重载函数的模拟
 - 不支持使用捕获闭包进行模拟（仅支持纯 lambda）
 - 不支持 @native function 的模拟
-

参数化测试 (@each)

@each 可以用多组参数运行同一个测试。

函数式语法 (推荐) :

```
@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
])
@it("should add {0} + {1} = {2}")
function test_add(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
```

Lambda 语法 :

```
@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
])
it("should add {0} + {1} = {2}", (a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)
```

- 列表必须包含与参数数量匹配的元组
- 描述中的 {0}、{1}、... 会被替换为参数值
- 每个元组生成一个独立的测试用例
- 支持的参数类型 : int、float、bool、str

基于属性的测试 (@property)

@property 生成随机输入并多次运行测试。

函数式语法 (推荐) :

```
@property(count=100)
@it("should verify addition is commutative")
function test_commutative(a: int, b: int):
    expect(a + b).to_eq(b + a)
```

Lambda 语法 :

```
@property(count=100)
it("should verify addition is commutative", (a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

- count=N 指定随机试验次数 (默认 : 100)
- 失败时会显示反例 (导致失败的输入值)
- 在第一次失败时停止测试
- 支持的参数类型 : int ([-1000, 1000])、float ([-1000.0, 1000.0])、bool、str (随机 ASCII、0-20 字符)

测试覆盖率

使用 --coverage (或 --cov) 标志来测量行覆盖率 :

```

ry test --coverage # 所有测试 + 覆盖率摘要
ry test --cov tests/spec/math.test.ry # 单个文件
ry test --coverage tests/spec/ # 目录

```

输出

```

Test Coverage Summary:
  tests/spec/math.test.ry    100.0% (74/74 lines)
  tests/spec/strings.test.ry 92.3%  (24/26 lines)
-----
Total                        95.1%  (98/100 lines)

```

- 仅报告用户代码；标准库文件会被排除
- `--coverage` 与 `--parallel` 同时指定时，会退回为顺序执行

测试大纲

使用 `--outline` 可以显示测试文件的 `describe/it` 结构而不执行任何测试体：

```
ry test --outline tests/spec/mock.test.ry
```

输出：

```

describe mock
  it replaces function
  it auto-restores after it block
  it with arguments
describe verify
  it counts calls
  it zero calls

```

- 适用于单个文件、目录和 `-p`（所有测试文件）
- `@each` 参数化测试显示格式模板并带有 `(@each)` 后缀
- `@property` 测试显示标签并带有 `(@property)` 后缀

测试描述风格

`it` 描述应以 `should` 开头，使其在测试输出中读起来像完整的句子：

```

it should add integers
it should reject invalid input
it should return error when file is missing

```

推荐：

描述	备注
"should add integers"	动词原形
"should reject short passwords"	动词原形
"should return error for missing file"	动词原形
"should add {0} + {1} = {2}"	参数化：动词原形
"should verify addition is commutative"	基于属性

避免：

描述	原因
"adds integers"	第三人称动词，读为 “it adds” 时显得别扭
"integer addition"	名词短语，不是句子
"handles error"	第三人称动词

describe 块使用名词或主题短语（例如"Arithmetic"、"List"、"GET /users"）—它们不需要 should。

限制

- 不支持嵌套 describe (lambda 语法)；请使用基于函数的@describe 指令语法进行嵌套分组
- 不支持 before_each / after_each

[English](#) | [日本語](#) | [简体中文](#)

线程参考

thread 包提供操作系统级别的线程原语，用于 CPU 密集型并行工作负载和细粒度的并发控制。

类型

类型	说明
Thread	OS 线程的不透明句柄
Lock	互斥锁的不透明句柄
RWLock	读写锁的不透明句柄
Semaphore	计数信号量的不透明句柄
Barrier	线程屏障的不透明句柄
AtomicInt	原子 64 位整数的不透明句柄
AtomicBool	原子布尔值的不透明句柄

所有类型都是由 ARC（自动引用计数）管理的不透明指针。使用相应的*_new() 函数创建实例。当不再被引用时，资源会自动清理——显式调用*_free() 是可选的，但支持用于立即清理。

导入

```
from thread import thread_spawn, thread_join, lock_new, lock_acquire, lock_release
```

Thread

函数	签名	说明
thread_spawn	(body: function() -> T) -> Thread	创建并启动一个执行 body 的新 OS 线程。捕获的变量按值复制。T 可以是 Unit、int、float 或 bool；请参阅下方的限制章节。
thread_join	(thread: Thread) -> Result<T, Error>	等待线程完成并以 Ok(value) 返回工作线程的值。对已 join 的 Thread 再次 join 会返回 Err("thread already joined")。

示例 — 副作用工作线程 (Unit)

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add

counter = atomic_int_new(0)
t = thread_spawn():
    atomic_int_add(counter, 1)
)
thread_join(t)
print(atomic_int_load(counter)) # 1
```

示例 — 返回值的工作线程 (int / float / bool)

```
from thread import thread_spawn, thread_join

t = thread_spawn() => 42
case thread_join(t):
    Ok(v):
        print(v) # 42
    Err(e):
        print(e.message)
```

捕获也适用于返回值的工作线程：

```
x = 10
t2 = thread_spawn() => x * x
case thread_join(t2):
    Ok(v):
        print(v) # 100
    Err(_):
        print("error")
```

注意：捕获的变量会被复制到线程的环境中。对于基本类型（int、float、bool、str），这会产生一个独立的副本。对于不透明句柄类型（Lock、AtomicInt 等），指针被复制，共享底层资源——这是同步原语的预期行为。

限制 (MVP)

- **返回类型**。工作线程可以返回 Unit、int、float 或 bool。ARC 管理的类型（str、List、Map、Set、记录）和 sum 类型（Option、Result、enum）尚不支持；传递此类工作线程会产生指向后续 issue 的代码生成错误。
- **Lambda 主体形式**。仅表达式主体的 lambda（() => <expr>）可以返回值。块主体的 lambda 仍可用于 Unit 工作线程，但目前无法携带非 Unit 返回值。
- **变量引用工作线程**。thread_spawn(my_fn) 仍将 my_fn 视为 Unit 工作线程；目前读取其返回值需要使用内联 lambda 形式。
- **Panic**。工作线程内的运行时 panic（例如除以零、数组越界、契约违反）会终止整个进程。它目前不会作为 Err 从 thread_join 返回 — 这需要单独重构 Ry 的 panic 机制，作为后续 issue 跟踪。

Lock (互斥锁)

函数	签名	说明
lock_new	() -> Lock	创建一个新的互斥锁。
lock_acquire	(lock: Lock) -> Result<Unit, Error>	获取锁。阻塞直到可用。
lock_release	(lock: Lock) -> Result<Unit, Error>	释放锁。

函数	签名	说明
<code>lock_free</code>	<code>(lock: Lock) -> Unit</code>	立即释放锁。可选——ARC 会自动处理清理。

RWLock（读写锁）

函数	签名	说明
<code>rwlock_new</code>	<code>() -> RWLock</code>	创建一个新的读写锁。
<code>rwlock_read_lock</code>	<code>(rwlock: RWLock) -> Result<Unit, Error></code>	获取共享读锁。允许多个读者同时持有。
<code>rwlock_write_lock</code>	<code>(rwlock: RWLock) -> Result<Unit, Error></code>	获取独占写锁。阻塞直到所有读者和写者释放。
<code>rwlock_unlock</code>	<code>(rwlock: RWLock) -> Result<Unit, Error></code>	释放锁（共享或独占）。
<code>rwlock_free</code>	<code>(rwlock: RWLock) -> Unit</code>	立即释放读写锁。可选——ARC 会自动处理清理。

Semaphore

函数	签名	说明
<code>semaphore_new</code>	<code>(count: int) -> Result<Semaphore, Error></code>	创建具有给定初始计数的信号量。如果计数为负则返回 <code>Err</code> 。
<code>semaphore_acquire</code>	<code>(sem: Semaphore) -> Result<Unit, Error></code>	递减信号量。如果计数为零则阻塞。
<code>semaphore_release</code>	<code>(sem: Semaphore) -> Result<Unit, Error></code>	递增信号量并唤醒一个等待的线程。
<code>semaphore_free</code>	<code>(sem: Semaphore) -> Unit</code>	立即释放信号量。可选——ARC 会自动处理清理。

Barrier

函数	签名	说明
<code>barrier_new</code>	<code>(count: int) -> Result<Barrier, Error></code>	创建同步 <code>count</code> 个线程的屏障。如果计数不为正则返回 <code>Err</code> 。
<code>barrier_wait</code>	<code>(barrier: Barrier) -> Result<Unit, Error></code>	阻塞直到所有 <code>count</code> 个线程都调用了 <code>barrier_wait</code> 。
<code>barrier_free</code>	<code>(barrier: Barrier) -> Unit</code>	立即释放屏障。可选——ARC 会自动处理清理。

AtomicInt

函数	签名	说明
<code>atomic_int_new</code>	<code>(value: int) -> AtomicInt</code>	创建具有给定初始值的原子整数。
<code>atomic_int_load</code>	<code>(atomic: AtomicInt) -> int</code>	原子地读取值。
<code>atomic_int_store</code>	<code>(atomic: AtomicInt, value: int) -> Unit</code>	原子地写入值。

函数	签名	说明
<code>atomic_int_add</code>	<code>(atomic: AtomicInt, delta: int) -> int</code>	原子地加上 <code>delta</code> 并返回之前的值。
<code>atomic_int_sub</code>	<code>(atomic: AtomicInt, delta: int) -> int</code>	原子地减去 <code>delta</code> 并返回之前的值。
<code>atomic_int_cas</code>	<code>(atomic: AtomicInt, expected: int, desired: int) -> bool</code>	比较并交换：如果当前值等于 <code>expected</code> ，则设置为 <code>desired</code> 并返回 <code>true</code> ；否则返回 <code>false</code> 。
<code>atomic_int_free</code>	<code>(atomic: AtomicInt) -> Unit</code>	立即释放原子整数。可选——ARC 会自动处理清理。

AtomicBool

函数	签名	说明
<code>atomic_bool_new</code>	<code>(value: bool) -> AtomicBool</code>	创建具有给定初始值的原子布尔值。
<code>atomic_bool_load</code>	<code>(atomic: AtomicBool) -> bool</code>	原子地读取值。
<code>atomic_bool_store</code>	<code>(atomic: AtomicBool, value: bool) -> Unit</code>	原子地写入值。
<code>atomic_bool_free</code>	<code>(atomic: AtomicBool) -> Unit</code>	立即释放原子布尔值。可选——ARC 会自动处理清理。

与 `async/await` 的比较

特性	<code>async/await</code>	thread 包
执行模型	工作窃取线程池	专用 OS 线程
适用场景	I/O 密集型任务、大量轻量级任务	CPU 密集型任务、细粒度控制
同步方式	通过 <code>await</code> 自动同步	通过 <code>Lock</code> 、 <code>Semaphore</code> 等手动同步
开销	低（任务调度）	较高（OS 线程创建）

[English](#) | [日本語](#) | [简体中文](#)

类型参考

类型一览

类型	内部表示	字面值示例	说明
<code>int</code>	<code>i64</code>	<code>42</code> , <code>-7</code> , <code>0xFF</code> , <code>0b1010</code> , <code>100_000</code>	64 位有符号整数
<code>u8</code>	<code>i8</code>	(无专用字面值)	无符号 8 位整数 (0-255)。通过类型标注 <code>b: u8 = 42</code> 使用
<code>float</code>	<code>f64</code>	<code>3.14</code> , <code>0.5</code> , <code>3.14_159</code> , <code>1e10</code> , <code>1.5e-3</code> , <code>2.5E+2</code>	64 位浮点数（支持科学记数法）
<code>bool</code>	<code>!l</code>	<code>true</code> , <code>false</code>	布尔值
<code>str</code>	<code>ptr</code>	<code>"hello"</code> , <code>""</code> , <code>"a\nb"</code>	字符串（堆上的不可变字节序列）

类型	内部表示	字面值示例	说明
Unit	void	(无返回值)	无返回值函数的返回类型。必须用 <code>-> Unit</code> 显式指定可能存在值的类型
Option<T> (T1, T2, ...)	{ i1, T } LLVM StructType (literal)	Some(42), None (1, 3.14)	元组类型
List<T>	ptr (堆)	[1, 2, 3]	动态数组
Map<K, V>	ptr (堆)	{"a": 1}	哈希映射
Set<T>	ptr (堆)	{1, 2, 3}	不重复的集合
function(T1, T2) -> R	ptr (函数指针)	(x: int) => x * 2	函数类型
用户定义类型	LLVM StructType (named)	record Point: ...	以 record 关键字定义的结构体
enum	i64 / 标签联合	Color::Red, Shape::Circle(3.14)	以 enum 关键字定义的枚举类型 (支持关联数据)
Error	{ ptr, i64 }	Error("msg"), Error("msg", 404)	内置错误类型
Type	{ i64, ptr }	type_of(42)	由 type_of 返回的编译时类型标识。请参阅 Type
any	{ i64, [8 x i8] }	x: any = 42	可持有任意基本值的标签联合
T1 \ T2	{ i64, [N x i8] }	int \ str	union 类型 (可持有多种类型之一)
Int 字面量	i64	42, 0 \ 1	int 字面量类型 (值约束)
String 字面量	ptr	"N" \ "S"	str 字面量类型 (值约束)
范围	i64	1..12, -10..10	范围类型 (包含两端的整数范围约束)
i8	i8	x: i8 = 42, x = 42i8	8 位有符号整数 (低级, 无隐式转换)
i16	i16	x: i16 = 100, x = 100i16	16 位有符号整数 (低级, 无隐式转换)
i32	i32	x: i32 = 42, x = 42i32	32 位有符号整数 (低级, 无隐式转换)
i64	i64	x: i64 = 100, x = 100i64	64 位有符号整数 (低级, 无隐式转换)
u8	i8	x: u8 = 200, x = 200u8	8 位无符号整数 (低级, 无隐式转换)
u16	i16	x: u16 = 60000, x = 60000u16	16 位无符号整数 (低级, 无隐式转换)
u32	i32	x: u32 = 4294967295, x = 100u32	32 位无符号整数 (低级, 无隐式转换)
u64	i64	x: u64 = 18446744073709551615, x = 0xFFFFFFFFFFFFFFFFFu64	最大 $2^{64} - 1$ 的 64 位无符号整数 (低级, 无隐式转换)
f32	float	x: f32 = 3.14, x = 1e10f32	32 位浮点数 (低级, 无隐式转换)
weak T	ptr (header)	weak s	ARC 管理值的弱引用 (不阻止释放)
Regex	ptr	/[a-z]+/, /\d{3}/	正则表达式模式 (通过正则字面量语法创建)
Result<T, E>	{ i1, T/E }	Ok(42), Err(Error("fail"))	表示成功 (Ok) 或失败 (Err) 的类型

类型	内部表示	字面值示例	说明
Task<T>	ptr	(由 async 函数返回)	异步任务句柄 (与 await 和 block_on 配合使用)
Iterator<T>	ptr	(由 iter() 创建)	用于顺序元素访问的惰性迭代器
T[N]	[N x T]	buf: i32[8]	固定长度连续数组。低级类型 T 的 N 个元素 (栈分配)

类型标注语法

声明变量时可以显式指定类型。当类型可推断时可以省略。

```
x: int = 42
b: u8 = 255
f: float = 3.14
s: str = "hello"
b: bool = true
opt: Option<int> = Some(10)
t: (int, float) = (1, 3.14)
xs: List<int> = [1, 2, 3]
m: Map<str, int> = {"a": 1}
s: Set<int> = {1, 2, 3}
fn_val: function(int) -> int = (x: int) => x * 2
rx: Regex = /[0-9]+/
u: int | str = 42
a: any = 42
```

可用类型名称一览

类型名称	备注
int	内置标量类型
u8	内置标量类型 (无符号 0-255)
float	内置标量类型
bool	内置标量类型
str	内置字符串类型
Unit	无返回值函数的返回类型
Option<T>	泛型类型 (T 为任意类型)
(T1, T2, ...)	元组类型 (元素数量和类型组合任意)
List<T>	泛型动态数组类型
Map<K, V>	泛型哈希映射类型
Set<T>	泛型集合类型
function(T1, ...) -> R	函数类型
Error	内置错误类型 (message: str、code: int)
any	可持有任意基本值 (int, float, bool, str) 或 Unit 的内置类型。支持隐式转换: 具体值赋值给 any 时自动包装, any 值赋值给具体类型时自动解包 (带运行时类型检查)。支持 any(int) → float 的自动提升。详见 any 类型
T1 \ T2 \ ...	union 类型 (以 \ 分隔的多个类型之一)
i8	低级 8 位有符号整数 (无隐式转换)
i16	低级 16 位有符号整数 (无隐式转换)

类型名称	备注
i32	低级 32 位有符号整数（无隐式转换）
i64	低级 64 位有符号整数（无隐式转换）
u8	低级 8 位无符号整数（无隐式转换）
u16	低级 16 位无符号整数（无隐式转换）
u32	低级 32 位无符号整数（无隐式转换）
u64	低级 64 位无符号整数（无隐式转换）
f32	低级 32 位浮点数（无隐式转换）
T[N]	低级类型 T 的 N 个元素的固定长度数组。栈分配，连续内存。支持索引读写和 length()
用户定义类型名称	以 record 或 enum 关键字声明的类型

类型别名

type 关键字为现有类型创建新的名称。别名与原始类型完全互换。

```
type Meters = float
type StringList = List<str>
```

```
d: Meters = 3.14
names: StringList = ["Alice", "Bob"]
```

命名约定：类型别名名称必须使用 PascalCase（如 Meters、StringList）。编译器会强制执行此约定。

类型别名也可以用于函数类型、字面量类型和范围类型：

```
type Callback = function(int, int) -> int

add: Callback = function(a: int, b: int) => a + b
print(add(3, 4))    # 7

type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

m: Month = 6
d: Direction = "N"
n: Digit = 5
```

类型别名也可以以 union 类型（包括基本和用户定义类型）作为目标，且别名的行为与内联 union 完全相同：

```
type Simple = int | str | bool

x: Simple = 42
y: Simple = "hello"
z: Simple = true

function describe(v: Simple) -> str:
    return to_str(v)
```

union 组件本身是别名的嵌套别名会被透明地展平，且重复成员会被去重。以下三种形式是等价的：

```
type A = int | str
type B = A | bool           # 与 `int | str | bool` 相同
type C = B | int            # 与 `int | str | bool` 相同（int 被去重）

x: B = 42
```

```
y: B = "hello"
z: B = true
```

数值字面量

整数字面量

接受十进制、十六进制 (0x/OX) 和二进制 (0b/0B) 形式。下划线允许在数字之间作为视觉分隔符 (1_000_000、0xFFFF_FFFF)。

接受的大小由目标类型决定：

目标	范围
裸 int / i64	-9_223_372_036_854_775_808 .. 9_223_372_036_854_775_807 (i64)
i8 / i16 / i32	对应的有符号范围
u8 / u16 / u32	0 .. 2 ^N - 1
u64	0 .. 18_446_744_073_709_551_615 (2 ⁶⁴ - 1)

大型无符号字面量需要后缀(18446744073709551615u64)或接收变量上的类型注解(x: u64 = 18446744073709551615)。负字面量以一元负号作用于非负数大小，因此-1i8 被接受，而 -1u8 被拒绝。

```
max_u64: u64 = 18446744073709551615      # 2^64 - 1
mask:    u64 = 0xFFFF_FFFF_FFFF_FFFF    # 通过 hex 的相同值
word:    u32 = 4294967295                 # 2^32 - 1
```

浮点字面量

```
FloatLiteral := DecDigits '.' DecDigits Exponent? FloatSuffix?
               | DecDigits Exponent FloatSuffix?
Exponent      := ('e' | 'E') ('+' | '-')? DecDigits
FloatSuffix   := 'f32' | 'f64'
```

任何期望浮点的地方都支持科学记数法：

```
avogadro = 6.022e23
planck   = 6.626e-34
light_spd = 2.998E8
big      = 1e10f32
```

溢出指数会产生+Inf/-Inf (不是编译错误)。请注意，运行时 to_float() 转换器更严格：它在溢出时返回 Err(Error) 而不是产生 +Inf。

字面量类型

字面量类型将变量的值限制为特定的常量值。对于常量值，在编译时进行约束检查；对于动态值，在运行时进行约束检查。

int 字面量类型

```
x: 42 = 42      # 单一字面量类型
y: 0 | 1 = 0     # int 字面量的 union
z: 0 | 1 = 0
```

```

z = 1                # OK
# z = 2              # 编译错误（常量）或运行时错误（动态值）

```

str 字面量类型

```

dir: "N" | "S" | "E" | "W" = "N"
# @const bad: "N" | "S" = "X"    # 编译错误

```

约束检查

- 编译时：当赋值为常量（ConstantInt 或字符串字面量）时，在编译时检查，违反时产生编译错误。
- 运行时：当值为动态（如函数返回值）时，在运行时检查，违反时程序以错误退出。

范围类型

范围类型将整数变量的值限制在连续的范围内（包含两端）。

```

month: 1..12 = 6      # OK
# @const bad: 1..12 = 0    # 编译错误：超出范围
# @const bad: 1..12 = 13   # 编译错误：超出范围

t: -10..10 = -5       # 支持负数范围

```

使用可变变量重新赋值（运行时检查）

```

x: 1..12 = 6
x = 12                # OK
# x = dynamic_value() # 运行时检查：超出范围则错误退出

```

在函数参数中使用

```

function set_month(m: 1..12) -> int:
    return m

set_month(6)          # OK
# set_month(13)       # 编译错误（常量参数）

```

none 关键字与 Option 类型简写

none 关键字表示 Option 类型的值不存在，等同于 None。

T? 语法是 Option<T> 的简写。

```

x: int? = 42          # 等同于 Option<int>
y: int? = none         # 等同于 None

function find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none

```

弱引用 (weak T)

weak 引用是对 ARC 管理值的非持有引用。与强引用不同，弱引用不会递增强引用计数。当最后一个强引用被释放时，被引用的对象会被释放——所有存活的弱引用会自动变为 None。

弱引用是用户层面打破引用循环的机制。

创建弱引用

在类型标注和表达式位置都使用 weak 关键字：

```
s = "hello"
w: weak str = weak s
```

类型 weak T 是一个新的类型构造器，其中 T 必须是 ARC 管理的类型（目前为 str、List<T>、Map<K, V>、Set<T>）。

访问弱引用（升级）

访问弱变量会自动执行升级——对强引用计数进行原子检查和递增。结果始终为 Option<T>：

- 如果被引用的对象仍然存活（strong count > 0），则为 Some(value)
- 如果被引用的对象已被释放（strong count == 0），则为 None

```
s = "alive"
w: weak str = weak s
case w:
  Some(v):
    print(v)           # "alive"
  None:
    print("deallocated")
```

合并运算符 (??) 也可以与弱引用配合使用：

```
w: weak str = weak s
val = w ?? "default"
```

重新赋值

弱引用可以重新赋值。旧的弱引用被释放，新的被保留：

```
a = "first"
b = "second"
w: weak str = weak a
w = weak b
```

线程安全

升级操作在内部使用比较并交换（CAS）循环，因此跨线程使用是安全的。这是必要的，因为强引用可能被并发释放。

作用域清理

弱引用在超出作用域时自动释放。如果强引用计数和弱引用计数都达到零，则 ARC 头部会被释放。

F-String（字符串插值）

使用 f"..." 语法进行字符串插值。{} 内的表达式会被求值并转换为字符串。

```
name = "world"
print(f"Hello {name}")           # Hello world

a = 1
b = 2
print(f"{a} + {b} = {a + b}")    # 1 + 2 = 3
```

插值中支持的类型

{} 内可使用求值结果为 int、float、bool、str、record 类型、元组或集合类型（List、Map、Set）的任意表达式。

```
xs = [1, 2, 3]
print(f"items: {xs}")            # items: [1, 2, 3]

t = (1, "hello")
print(f"tuple: {t}")             # tuple: (1, hello)
```

转义序列

序列	输出
{{	{（字面大括号）
}}	}（字面大括号）
\n \r \t \\ \"	与普通字符串相同

```
print(f"{{braces}}")            # {braces}
```

类型转换（as）

使用 as 关键字进行显式的类型转换。

```
x = 42 as float                  # 42.0
y = 3.14 as int                  # 3
z = 1 as bool                    # true
s = 42 as str                    # "42"
b = 255 as u8                    # u8 值 255
```

支持的转换

来源	目标	行为
int	float	SIToFP
float	int	截断（FPToSI）
int	bool	0 -> false、非零 -> true
bool	int	false -> 0、true -> 1
int / float / bool	str	字符串表示
int	u8	截断（低 8 位）
u8	int	零扩展

int | i8 / i16 / i32 / i64 | 截断（或 i64 时为恒等）|
i8 / i16 / i32 / i64 | int | 符号扩展（SExt）|

`int` | `u8` / `u16` / `u32` / `u64` | 截断 (或 `u64` 时为恒等) |
`u8` / `u16` / `u32` / `u64` | `int` | 零扩展 (`ZExt`) |
 有符号 | 有符号 (更宽) | 符号扩展 (`SExt`) |
 有符号 | 有符号 (更窄) | 截断 |
 无符号 | 无符号/有符号 (更宽) | 零扩展 (`ZExt`) |
 无符号 | 无符号/有符号 (更窄) | 截断 |
 有符号 / 无符号整数 | `float` | `SIToFP` / `UIToFP` 然后 `f64` |
`float` | 有符号 / 无符号整数 | `FPToSI` / `FPToUI` |
`float` | `f32` | `FPTrunc` |
`f32` | `float` | `FPEExt` |
 有符号整数 | `f32` | `SIToFP` |
 无符号整数 | `f32` | `UIToFP` |
`f32` | 有符号 / 无符号整数 | `FPToSI` / `FPToUI` |

as 转换的目标类型支持完整的类型语法，包括泛型类型：

```
x = value as Option<int>
y = data as Map<str, int>
```

任何 as 转换 (包括泛型) 必须是内置转换或具有匹配的用户定义 operator as，否则为编译错误。字符串转数值请使用 `to_int()` / `to_float()`。

带关联数据的 enum (ADT)

在变体名称后面加上括号并指定类型，enum 变体就可以携带关联数据。不带括号的变体仍然是单纯的标签。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

命名字段

变体可以选择性地使用命名字段以提高文档清晰度。命名字段使变体定义具有自描述性，但不改变运行时行为——构造和模式匹配仍然是位置性的。

```
enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point
```

规则：- 字段名必须为 `snake_case`。- 在单个变体内，所有字段必须全部命名或全部未命名 (不可混用)。- 同一变体内的重复字段名是编译错误。

构造函数

使用 `EnumName::Variant(value)` 语法构建带有数据的变体。参数始终是位置性的，即使字段有名称也是如此。

```
c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point
```

带绑定的模式匹配

使用 `case EnumName::Variant(binding):` 形式取出关联数据。绑定使用用户选择的变量名，而非字段名。

```
case c:
    Shape::Circle(r):
        print(r)           # 3.14
```

```
Shape::Rectangle(w, h):
    print(w)
    print(h)
Shape::Point:
    print("point")
```

内部表示

ADT enum 以标签联合的形式存储：`{ i64 tag, [N x i8] data }`，`N` 的大小足以容纳最大变体的载荷。

泛型 enum

enum 可以使用角括号语法 `<T>` 带有类型参数，使相同的 enum 结构可以持有不同类型的载荷。

```
enum MyOption<T>:
    MySome(T)
    MyNone
```

使用方式

提供具体的类型参数来实例化。当编译器无法推断类型时，需要显式指定类型参数。

```
a = MyOption<int>::MySome(42)
b = MyOption<int>::MyNone
```

```
case a:
    MyOption::MySome(v):
        print(v)          # 42
    MyOption::MyNone:
        print("none")
```

Error 类型

用于错误处理的内置类型。Error 具有两个字段：`message` (str) 和 `code` (int)。

```
e = Error("something went wrong")      # code 默认为 0
e2 = Error("not found", 404)           # 显式指定 code
```

```
print(e.message)    # something went wrong
print(e2.code)       # 404
print(e2)            # Error: not found (code: 404)
```

使用 Result 进行错误处理

可能失败的函数返回 `Result<V, E>`：

```
function divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

case divide(10, 2):
    Ok(v):
        print(v)          # 5
```



```
Err(e):
    print(e.message)
```

当返回值没有意义时，使用 `Result<Unit, Error>`：

```
function save(path: str, data: str) -> Result<Unit, Error>:
    return Ok(0 as u8)    # Unit 占位符

case save("/tmp/test.txt", "hello"):
    Ok(_):
        print("saved")
    Err(e):
        print(e.message)
```

Result 类型

`Result<V, E>` 是一个内置的参数化类型，有两个构造函数：

- `Ok(value)` — 成功变体
- `Err(error)` — 错误变体

与 `case` 配合使用进行穷举的错误处理。`Ok` 和 `Err` 两种情况都必须覆盖（或使用 `_` 通配符）。

相等性：`Result<T, E>` 支持 `==` 和 `!=`。当两个 `result` 的变体匹配（`Ok/Ok` 或 `Err/Err`）且内部值相等时，它们相等。

```
function make_ok(v: int) -> Result<int, Error>: return Ok(v)
make_ok(42) == make_ok(42)    # true
make_ok(1)  == make_ok(2)     # false
make_ok(1)  != Err(Error("e")) # true
```

测试匹配器：`- expect(x).to_be_ok()` — 断言结果为 `Ok` - `expect(x).to_be_err()` — 断言结果为 `Err`

内部表示

`Error` 以 `{ ptr message, i64 code }` 表示。`Result<V, E>` 以 `{ i1 isOk, V okValue, E errValue }` 表示。

Type

`Type` 是内置 `type_of` 函数返回的值。它表示类型的编译时标识，并允许在运行时进行反射比较。

```
print(to_str(type_of(42)))    # int
print(to_str(type_of([1, 2, 3]))) # List

print(type_of(42) == type_of(100)) # true
print(type_of(42) == type_of(3.14)) # false
```

关键属性：

- 每个不同的类型定义（基本类型、集合、`record`、`enum`、`Option`、`Result`、`function`、`Type` 本身等）在编译时都会获得唯一的标识。
- `Type` 值上的 `==` / `!=` 比较的是标识，而不是显示名称。两个不同的 `record`（或同名的 `record` 与 `enum`）始终可区分。
- `print` 和 `to_str` 显示人类可读的类型名称（例如 `"int"`、`"List"`、`"Point"`、`"i32"`）。
- 低级数值类型（`i8`、`i16`、`...`、`f32`）与 `int` / `float` 区分。
- 集合泛型折叠为它们的基础名称：`type_of([1, 2])` 返回 `"List"`，而不是 `"List<int>"`。
- `Type` 是反射性的：`type_of(type_of(x))` 返回表示 `Type` 本身的 `Type` 值。

内部表示

Type 表示为 { i64 id, ptr name }。id 字段用于相等性，name 字段用于显示。两个字段都在编译时由 type_of 填充。

union 类型

可以使用 | 声明可能持有多种类型的变量。

```
x: int | str = 42
x = "hello"      # 可重新赋值 (union 中的任一类型)
print(x)         # hello
```

在函数参数与返回值中的使用

```
function show(x: int | str) -> int:
    print(x)
    return 0

function get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"
```

内部表示

union 类型以 { i64 tag, [N x i8] data } 表示。tag 表示各组成类型的索引（按字母顺序排序后），data 是最大组成类型大小的字节数组。

相等性

union 类型目前支持基本变体 (int、float、str、bool) 的 == 和 !=。当两个 union 值持有相同变体（相同 tag）且内部值相等时，它们相等。

```
x: int | str = 42
y: int | str = 42
x == y      # true

z: int | str = "42"
x == z      # false (不同 tag: int vs str)
```

约束

- 赋值不属于 union 的类型会产生编译错误
- int | str 和 str | int 是相同的类型（会被规范化）
- 使用 print() 输出 union 值时，会根据运行时的 tag 以适当的类型显示
- == 和 != 支持基本变体 (int、float、str、bool)；不支持 closure 变体

any 类型

any 类型是一种内置的动态类型，可以持有任意基本值。它采用类似 Python 的灵活类型方式——当不需要静态类型保证时，any 让您无需使用泛型或 union 类型即可处理多种类型。

支持的类型

any 可以持有以下类型：

类型	标签	说明
int	0	64 位有符号整数
float	1	64 位浮点数
bool	2	布尔值
str	3	字符串
Unit	4	Unit 值（用于无返回值的函数）

any 无法持有集合类型（List、Map、Set）、资源类型（TcpListener、TcpStream 等）、函数指针或用户定义类型（record、enum）。

内部表示

any 以标签联合实现：

```
{ i64 tag, [8 x i8] data } // 共 16 字节
```

tag 字段标识存储的类型，data 字段持有值（最多 8 字节）。

包装与解包

具体类型的值在赋值给 any 时自动包装，any 的值在赋值给具体类型时自动解包。

```
# 包装：具体类型 → any
x: any = 42           # int 被包装成 any
x = "hello"          # 可以重新赋值为不同类型
```

```
# 解包：any → 具体类型
function get_value() -> any:
    return 42
n: int = get_value() # any(int) 被解包为 int
```

```
# 解包时的 int → float 自动提升
f: float = get_value() # any(int) 被解包并提升为 float
```

如果运行时类型与目标类型不符（例如将 any(str) 解包至 int 变量），会产生运行时错误。

重新赋值

any 变量可以重新赋值为任何可持有类型的值：

```
x: any = 42
x = 3.14      # OK: 现在持有 float
x = "hello"   # OK: 现在持有 str
x = true      # OK: 现在持有 bool
```

算术运算

当两个操作数都是 any 时，运算会在运行时根据实际类型进行分派：

运算	类型	结果
+	int + int	int
+	float + float	float
+	int + float	float
+	str + str	str（拼接）
-	数值	int 或 float

运算	类型	结果
*	数值	int 或 float
*	str * int / int * str	str (重复)
/	数值	float (总是)
//	int // int	int
//	含 float	float
%	数值	int 或 float
**	数值	float (总是)
一元 -	int	int
一元 -	float	float

当一个操作数是 any、另一个是具体类型时，具体值会在运算前自动包装。

```
x: any = 10
y: any = x + 20    # 20 被自动包装；结果是 any(int) = 30
```

不兼容的类型组合（例如 str - int）会导致运行时错误。

比较运算

运算	行为
==、!=	相同类型之间有效；int/float 混合比较可行
<、<=、>、>=	数值（int/float 混合可）和字符串（字典序）

```
x: any = 3
y: any = 3.0
print(x == y)    # true (int/float 比较)
```

比较时类型不符（例如 int < str）会导致运行时错误。

字符串转换

any 值支持 print() 和 f-string 插值：

```
x: any = 42
print(x)          # 42
print(f"value: {x}") # value: 42
```

转换规则：int → 十进制字符串、float → %g 格式、bool → "true"/"false"、str → 原样、Unit → "Unit"。

将 any 传递给有类型的函数

any 值可以传递给具有具体参数类型的函数。值会通过运行时类型检查自动解包：

```
function add_one(x: int) -> int:
    return x + 1

v: any = 42
result = add_one(v)    # any(int) 被解包为 int；结果是 43
```

类型规则（运算时的类型转换）

运算	左操作数	右操作数	结果类型	备注
+ - *	int	int	int	低级类型：原生宽度的无符号运算，无隐式提升 隐式 float 提升
+ - *	u8	u8	u8	
+ - *	float 或 int	float 或 int（其中一方为 float）	float	
/	任意数值	任意数值	float	始终为 float 向下取整除法（向-∞）；int 操作数结果为 int，含 float 则结果为 float 使用 libm pow
//	任意数值	任意数值	int 或 float	
**	任意数值	任意数值	float	
%	int	int	int	
%	float 或 int	float 或 int（其中一方为 float）	float	
+	str	str	str	字符串拼接 字典序比较
== != < <= > >=	str	str	bool	
== != < <= > >=	数值或 bool	数值或 bool	bool	
in	任意	Set	bool	元素是否包含在集合中
& \ ^ ~ << >>	int	int	int	对 float 会产生错误
+ - *	i32	i32	i32	低级类型：无隐式转换，需要相同类型
/ //	i32	i32	i32	有符号整数除法 (SDiv)
/ //	u32	u32	u32	无符号整数除法 (UDiv)
%	i32	i32	i32	有符号取余 (SRem)
%	u32	u32	u32	无符号取余 (URem)
+ - * /	f32	f32	f32	
== !=	i32/u32	i32/u32	bool	符号无关的相等比较
< <= > >=	i32	i32	bool	有符号比较 (ICMP_SLT 等)
< <= > >=	u32	u32	bool	无符号比较 (ICMP_ULT 等)
>>	i32	i32	i32	算术右移（保留符号位）
>>	u32	u32	u32	逻辑右移（零填充）
**	低级	任意	错误	低级类型不支持幂运算符
混合	低级	不同	错误	混合低级和高级类型是编译错误

转义序列（str 字面值内）

序列	含义
\n	换行
\r	回车
\t	制表符
\\	反斜杠

序列	含义
\"	双引号
\0	空字符

类型安全约束

- 隐式拓宽转换 — 函数调用中支持安全的拓宽转换：`u8 → int`、`u8 → float`、`int → float`。对于二元运算符，混合 `int` 和 `float` 会触发 `float` 提升。`u8` 是低级类型，使用原生宽度的无符号运算；在二元运算符中混合 `u8` 与 `int` 是编译错误。窄化转换（例如 `float → int`）不允许隐式进行。从 `int` 字面量到 `u8` 的窄化转换仅在带有类型标注 `b`：`u8 = 42` 时允许。
- 变量类型在声明时固定 — 一旦以 `int` 声明的变量，就无法重新赋值为 `float`。
- 位运算仅限 `int` — 对 `float` 或 `bool` 使用位运算会产生编译错误。
- 非 `bool` 类型也可用于条件式 — `if` 的条件式可使用 `int` (`0 = false`、非 `0 = true`) 等 `bool` 以外的类型。
- 数值字面量分隔符 — 下划线可作为数值字面量中的视觉分隔符：`100_000` `0xFF_FF` `0b1010_0101` `3.14_159`。下划线必须出现在数字之间（不允许前导、尾随或连续的下划线）。
- 数值字面量后缀 — 低级类型可通过字面量后缀指定：`42i32`、`255u8`、`3.14f32`、`.5f32`、`0xFFu8`、`0b1010u8`。带有 `float` 后缀的整数字面量（`42f32`）会产生浮点值。带有整数后缀的浮点字面量（`3.14i32`）是编译错误。超出范围的值（例如 `256u8`、`129i8`）也是编译错误。
- 低级数值类型（`i8`、`i16`、`i32`、`i64`、`u8`、`u16`、`u32`、`u64`、`f32`）无隐式转换 — 混合低级类型之间或低级与高级类型（`int`、`float`）之间会产生编译错误。请使用显式 `as` 转换。低级整数的 `/` 运算符执行整数除法（类似 `Rust`），而非浮点除法。有符号类型使用 `SDiv/SRem`，无符号类型使用 `UDiv/URem`。
- 有符号与无符号 — 有符号类型（`i8`、`i16`、`i32`、`i64`）使用有符号比较（`ICMP_SLT` 等）和算术右移（`AShr`）。无符号类型（`u8`、`u16`、`u32`、`u64`）使用无符号比较（`ICMP_ULT` 等）和逻辑右移（`LShr`）。`>>>` 运算符无论符号性如何，始终执行逻辑移位。
- `int` 算术溢出是运行时错误 — 高级 `int` 类型的算术运算（`+`、`-`、`*`、一元`-`）在溢出时产生运行时错误，类似 `Swift` 的默认行为。这防止了补码回绕导致的静默数据损坏。溢出的常量表达式在编译时被捕获。
- 低级整数溢出会回绕 — 低级整数类型的算术运算在溢出时使用 `Ry` 定义的补码回绕（有符号）或模运算（无符号）。例如，`2147483647i32 + 1i32` 会回绕为 `-2147483648`。如需显式溢出控制，请使用 `checked_add/sub/mul`（返回 `Result<T, Error>`）、`saturating_add/sub/mul`（钳制到类型边界）或 `wrapping_add/sub/mul`（自文档化的回绕行为）。参见[函数参考](#)。